



Eval Harness 源码内核

从一个样本到一次发布决定，读懂评测系统如何运行

完整中文离线版 · 源码课程、工程机制、案例与实验

原创文档：CC BY 4.0 原创代码：MIT

目录

目录	2
Eval Harness 源码内核	5
从这里开始：第一次把 Eval 跑成完整证据链	6
Eval Harness 源码内核：完整目录	8
学习路线	11
01 Agent Harness 与 Eval Harness：先分清“执行”与“评测”	12
02 Task、Dataset、Target 与 Environment：把问题定义完整	16
03 Sample、Trial 与 Attempt：别让重试改写统计分母	20
04 Trace、Artifact 与 Observation：让分数能回到证据	24
05 Scorer、Judge、Score 与 Metric：先判单次，再描述总体	27
06 不确定性、比较与 Gate：从分数到可执行决定	30
07 Eval-to-RL 与独立 Release Eval：改进闭环不能吃掉验收集	33
lm-evaluation-harness 源码课程：Task 怎样变成批量模型请求	37
01 入口与 Task 加载：simple_evaluate 实际装配了什么	40
02 请求执行：Instance 怎样分桶、批量调用并回填	43
03 评分、聚合与测试：从 process_results 到 Group 结果	46
Inspect AI 源码课程：把 Solver、Sandbox、Scorer 和 EvalLog 放进同一条链	49
01 Eval、Task 与 Solver：从公共 API 到可执行 Plan	52
02 Sandbox 与 Sample：一次 Agent Eval 怎样真正运行	55
03 Scorer、EvalLog 与 Retry：怎样保留可重评分的运行事实	58
OpenAI Evals 源码课程：Registry 怎样驱动 Eval 与事件记录	61
01 Registry 与 EvalSpec：字符串怎样变成真实对象	64

02 CompletionFn 与 Sample: 谁负责调用, 谁负责判断	66
03 Recorder 与 Metric 边界: 事件很多, 推断仍要另建合同	69
Promptfoo 源码课程: 配置矩阵怎样变成可判定的评测运行	72
Promptfoo 配置与 Provider: 先解析身份, 再生成请求	75
Promptfoo 测试运行时: 展开、串并行与结果落盘	78
Promptfoo 断言与 CI: 从响应判分到门禁还差哪一步	81
DeepEval 源码课程: 从测试用例到 MetricData 的评测生命周期	84
DeepEval 数据对象: Golden 何时变成 LLMTestCase	87
DeepEval Metric: 有状态评分器怎样产出可解释结果	90
DeepEval 执行策略: 异步、缓存与错误不能改变统计单位	93
Harbor 与 Terminal-Bench 1: Agent 终端任务怎样成为可核对的 Trial	96
Job、Dataset 与 Trial: 运行计划怎样固定统计单位	99
Environment 与 Agent 生命周期: 副作用怎样被隔离和观察	102
Verifier、Reward 与 Result: 成功数字怎样回连执行证据	105
最小 Eval Loop: 从冻结规范到可复核报告	108
Run Identity 与可复现性: 保存“实际运行了什么”	111
Retry 与 Recovery: 恢复基础设施, 不能重抽产品答案	114
LLM-as-a-Judge: 先定义测量合同, 再接入模型	117
统计比较: 先配对 Trial, 再看效果量与区间	120
Agent Environment: 把命令、文件和终态变成评测观察	123
Quality Gate: 把证据转成三态/四态决定	125
Eval-to-RL: 评测信号怎样进入训练而不污染发布门禁	128
横向比较一: Task、Dataset 与 Target 怎样对齐	131

横向比较二：Runner、并发、缓存与 Retry	133
横向比较三：Trace、Artifact 与血缘	135
横向比较四：Scorer、Judge 与 Outcome	137
横向比较五：Metric、统计单位与不确定性	139
横向比较六：Agent Environment 与 Final State	141
横向比较七：Report、CI 与 Release Gate	143
平台适配器：LangSmith、Phoenix、MLflow 与 Braintrust 应该接在哪里	145
案例一：运费边界错误怎样形成完整评测证据	151
案例二：退款 Agent 的副作用、审批与幂等性	153
案例三：企业知识助手的 RAG、ACL 与泄漏门禁	155
案例四：合同审查 Agent 的多值 Reference 与 Judge 仲裁	157
SWE-bench：补丁评测为什么不等于“跑一次测试”	159
实验一：运行一份确定性 Eval	165
实验二：新增一个 Target Adapter	167
实验三：编写一个确定性 Scorer	169
实验四：重复运行并进行配对比较	171
实验五：导入并评测 Agent Trace	173
实验六：构建一个不会吞掉缺证的 Release Gate	175
术语表	177
怎样验证本仓库	178
来源与许可证边界	179
Eval Harness 源码内核品牌	180

Eval Harness 源码内核



Eval Harness 源码内核

EVAL HARNESS INTERNALS

从一个样本到一次发布决定，读懂评测系统如何运行

从一个样本到一次发布决定，读懂评测系统如何运行。

这是一套面向开发者的中文 Eval Harness 源码教材，并包含一个可离线运行的 Python Reference Harness。你会从 Task、Dataset、Trial、Attempt 和 Trace 开始，沿六套真实开源实现读懂执行、评分、统计比较与发布 Gate。开始学习 选择学习路线 下载完整中文 PDF

三条主线

主线	你会得到什么
共同语言	分清 Agent Harness 与 Eval Harness，建立 Sample、Trial、Attempt、Artifact、Score、Metric 和 Gate 的严格边界
源码知识库	沿锁定 commit 阅读 lm-evaluation-harness、Inspect AI、OpenAI Evals、Promptfoo、DeepEval、Harbor 与 Terminal-Bench 1
可运行参考	在无模型凭据、无容器的条件下执行 Target、保留证据、重评分、比较 Candidate/Baseline 并生成 Gate

第一次运行

```
uv sync --frozen
uv run eval-harness-ref run reference/examples/shipping/eval.yaml --output output/shipping
uv run eval-harness-ref inspect output/shipping
uv run eval-harness-ref compare output/shipping --candidate-target fixed --baseline-target buggy
```

完整目录、案例、实验和验证方法见文档总目录。

从这里开始：第一次把 Eval 跑成完整证据链

[返回目录](#) · [进入第一章](#)

这套教材回答什么

你可能已经会调用模型、写几条测试样本，甚至能得到一个准确率。但一个可信 Eval Harness 还要回答：测的是哪个系统版本？哪些 Sample 在运行前进入计划？超时重试有没有把失败洗掉？Scorer 看了什么证据？分母为什么是这个数？Candidate 相对 Baseline 的差异有多稳定？缺证据时 Gate 为什么没有通过？

本仓库从这些问题出发，把一次评测拆成下面的对象链：

```
EvaluationSpec
  → Sample × Target × Repetition
  → Trial
  → Attempt
  → TraceEvent / Artifact
  → ObservationBundle
  → ScoreRecord
  → MetricEstimate / Comparison
  → GateDecision / Report
```

先不要背类名。每读到一个对象，都问它为哪种错误负责、身份在何处固定、后续怎样反向核对。

贯穿全书的第一个问题

规则规定：订单金额达到 100 元时免运费。buggy 程序写成 `amount > 100`，fixed 程序写成 `amount >= 100`。Dataset 有金额 99、100、101 三个 Sample。对两个 Target 各执行一次，所以运行前就有 6 个 Trial。

金额 100 的 buggy Trial 会正常返回一个错误答案。这不是基础设施失败：Trial completed、Attempt succeeded/canonical、Score failed、Metric 2/3、Gate failed。这个分层是整套教材的起点。

第一次运行

安装 Python 3.12 和 uv，然后执行：

```
uv sync --frozen
uv run eval-harness-ref run reference/examples/shipping/eval.yaml --output output/shipping
uv run eval-harness-ref inspect output/shipping
```

依次查看：

- 1 output/shipping/run.json：每个 Trial 有几个 Attempt，哪个是 canonical；
- 2 output/shipping/traces/：事件怎样按因果顺序写入 JSONL；
- 3 output/shipping/artifacts/：输出字节怎样按 SHA-256 内容寻址；
- 4 output/shipping/evidence.json：Observation Bundle 怎样冻结评分输入；
- 5 output/shipping/report.json：Score、Metric 和 Gate 怎样保留引用。

再运行 score 和 gate。它们不会重跑 Target，而是从冻结证据重算，分别写出 rescore.json 与 regate.json。如果重算改变结果，应先检查证据或 Scorer 身份，而不是挑一个更好看的报告。

阅读约定

- Python 标识符和上游项目名称保留英文，首次出现给出中文解释；
- “源码事实” 必须落到锁定 commit 的永久链接；
- 跨多个调用点重建的责任边界标为“机制解释”；
- 本仓库缩小的伪代码和 Reference Harness 标为“教学简化”；
- 公开源码不足时明确写“不可核对”，不猜闭源内部实现；
- 每篇核心课都给实验、预期输出和答案，你不需要等待逐题互动才能继续。

接下来怎么走

如果你是第一次系统学习，从Agent Harness 与 Eval Harness开始按顺序读七篇基础课。已经做过评测工程的读者可以去学习路线选择源码、Agent 环境或 Eval-to-RL 路径。

Eval Harness 源码内核：完整目录

第一次阅读请从学习入口开始。本目录按“共同语言 → 最小实现 → 上游源码 → 工程专题 → 横向比较 → 案例与实验”组织，避免读者直接从工具名称进入后，把某个项目的内部命名误认为通用语义。

入口

- 开始学习：贯穿案例、阅读约定和第一次运行；
- 学习路线：新人、源码阅读、Agent 评测和 Eval-to-RL 路径；
- [仓库首页](#)：项目定位、内容地图和快速命令；
- 完整中文 PDF：可离线阅读的整书版本；
- [第三方来源](#)：锁定提交、许可证与引用边界。

第一部分：基础篇

- 1 Agent Harness 与 Eval Harness
- 2 Task、Dataset、Target 与 Environment
- 3 Sample、Trial 与 Attempt
- 4 Trace、Artifact 与 Observation
- 5 Scorer、Judge、Score 与 Metric
- 6 不确定性、比较与 Gate
- 7 Eval-to-RL 与独立发布评测

第二部分：可运行参考

- [运费边界案例](#)
- [eval-harness-ref 命令入口](#)
- [Reference Harness 自动化测试](#)

Reference Harness 展示 Trial/Attempt、Trace、内容寻址 Artifact、Observation、Score、Metric、Comparison、Gate 和离线报告的最小合同。它用于验证教材中的机制，不用于声称复刻任一上游工具。

第三部分：上游源码课程

- 1 lm-evaluation-harness: Task 怎样变成批量模型请求
- 2 Inspect AI: Solver、Sandbox、Scorer 与 EvalLog
- 3 OpenAI Evals: Registry、CompletionFn 与 Recorder
- 4 Promptfoo: 配置矩阵、Provider、Assertion 与 CI
- 5 DeepEval: Golden、TestCase、Metric 与执行策略
- 6 Harbor 与 Terminal-Bench 1: 环境、Agent、Verifier 与 Trial

每条课程都有导读与三篇深入章节。正文沿锁定提交给出源码入口、调用链、关键数据结构、失败传播、设计取舍、实验、预期输出和参考答案。

第四部分：工程篇

- 1 最小 Eval Loop
- 2 Run Identity 与可复现性
- 3 Retry 与 Recovery
- 4 LLM-as-a-Judge
- 5 统计比较
- 6 Agent Environment
- 7 Quality Gate
- 8 Eval-to-RL

第五部分：横向比较

- 1 Task、Dataset 与 Target
- 2 Runner、并发、缓存与 Retry
- 3 Trace、Artifact 与血缘
- 4 Scorer、Judge 与 Outcome
- 5 Metric、统计单位与不确定性
- 6 Agent Environment 与 Final State
- 7 Report、CI 与 Release Gate
- 8 平台适配器边界

第六部分：案例

- 运费边界
- 退款 Agent
- 企业知识助手
- 合同审查 Agent
- SWE-bench 补丁评测机制

前四个案例提供确定性的 buggy/fixed Target 和可运行门禁；SWE-bench 作为环境型机制案例，解释补丁、隔离环境、测试证据和实例级判定之间的关系。

第七部分：实验

- 1 运行一份确定性 Eval

- 2 新增 Target Adapter
- 3 编写 Scorer
- 4 重复运行并比较
- 5 导入 Agent Trace
- 6 构建 Release Gate

附录

- 术语表
- 验证与证据边界
- 来源与许可证

怎样读一篇源码课

先看锁定版本与入口，再沿调用链逐站回答：谁调用、输入是什么、状态在哪里改变、返回什么、失败怎样传播、哪组测试锁住行为。最后运行本仓库的确定性实验，验证你理解的是机制而不是 API 记忆。

学习路线

[返回目录](#) · [回到最后一篇基础课](#)

路线 A：第一次系统学习 Eval Harness

按七篇基础课顺序阅读，每篇先运行实验，再看答案。之后学习 Reference Harness 工程篇，最后进入六条上游源码课程。目标不是记住某个工具的配置，而是能从任意 Harness 中辨认 Trial、证据、评分、聚合和 Gate 的真实责任站点。

验收问题：给你一份只有最终准确率的报告，你能列出还缺哪些身份、分母、血缘和不确定性证据，并说明缺失会使 Gate 失败、阻断还是无法判断。

路线 B：源码阅读与工具选型

先读基础 01、02、04、05，再按“完整端到端 README → 入口 → 数据结构 → 执行循环 → Scorer → 报告 → 测试”阅读每条源码课程。最后进入横向比较，不用功能勾选表代替语义分析。

验收问题：对于两个项目里的同名 metric，你能说明它们输入是一条 Observation 还是一组 Score，是否承担 Judge、聚合或 Gate 职责，并给出锁定源码证据。

路线 C：Agent 与 Coding Agent 评测

重点阅读基础 01、03、04、06，再学习 Harbor/Terminal-Bench、SWE-bench 机制案例和 Agent Environment 工程篇。把工具事件、Diff、测试结果和环境终态分别建模，不使用 Agent 最终自述替代独立断言。

验收问题：面对一个 Coding Agent 超时后成功的案例，你能区分 Agent 内部恢复和 Harness Attempt，证明统计分母没有变化，并从 Gate 反查到补丁与测试 Artifact。

路线 D：Eval-to-RL 与质量发布

重点阅读基础 05、06、07，再学习 Judge 校准、统计比较、质量 Gate 和 Eval-to-RL 工程篇。始终把训练 reward、checkpoint choice 和 independent release eval 分开。

验收问题：给定一条 Score，你能写出 RewardAdapter 能力合同，说明它适合 DPO 偏好、GRPO/RFT 标量 reward、部分支持还是不可用，并设计不参与训练的独立发布集。

推荐实践节奏

每一阶段都交付一个可核对产物：第一阶段是完整运行目录，第二阶段是带永久链接的调用链笔记，第三阶段是一个新增 Target Adapter 或 Scorer，第四阶段是带计划分母与独立 Gate 的对比报告。产物能运行和回放，比“读完多少页”更能暴露理解缺口。

01 | Agent Harness 与 Eval Harness：先分清“执行”与“评测”

[上一章](#) · [下一章](#)

本篇要解决什么问题

看到 Agent 调用模型、执行工具、保存 Trace，再看到评测系统也读取 Trace、设置超时和报告成功率，新人很容易把两者理解成同一个 Harness。真正的分界不在“是否调用模型”，而在它为哪一种决定负责：Agent Harness 要完成一次用户任务；Eval Harness 要让许多次被测行为在相同规则下可比较，并把证据转成质量结论。

如果边界不清，评测层会悄悄替被测 Agent 重写提示、补工具结果或重试错误答案，最终测到的是评测器与 Agent 的混合系统。反过来，只保存 Agent 的最终回答而不冻结 Dataset、Target 身份和评分策略，又无法回答“这个版本在这组任务上是否更好”。

学完你能解释什么

- 为什么 Agent Loop 的停止条件属于被测系统，而 Trial 的停止、取消和预算属于评测协议；
- 为什么 Trace 是两个仓库的接口，却不能因此把两套 Harness 合并；
- 为什么一个 Agent 自己声称“任务完成”只是观察值，不是独立 Score；
- 怎样判断一项能力应该写进 Agent Harness 课程还是 Eval Harness 课程。

贯穿案例

运费函数规定订单金额达到 100 元免运费，旧实现却写成 `amount > 100`。Agent Harness 的视角是：模型怎样读文件、修改比较符、运行测试并停止。Eval Harness 的视角是：怎样冻结金额 99、100、101 三个样本，让 buggy 与 fixed 两个 Target 各运行一次，保存输出，再用同一个规则评分并形成 Gate。前者解释一次修复如何发生；后者解释我们凭什么相信修复改善了边界行为。

核心概念与边界

Agent Harness 管理模型上下文、Agent Loop、工具暴露、权限、Session、压缩和恢复。它的成功语义通常与“这一次任务能否继续或结束”有关。**Eval Harness** 管理 EvaluationSpec、Dataset、Trial Plan、Target Adapter、Observation、Scorer、Metric、比较和 Gate。它的成功语义分层存在：运行可能完成但评分失败，评分可能通过但整体 Gate 因缺失样本而无法判断。

Target Adapter 是明确接口：它接收 Trial 与运行约束，返回被测行为和可观察证据。它可以包住一个普通函数、RAG 服务、Coding Agent 或多智能体系统，却不应在适配层重新实现这些系统的内部决策。Agent Harness 生产 Trace 的语义；Eval Harness 检查 Trace 是否完整、是否属于正确 Target，并决定哪些字段进入评分。

机制图



调用链与状态变化

- 1 Eval Planner 依据冻结配置生成 Trial，把输入、Target 身份和重复序号固定下来。
 - 2 Runner 把 Trial 交给 Target Adapter。此后，模型选择工具、Agent 是否继续、如何恢复 Session，均由被测 Agent Harness 负责。
 - 3 Adapter 返回输出、Trace、Diff、日志或环境终态。Runner 只区分产品结果与基础设施故障，不替 Agent 改答案。
 - 4 Eval Harness 把 canonical Attempt 的事件和 Artifact 组成 Observation Bundle。
 - 5 独立 Scorer 消费 Bundle，Metric 对预声明 Trial 集合聚合，Gate 再依据冻结阈值作决定。
- 这条边界可以在 Reference Harness 的 run_evaluation 与 SubprocessTarget 之间直接看到。

关键数据结构

对象	属于哪一侧	必须保留的信息
Agent Session	Agent Harness	消息、工具状态、压缩与恢复点
TargetSpec	Eval Harness	被测适配器、版本与有效配置
Trial	Eval Harness	Sample、Target、重复序号和稳定 ID
TraceEvent	接口	因果父子关系、事件类型与可观察 payload
ObservationBundle	Eval Harness	canonical Attempt、Trace 与 Artifact 摘要

设计取舍

最干净的实现是依赖倒置：Eval Harness 只依赖 Target Adapter 协议，不依赖 Claude、Codex 或某个 RAG 框架的内部类。优点是同一 Dataset 和 Scorer 可以比较不同系统；代价是 Adapter 必须明确声明能力，例如能否导出工具事件、能否重置环境、能否报告实际模型版本。无法提供的能力应标为不可用，而不是伪造空字段。

Inspect AI 的锁定源码把通用评测入口放在 `_eval/eval.py`，并把 Task 执行进一步下沉到 `_eval/task`。这是**上游源码事实**。本篇把它抽象成两层责任，是帮助比较多套实现的**机制解释**，不是说所有项目使用相同类名。

失败语义

- Agent 返回错误运费：Target 运行完成、Score 失败；不得换一个 Attempt 再答一次。
- 宿主进程启动失败：Attempt 为基础设施失败；在预算内可创建恢复 Attempt。
- Trace 缺关键终态：运行也许完成，但 Scorer 应返回 `unscorable` 或 `invalid`。
- Target 身份与计划不一致：Trial 证据无效，不能用看似正确的输出替代身份错误。
- 质量阈值未达到：Gate 失败；这不等于 Harness 崩溃。

动手实验

在仓库根目录运行：

```
eval-harness-ref run reference/examples/shipping/eval.yaml --output output/shipping
eval-harness-ref inspect output/shipping
```

打开 `output/shipping/report.json`，找出 buggy Target 的运行状态、Score 状态和 Gate 状态。再看 `run.json`，确认错误答案没有触发第二个 Attempt。

预期输出与答案

应看到 6 个计划 Trial：3 个样本乘 2 个 Target。buggy 在金额 100 上输出错误，但该 Trial 仍是 `completed`，对应 Score 为 `failed`，buggy Gate 为 `failed`；fixed 的三个 Score 均通过，Gate 为 `passed`。答案的关键不是“buggy 运行失败”，而是“运行完成且产品结果被独立判错”。

常见误解

“Agent 已运行测试，所以不需要外部 Scorer”忽略了被测系统可以漏测、修改测试或误读输出。“Eval Harness 也有循环，所以它就是 Agent Harness”混淆了调度循环和决策循环。“Trace 越多越可信”也不成立；没有身份、因果顺序和摘要约束的日志只是更多文本。

如何核对

先阅读 `runner.py` 中产品失败与基础设施异常的分支，再阅读 `pipeline.py` 中 Bundle、Score、Metric 和 Gate 的生成顺序。上游部分可从锁定的 Inspect AI 入口继续追到 `_eval/task/run.py`，核对“公共 Eval 入口”和“Task 执行”确实是不同责任站点。

与其他 Harness 的关系

lm-evaluation-harness 更突出 Task、Model Adapter 和批量请求；Promptfoo 更突出 Provider、Test Case 与 Assertion；Harbor 更突出 Agent 与环境生命周期。它们切分代码的位置不同，却都可以放回“被测执行—证据—评分—聚合”坐标系。Agent Harness 仓库研究 Claude、Codex、Gemini、DeepSeek Harness、pi、OpenCode 等运行时内部；本仓库只研究它们作为 Target 时怎样被公平执行和评分。

本篇不能证明什么

这条边界不能证明某个 Agent 安全、某套 Eval Harness 生产就绪，也不能把一次确定性示例扩大成真实业务发布授权。它只给出职责划分和可核对的最小实现，实际系统还需要独立 Dataset、环境隔离、统计设计和风险门禁。

[上一章](#) · [下一章](#)

02 | Task、Dataset、Target 与 Environment：把问题定义完整

[上一章](#) · [下一章](#)

本篇要解决什么问题

“评测一下退款 Agent”不是可执行规格。它没有说要测退款资格判断、话术还是副作用；没有样本范围；没有标出被测版本；也没有说明账户、订单和工具状态。很多评测争论看似关于分数，实际是四个对象没有分开：Task 定义行为，Dataset 提供实例，Target 指定被测系统，Environment 提供可变世界。

学完你能解释什么

- 为什么同一 Dataset 可以服务多个 Target，但不能自动代表同一 Task；
- 为什么 prompt 或模型名字不足以形成 Target 身份；
- 为什么 Agent 评测必须把 Environment 初态和终态纳入协议；
- 怎样从一句业务问题得到可物化的 EvaluationSpec。

贯穿案例

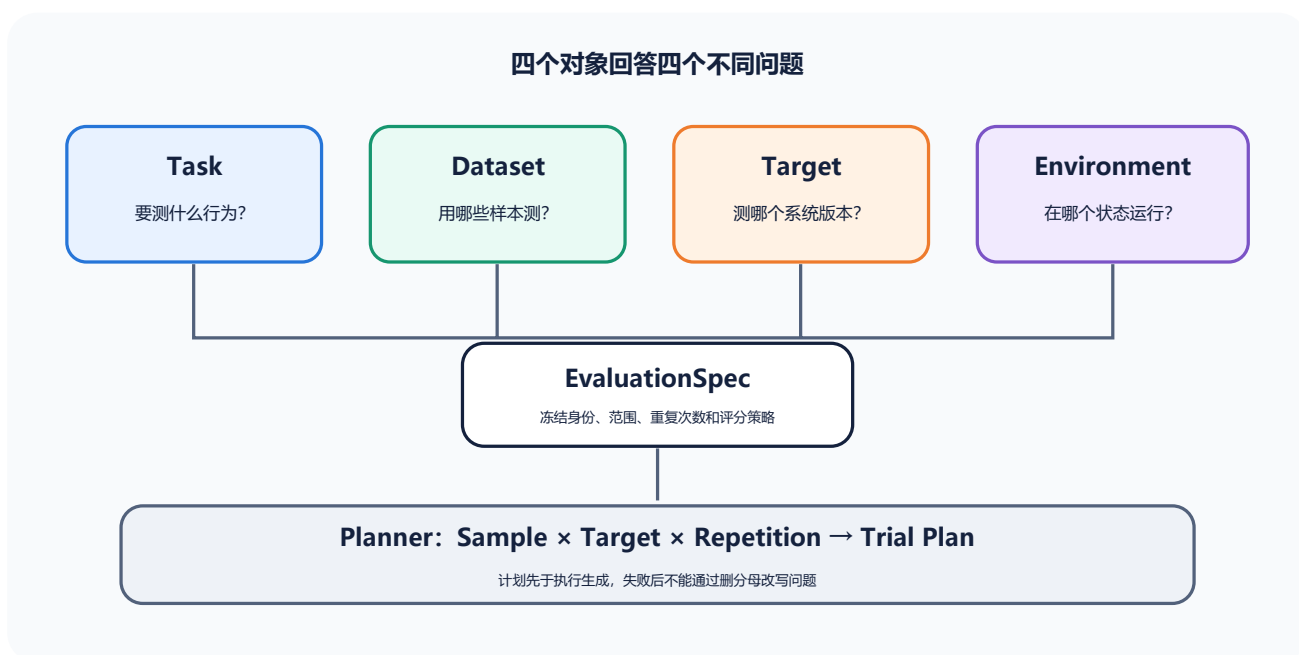
运费案例的 Task 是“根据订单金额返回运费”，判定边界是金额大于等于 100 时费用为 0。Dataset 包含 99、100、101 三个 Sample。Target 有 buggy 与 fixed 两个本地脚本。Environment 在这个最小例子中只是 Python 3.12 子进程和只读输入；若换成退款 Agent，Environment 就必须包含订单余额、退款 API、权限和可复位数据库。

核心概念与边界

Task 描述要观察的行为、输入输出契约和允许条件，不是某条样本。**Dataset** 是经过选择、版本化和分组的一组 Sample，不等于整个业务分布。**Target** 是被测系统的实际身份，至少包括 Adapter、模型或程序版本、有效配置与依赖。**Environment** 是运行行为能够读取或改变的外部状态，包含文件、容器、服务、账户、时钟和网络策略。

四者组合后才能回答一个具体问题：“在环境 E 的冻结初态下，Target T 对 Dataset D 中代表 Task K 的样本表现怎样？”改变其中任何一个都可能改变结论。尤其不要把 Environment 藏进 Target 名称，也不要把 Reference 答案塞进 Agent 可见环境。

机制图



调用链与状态变化

- 1 设计者把业务决定改写为 Task：输入是什么、允许什么行为、什么证据支持通过。
- 2 Dataset Builder 选择 Sample，并固定 Sample ID、Split、Reference 与来源。
- 3 Target Resolver 把逻辑名称调和为实际 Adapter 和版本，拒绝身份漂移。
- 4 Environment Factory 创建干净初态，限制网络和凭据，声明重置方式。
- 5 EvaluationSpec 冻结 Target 列表与重复次数；Planner 执行笛卡尔积，生成不可随结果缩减的 Trial Plan。

Reference Harness 的 EvaluationSpec、plan_trials 和 shipping 配置展示了最小实现。复杂 Agent 环境会在后续工程篇增加环境工厂，而不会改变这四个对象的语义。

关键数据结构

```

evaluation_id: shipping-boundary
dataset: dataset.jsonl
repetitions: 1
targets:
  - target_id: buggy
    adapter: python_script
    script: target_buggy.py
scorer:
  scorer_id: shipping-fee:v1
  field: fee
  
```

这里 target_id 是计划内逻辑身份，script 是解析后执行入口。真实模型 Target 还应记录模型版本、系统提示摘要、工具集和服务端返回的实际身份。Environment 则应有自己的镜像摘要、初态 Fixture 和重置结果，避免全部塞入一个无法审计的 config 字符串。

设计取舍

Dataset 可以内联在配置，也可以外部版本化。外部 JSONL 更利于稳定 ID、Diff 和分片；代价是要管理文件血缘。Target 可以由构造函数直接创建，也可以 Registry 驱动；Registry 易复用但可能隐藏实际配置，因此运行报告必须保存解析后的身份。Environment 使用容器隔离更强，却不是所有任务都需要；纯函数评测应保持轻量，Agent 副作用任务则必须优先隔离和重置。

lm-evaluation-harness 的 Task 与 LM 分别承载任务与模型适配，是**上游源码事实**。本篇的四对象模型是跨 Harness 的**机制解释**，并不要求上游采用 Environment 这个类名。

失败语义

- Dataset 为空或 Sample ID 重复：规格无效，不能开始运行。
- Target 声明模型 A，服务实际返回模型 B：身份调和失败，证据应标为 invalid。
- Environment 创建失败：Trial blocked，可在基础设施预算内恢复；不能当作产品零分。
- Environment 未成功重置：后续 Trial 可能互相污染，应阻断而非继续累计分数。
- Task 只写“效果好”：Scorer 无法知道观察边界，评测设计尚未完成。

动手实验

复制 shipping 配置到临时目录，把 repetitions 改成 2，并保留 3 个 Sample、2 个 Target。运行 eval-harness-ref run 后查看 evidence.json 中的 trials，不要先看报告分数。

预期输出与答案

应得到 12 个 Trial：3 × 2 × 2。每个 Trial 都保存相同 Task 语义下的一个 Sample、一个 Target 和一个重复序号。重复次数改变的是实验设计，不会复制 Sample ID；Environment 若无法为每次 Trial 提供等价初态，则这 12 次不具有可直接比较的含义。

常见误解

“Dataset 就是 Task”会让换一个问题却沿用旧分数；“模型名就是 Target”忽略提示、工具和服务版本；“Docker 就等于 Environment 可复现”忽略外部 API、时间和凭据；“更多样本自然更代表业务”忽略采样偏差和重复实体。

如何核对

运行 `python -m pytest tests/test_planner.py -q`，确认 Target × Sample × Repetition 的数量和顺序稳定。再比较 `sources/sources.lock.yml` 与正文永久链接，理解“上游源码身份”同样需要 commit，而不是浮动 main。上游 Harbor 的 `environments/base.py` 可用于核对 Agent 环境为什么值得独立抽象。

与其他 Harness 的关系

OpenAI Evals 强调 Registry 与 Eval Spec，Promptfoo 用配置连接 Provider 与 Test Case，DeepEval 借用测试框架式 Test Case，Harbor 把任务包、Agent 和 Environment 组合为 Trial。名字不一——等价：Provider 可能同时包含 Target 解析与调用，Golden 可能同时承载输入和 Reference。比较时应先映射责任，再比较字段。

本篇不能证明什么

定义完整四对象并不保证 Dataset 有代表性、环境无泄漏或 Target 可复现。它只让这些风险有明确归属和可检查字段；有效性、可靠性和发布决策仍需后续章节的证据。

[上一章](#) · [下一章](#)

03 | Sample、Trial 与 Attempt: 别让重试改写统计分母

[上一章](#) · [下一章](#)

本篇要解决什么问题

调用超时后再试一次很合理；模型答错后再问一次有时也能得到正确答案。但这两种“重试”不能混为一谈。前者是在恢复同一个实验对象的基础设施执行，后者是在给被测产品额外机会。如果只记录最后一次成功，失败样本会被重试成通过，成功率分母也会随着可用结果变化。

学完你能解释什么

- Sample、Trial、Attempt 为什么是三层对象，而不是三个同义词；
- 为什么 Trial 是统计单位，Attempt 是恢复记录；
- canonical Attempt 怎样阻止多个结果同时进入评分；
- blocked、completed 与 Score failed 为什么可以同时出现在不同层。

贯穿案例

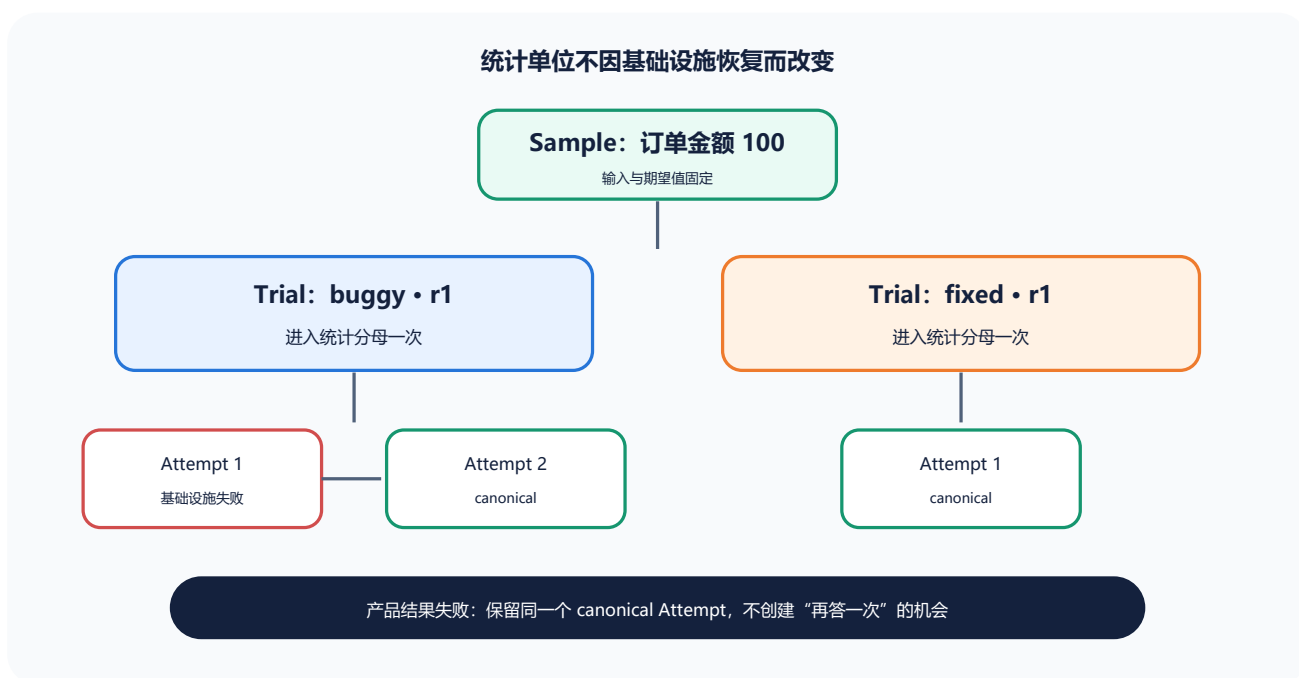
金额 100 的 Sample 分别交给 buggy 与 fixed Target，形成两个 Trial。如果 buggy 脚本正常启动并返回运费 10，这个 Trial 已完成；答案虽错，也不能再启动一次“希望它答对”。若 Python 子进程因临时资源错误没有启动，Harness 可以在预声明的两次基础设施预算内创建 Attempt 2。恢复成功后 Attempt 2 成为唯一 canonical Attempt，Attempt 1 仍保留在证据中。

核心概念与边界

Sample 是 Dataset 中稳定的输入与 Reference 单元。**Trial** 是 Sample × Target × Repetition 物化后的统计对象，它在执行前已进入计划分母。**Attempt** 是为完成一个 Trial 而发生的基础设施执行记录，记录序号、错误码和是否 canonical。

重复次数不是 Attempt。对随机模型做 5 次重复会生成 5 个 Trial，因为每次都是计划内观测；某个 Trial 因网络握手失败而恢复 2 次，仍只有 1 个统计单位。产品级 retry 若本来就是 Target 的算法，例如 Agent 自己在 Loop 内重新调用模型，应封装在同一个 Target 行为里，并在 Trace 中可见，而不是由 Eval Harness 偷偷增加 Attempt。

机制图



调用链与状态变化

- 1 Planner 在运行前生成稳定 Trial ID，格式包含 run、Target、Sample 与重复序号。
- 2 Runner 创建 Attempt 1，调用 Target Adapter。
- 3 Adapter 返回正常结果时，Attempt 状态为 succeeded 且成为 canonical；Trial 状态为 completed。
- 4 Adapter 抛出被声明为基础设施错误的异常时，Attempt 为 infra_failed；若还有预算，Runner 创建下一个 Attempt。
- 5 所有恢复机会耗尽后，Trial 为 blocked，没有 canonical Attempt，也不能伪造 Observation Bundle。
- 6 产品失败由 Adapter 作为正常返回的一种结果类型传回。它仍产生 canonical Attempt，随后由 Scorer 判断失败。

这条状态机直接实现在 `run_trial`；`tests/test_runner.py` 分别锁住基础设施恢复、产品失败不重试和 canonical 唯一性。

关键数据结构

```

Trial
  trial_id
  target_id
  sample
  repetition
  
```

```

Attempt
  attempt_id
  trial_id
  ordinal
  status
  canonical
  error_code
  
```

Trial 不保存“最终分数”，因为执行和评分是两个阶段。Attempt 也不保存 Dataset 权重，因为统计设计不属于恢复层。canonical 不是“最好的一次”，而是协议允许进入后续证据链的唯一一次；选择规则必须在看到结果前确定。

设计取舍

最简单的本地 Harness 可以同步执行并在内存里选择 canonical。分布式系统还需要 Lease、Fencing Token 和幂等提交，防止超时 Worker 在新 Worker 已接管后迟到写入。首版 Reference Harness 不假装实现分布式一致性，而是把“最多一个 canonical”作为模型不变量。这样课程能清楚展示语义，同时避免用复杂基础设施遮住统计问题。

Harbor 的锁定 trial.py 展示 Agent Trial 生命周期，是**上游源码事实**。Reference Harness 的 Attempt 分层是本仓库的**教学实现**，字段不能直接当作 Harbor 内部结构的逐项翻译。

失败语义

- 超时、进程无法启动、临时工作区失败：基础设施错误，可按预算增加 Attempt。
- 程序返回非零且表示被测测试失败：产品失败，Trial 完成，不由 Harness 重试。
- 两个 Attempt 都标 canonical：Trial invalid，后续 Score 不可信。
- 只有失败 Attempt：Trial blocked，不是 Score 0。
- 运行后才删除困难 Trial：破坏预声明分母，整次 Metric 无效。

动手实验

运行 `python -m pytest tests/test_runner.py -q`。随后打开测试中的 FlakyTarget 和产品失败 Target，手工预测每种情况下 Attempt 数量、canonical 序号和 Trial 状态，再对照断言。最后运行 shipping 示例，统计 run.json 中每个 Trial 的 Attempt 数。

预期输出与答案

基础设施第一次失败、第二次成功时：2 个 Attempt，第二个 canonical，Trial completed。产品失败时：1 个 succeeded/canonical Attempt，Trial completed，`product_failed=true`。所有基础设施机会失败时：Attempt 数等于预算，均非 canonical，Trial blocked。shipping 正常运行中每个 Trial 只有一个 Attempt，即使 buggy 的一个 Score 最终失败。

常见误解

“只要最终成功，前面的 Attempt 可以删掉”会丢失可靠性与成本证据；“Attempt 越多样本越多”会造成伪重复；“blocked 就按 0 分最保守”把平台故障错误归因给产品；“模型自洽重试也必须拆 Attempt”则混淆 Target 内部策略与 Harness 恢复。

如何核对

阅读 models.py 的分层状态枚举，再运行 `eval-harness-ref inspect` 查看计划 Trial 数和 Observation Bundle 数。检查 Metric 的 denominator 来自 Trial 清单而不是 Bundle 数。上游 SWE-bench 的 `infra_failure.py` 提供了环境型评测显式处理基础设施失败的另一处源码证据。

与其他 Harness 的关系

benchmark 型 Harness 常把请求重试藏在 Model Adapter；声明式应用评测可能把每个 Test Case 直接称作一条结果；Agent 环境 Harness 通常有更重的 Trial 生命周期。比较这些实现时要追踪“统计分母何时确定”“产品错误是否可重试”“哪个结果进入评分”，不能只对齐 retry 配置名。

本篇不能证明什么

正确分层不会自动选择最佳重试预算，也不能保证分布式执行恰好一次。它能证明的是：在声明的本地协议内，基础设施恢复不会悄悄增加成功样本或删除失败 Trial。

[上一章](#) · [下一章](#)

04 | Trace、Artifact 与 Observation：让分数能回到证据

[上一章](#) · [下一章](#)

本篇要解决什么问题

报告写着“通过 92%”并不等于结果可核对。我们还要知道每个 Score 依据哪次执行、看了哪些事件、读取了哪个文件、文件是否被改写，以及评分发生后能否沿引用回到原始观察。Trace、Artifact 与 Observation Bundle 正是把“日志很多”变成“证据有血缘”的三层结构。

学完你能解释什么

- 事件流、文件产物和评分输入为什么要分开；
- Trace 的父子关系和序号怎样表达因果，而不仅是时间戳；
- 内容摘要怎样检测 Artifact 被替换；
- 为什么 Score 必须绑定 Observation Bundle 和 canonical Attempt。

贯穿案例

shipping Target 返回 {"fee": 10}。Runner 写两个事件：Trial 开始、Target 完成；后者的 payload 同时保存 output 与 expected。原始输出还作为内容寻址 Artifact 保存。Bundle 把事件、Artifact 摘要和 canonical Attempt 绑定起来。Scorer 并不再次调用 Target，而是只读取 Bundle。因此之后执行 score 可以在不重跑脚本的情况下得到同一结果。

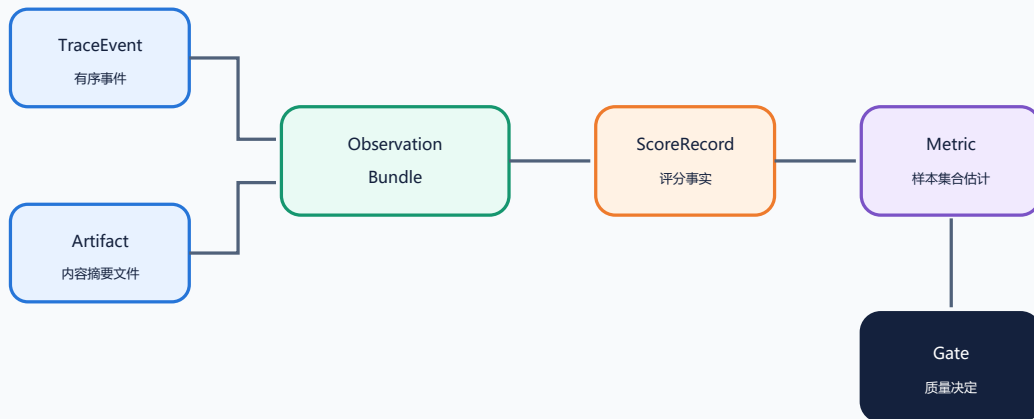
核心概念与边界

TraceEvent 是运行中可排序、可连接的事实，至少有稳定 event_id、sequence、type、parent_event_id 和 payload。**Artifact** 是不适合内联在事件里的字节对象，例如 Diff、日志、截图、测试报告或环境终态；它用 SHA-256 摘要和相对路径引用。**Observation Bundle** 是评分输入快照，声明“对这个 Trial 的这个 canonical Attempt，评分器被允许看到这些事件和产物”。

Trace 不等于隐藏思维链。Eval Harness 应记录外部可观察的模型消息、工具调用、状态变化和输出，不要求或假装获取模型内部推理。Artifact 也不等于任意附件：没有摘要、类型和来源关系的文件不能自动进入评分。

机制图

结论必须能沿引用反向走向原始观察



缺少摘要、canonical Attempt 或 Scorer 身份时，后续层只能阻断或无法判断

调用链与状态变化

- 1 Runner 为 canonical Attempt 产生有序事件；TraceWriter 拒绝重复 event_id 和先于父事件出现的子事件。
- 2 大对象进入 ArtifactStore。相同字节得到相同摘要并复用同一个内容地址。
- 3 Bundle Builder 验证 Trial 只有一个 canonical Attempt，再把事件和 ArtifactRef 规范化哈希。
- 4 Scorer 读取 Bundle，ScoreRecord 保存 Bundle digest、canonical_attempt_id 和 scorer_id。
- 5 Metric 保存参与聚合的 score_ids，Gate 保存 metric_ids。报告因此可以从决定逐层反查。

Reference Harness 分别在 tracing.py、artifacts.py 和 reporting.py 实现这三段。

关键数据结构

```

TraceEvent(event_id, sequence, type, parent_event_id, payload)
ArtifactRef(kind, digest, relative_path)
ObservationBundle(bundle_id, digest, trial_id,
                  canonical_attempt_id, events, artifacts)
ScoreRecord(..., observation_bundle_digest, scorer_id)
  
```

时间戳可用于性能分析，却不应代替 sequence 和 parent 关系，因为不同机器时钟可能漂移。relative_path 让报告可迁移；在公开文档和证据里写本机绝对路径既不可复现，也可能泄露用户名。digest 必须覆盖真正进入评分的内容，而不是只哈希文件名。

设计取舍

JSONL 适合追加事件、逐行恢复和命令行检查；代价是跨事件约束需要额外验证。把完整输出同时放事件和 Artifact 会有重复，但小型教学实现借此同时展示可读 payload 与内容寻址。生产系统可让事件只保存 ArtifactRef，但必须确保 Scorer 的实际读取被纳入 Bundle digest。

Inspect AI 的锁定 log/_log.py 是 Eval Log 模型的**上游源码事实**；OpenAI Evals 的 record.py 展示另一种事件记录边界。把二者统一解释为证据血缘是本篇的**机制解释**。

失败语义

- 子事件引用尚未出现的父事件：Trace invalid。
- Artifact 文件缺失或摘要不匹配：Bundle 不能评分；不能只相信报告里的旧值。
- Trial 有零个或多个 canonical Attempt：Bundle 构造失败。
- Scorer 需要字段但事件和 Artifact 都未提供：Score unscorable，不是 0。
- Trace 包含秘密或不必要个人数据：属于安全失败，应在保存前过滤并阻断发布。

动手实验

先运行 shipping 示例，再执行：

```
eval-harness-ref inspect output/shipping
eval-harness-ref score output/shipping
```

查看 evidence.json 的第一个 Bundle，找到 Artifact digest 与文件相对路径。复制该 Artifact 的内容并计算 SHA-256，对比文件名。然后临时移动一个 Artifact，再运行 inspect，观察证据检查失败；完成后恢复文件。

预期输出与答案

正常时 inspect 报告 6 个 Trial、6 个 Bundle、6 条 Score。Artifact 文件名应等于 sha256: 后的 64 位十六进制摘要。缺少任意已引用 Artifact 时，inspect 应以非零状态退出并显示“运行证据无效”，而不是继续打印原 Gate。score 会从冻结 Bundle 重算 6 条评分，不会产生新的 Attempt。

常见误解

“日志存在就能审计”忽略身份和因果；“数据库记录不可变所以不用摘要”忽略管理员、迁移和导出边界；“最终回答足够评分”忽略工具副作用与环境终态；“把全部内部推理都存下来更透明”则带来隐私、安全和不可验证来源问题。

如何核对

运行 `python -m pytest tests/test_lineage.py tests/test_cli.py -q`。检查测试是否覆盖父子顺序、事件去重、Artifact 去重、canonical Attempt 绑定和缺失文件。再从 report.json 的 Gate 找 metric_id，从 Metric 找 score_ids，从 Score 找 observation_bundle_digest，确认链条可逆。

与其他 Harness 的关系

Promptfoo 常围绕测试结果和 Trace 类型组织应用评测；Inspect AI 的 Eval Log 更完整地承载样本与事件；Agent Environment Harness 还需保存容器日志、补丁和终态。实现密度不同，但最小问题相同：Scorer 看见了什么、证据属于哪次运行、内容后来是否变化。

本篇不能证明什么

完整血缘只能证明报告与已保存证据的一致性，不能证明 Dataset 正确、Reference 无偏或被测环境没有未记录的外部副作用。证据链解决“结论来自哪里”，不自动解决“问题问得是否正确”。

[上一章](#) · [下一章](#)

05 | Scorer、Judge、Score 与 Metric：先判单次，再描述总体

[上一章](#) · [下一章](#)

本篇要解决什么问题

评测项目常把“指标”一词同时用于判断一句回答、计算一组平均值和决定是否发布。结果是 Judge 的一条主观判断被当成总体质量，或缺失输出被无声写成 0。要避免这种混乱，必须把执行评分器的组件、单 Trial 评分事实和跨 Trial 统计估计拆开。

学完你能解释什么

- Scorer、LLM-as-Judge、ScoreRecord 和 MetricEstimate 的责任差异；
- 为什么确定性证据应优先于模型自评；
- failed、uncertain、unscorable 与 invalid 为什么不能都压成 0；
- 为什么 Metric 的分母来自 Trial Plan，而不是成功返回的 Score 数。

贯穿案例

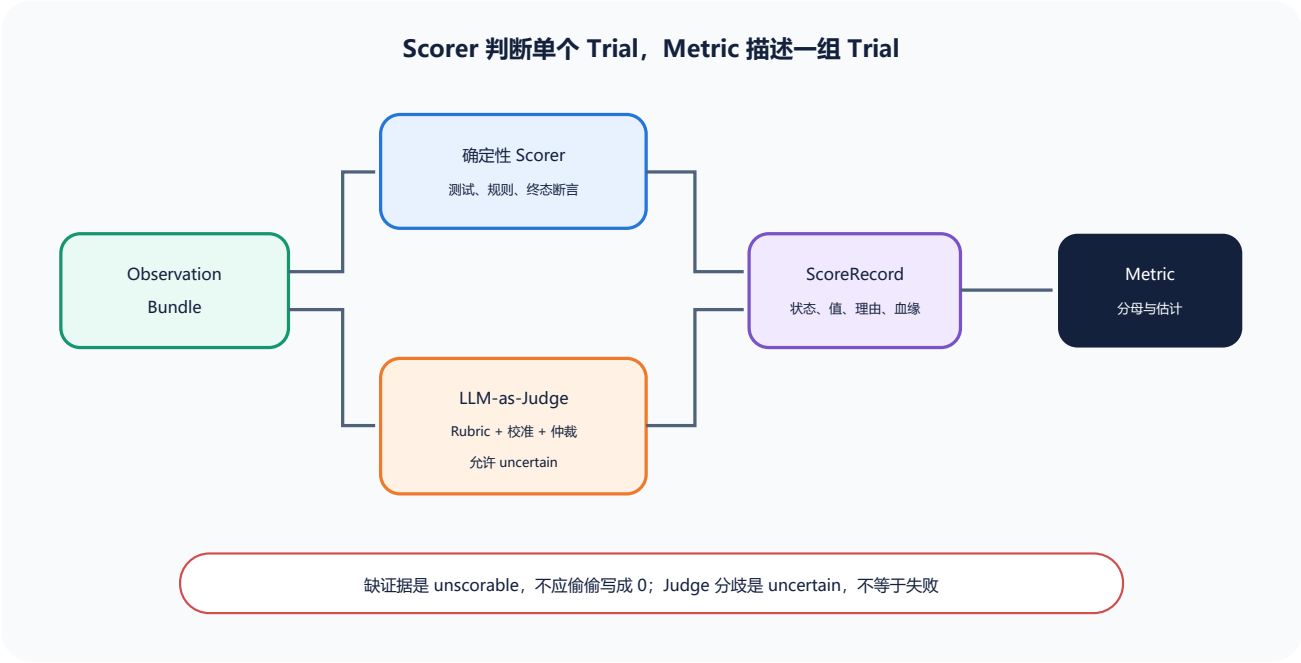
shipping 的 Scorer 读取 Observation 中的 fee 和样本期望值：相等得到 passed/1，不等得到 failed/0。它不关心 Target 是脚本还是模型。对三个 Sample 聚合时，Metric 分母固定为三个计划 Trial；即使某个 Bundle 缺少 fee，分母仍为三，该条 Score 是 unscorable，并进一步使 Gate 无法判断，而不是从分母删除。

核心概念与边界

Scorer 是把一个 Observation Bundle 转换成 ScoreRecord 的可版本化函数。**LLM-as-Judge** 是使用模型实现的 Scorer，不是独立层级；它还需 Rubric、采样设置、校准集、偏差检查和分歧处理。**ScoreRecord** 是对单个 Trial 的评分事实，包含状态、值、理由、Scorer 身份和证据血缘。**MetricEstimate** 对预声明的 Trial/Score 集合做聚合，包含分子、分母、估计值和 score_ids。

环境事实可直接验证时，优先用测试、规则或终态断言。Judge 适合多值文本质量、语义符合度和需要 Rubric 的判断，但其输出应允许 uncertain，并与人工标注或确定性切面校准。模型说“我完成了”只是 Target 输出，不能作为自己的独立评分。

机制图



调用链与状态变化

- 1 Scorer 从 Bundle 中寻找被协议允许的观察字段，不直接重跑 Target。
- 2 字段存在且满足规则时生成 passed Score；存在但不满足时生成 failed Score。
- 3 关键字段缺失时生成 unscorable；协议或血缘破坏时应生成 invalid；Judge 无法稳定裁决时生成 uncertain。
- 4 Aggregator 以计划 Trial ID 为分母，验证 Score 没有引用计划外 Trial，也没有重复 score_id。
- 5 Metric 保存 numerator、denominator、value 和参与的 score_ids，供比较与 Gate 使用。

本仓库的 FieldMatchesExpectedScorer 与 aggregate_pass_rate 是最小可运行实现。

关键数据结构

层	关键字段	不能缺少的身份
Scorer 配置	field、Rubric、阈值、采样参数	scorer_id 与版本
ScoreRecord	status、value、reason	Trial、Attempt、Bundle、Scorer
MetricEstimate	numerator、denominator、value	metric_id 与 score_ids

值和状态必须同时保留。value=0 可能代表规则明确判错，却不能表达缺证据或协议无效；只有状态能告诉 Gate 这个数是否可用于推断。理由用于人类核对，不应取代结构化状态。

设计取舍

二元规则简单、可复现，却可能遗漏部分质量。连续分数更细，但阈值、尺度和校准会影响解释。多个 Scorer 可以分别测正确性、安全和格式，再由非补偿 Gate 处理关键风险；不应先把关安全失败平均进高正确率。Judge 可以降低人工成本，但需要冻结模型、prompt、Rubric 和随机性，并定期与人工样本对照。

DeepEval 的 `base_metric.py` 展示 Metric/Judge 抽象，是**上游源码事实**。Promptfoo 的 `assertions/index.ts` 展示 Assertion 分派。把二者映射到统一的 Scorer 责任是**机制解释**，并非断言其状态枚举相同。

失败语义

- 可观察值与 Reference 不符：failed，可计入明确失败。
- 需要字段没有采集：unscorable，必须暴露缺证问题。
- Judge 两次判断冲突且无仲裁：uncertain，不能挑有利一次。
- Scorer 版本或 Bundle digest 不匹配：invalid，整个 Score 不应聚合。
- 计划 100 个 Trial 只产生 90 个 Score：Metric 分母仍为 100，并单独报告缺失。

动手实验

运行 shipping 示例后打开 `evidence.json`，将副本中某个 `target_completed` 事件的 output 移除，在测试代码里用 `FieldMatchesExpectedScorer` 对该 Bundle 评分。再对完整 3 个计划 Trial 中仅 2 条 passed Score 调用 `aggregate_pass_rate`。

预期输出与答案

缺字段的 Bundle 应产生 unscorable 且 value 为 null；它不能被默认为 failed。两条 passed Score 配三个计划 Trial 时 Metric 为 2/3，而不是 2/2。若把 unscorable Score 一起交给 Gate，Gate 应为 inconclusive，因为证据不足以支持通过或明确失败。

常见误解

“Judge 给数字就很客观”忽略校准与模型偏差；“unscorable 按 0 最安全”混淆产品质量和 Harness 可观测性；“平均分足够”会掩盖关键风险与分组差异；“同名 metric 可直接比较”忽略数据、Scorer 和分母定义。

如何核对

运行 `python -m pytest tests/test_scoring.py tests/test_metrics.py tests/test_gates.py -q`。重点检查缺字段状态、计划分母、重复 `score_id` 和不可用证据不能过 Gate。再阅读锁定 DeepEval 实现，区分上游所谓 Metric 何时同时承担本篇的 Scorer 与 Score 汇总责任。

与其他 Harness 的关系

Im-evaluation-harness 常让 Task 构造请求并聚合 metric；Inspect AI 明确暴露 Scorer 与 Metric；Promptfoo 以 Assertion 面向测试配置；DeepEval 用 Metric 类执行包括 Judge 在内的单案例判断。名称相同不代表层级相同，比较时应问“输入是一条 Observation 还是一组 Score”。

本篇不能证明什么

评分血缘和状态正确不能证明 Rubric 有效、Judge 无偏或 Dataset 代表线上流量。它只阻止不兼容的证据被静默压成一个看似精确的数字。

[上一章](#) · [下一章](#)

06 | 不确定性、比较与 Gate：从分数到可执行决定

[上一章](#) · [下一章](#)

本篇要解决什么问题

Candidate 得分 82%，Baseline 得分 80%，能否发布？只看两个点估计无法回答：它们是否测了相同样本、差异是否由少数样本驱动、重复运行是否相关、有没有缺失证据、关键安全切面是否失败，以及阈值是不是看到结果后才选。比较和 Gate 必须把统计问题与决策政策分开。

学完你能解释什么

- 为什么同一 Sample 上的 Baseline/Candidate 应保持配对；
- Bootstrap 区间表达什么、不表达什么；
- Metric、ComparisonResult 与 GateDecision 的区别；
- 为什么 Gate 需要 passed、failed、blocked、inconclusive 四种结果。

贯穿案例

shipping 有三个样本。buggy 通过 2/3，fixed 通过 3/3，点估计差为 1/3。因为两个 Target 使用完全相同的 Sample 和 repetition，可以计算每一对的 Score 差，再对配对差值重采样。样本只有三个，区间不会神奇地提供真实业务确定性；但配对至少避免把样本难度差异误当模型差异。发布 Gate 还要求 fixed pass-rate 达到 1.0 且没有 unscorable 证据。

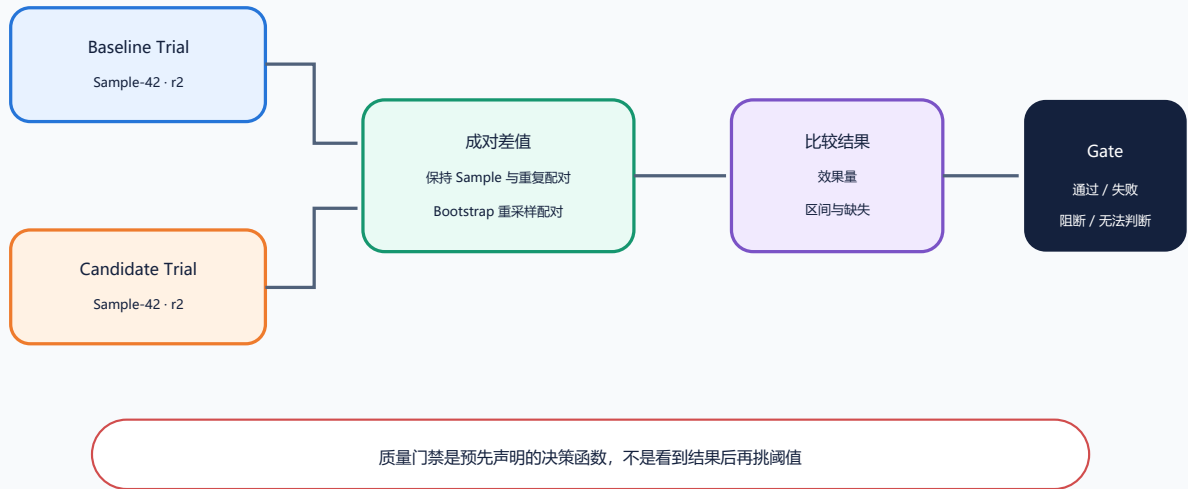
核心概念与边界

MetricEstimate 描述某个 Target 在一组计划 Trial 上的估计。**ComparisonResult** 描述 Candidate 相对 Baseline 的效果量、方向和不确定性。**GatePolicy** 是预先声明的决策函数，读取有效 Metric、比较和关键检查。**GateDecision** 是一次具体运行得到的通过、失败、阻断或无法判断，并保存依据。

blocked 表示计划条件没满足，例如环境不可用或唯一 Metric 缺失；inconclusive 表示已有运行证据却不足以作通过/失败推断，例如关键 Score uncertain。failed 表示有效证据明确未达政策；passed 才表示所有声明条件满足。四者不应压成布尔值。

机制图

不要只问“谁的平均分高”，还要问“差异有多稳”



调用链与状态变化

- 1 依据 Sample ID 和 repetition key 连接 Baseline 与 Candidate Trial，保留无法配对的缺失记录。
- 2 对每对 Score 计算差值；若一个实体产生多个相关样本，还需在实体或 cluster 层重采样。
- 3 用固定 seed 的 paired Bootstrap 反复抽取配对，得到效果量分布和区间。
- 4 Gate 先检查证据可用性与关键非补偿条件，再读取 Metric 或 Comparison。
- 5 GateDecision 保存 metric_ids、状态与原因；Reporter 不得把它扩大为真实生产发布授权。

Reference Harness 的 paired_bootstrap 和 evaluate_gate 分别承担统计与政策层。

关键数据结构

```

MetricEstimate(metric_id, numerator, denominator, value, score_ids)
ComparisonResult(baseline_id, candidate_id, mean_difference,
    interval_low, interval_high, pairs, seed)
GateDecision(gate_id, status, metric_ids, reason)
  
```

区间算法还应保存 iterations、重采样单位和缺失策略；否则同一个“95% 区间”可能来自完全不同假设。

GatePolicy 要版本化，阈值、关键切面和例外流程不能只存在 CI 脚本的一行 if 中。

设计取舍

非配对比较实现简单，但同一 Dataset 下会浪费样本内相关信息；配对比较更敏感，却要求身份和缺失处理严格。Bootstrap 对复杂分布直观，但不是小样本的万能证明，也不能修复有偏 Dataset。质量 Gate 可以只看绝对阈值，也可同时要求“不劣于 Baseline”；关键安全风险通常采用非补偿规则，不能被高平均正确率抵消。

lm-evaluation-harness 的 evaluator.py 与 evaluator_utils.py 展示任务运行和结果整理的上游源码事实。本仓库把发布 Gate 独立建模，是面向质量工程的教学实现，不是声称上游自带同一 Gate API。

失败语义

- Baseline 与 Candidate Sample 集不一致：比较 blocked，不能比较两个孤立平均值冒充配对。
- 区间跨越预声明的最小效果边界：结果 inconclusive，不等于 Candidate 一定更差。
- 关键安全 Score invalid/unscorable：Gate inconclusive 或 blocked，绝不能 passed。
- 有效 pass-rate 低于阈值：Gate failed。
- 看到 82% 后把阈值从 85% 改为 80%：政策污染，需新版本与独立运行。

动手实验

运行 `python -m pytest tests/test_metrics.py tests/test_gates.py tests/test_scoring.py -q`，再在 Python REPL 中对 `[0, 0, 1]` 这组三个配对差值调用 `paired_bootstrap(seed=7)` 两次。随后把 `seed` 改为 8，比较点估计与区间。

预期输出与答案

相同 `seed` 和 `iterations` 应生成完全相同结果，证明算法回放稳定；改变 `seed` 可能改变有限 Bootstrap 样本的区间，但原始配对差的均值仍为 1/3。shipping 的 fixed Gate passed、buggy Gate failed；若任一关键 Score 改为 unscorable，相关 Gate 必须 inconclusive，而不是用剩余样本计算 100%。

常见误解

“区间不重叠就一定可发布”忽略政策和多重比较；“样本数大就没有偏差”混淆方差与系统偏差；“p 值大说明两者相同”把未拒绝当等价；“Gate failed 等于程序失败”混淆有效负面结论和执行故障。

如何核对

查看 `tests/test_metrics.py` 是否锁住计划分母，查看 `tests/test_gates.py` 是否阻止无效证据通过。手工从 `report.json` 取 Score 重算 numerator/denominator，并确认 Gate 原因中的阈值与配置一致。

与其他 Harness 的关系

Promptfoo 常把评测接入 CI 并用阈值判断；DeepEval 适合测试断言式回归；benchmark Harness 更关注聚合与排行榜；平台型产品可能提供实验比较。无论界面如何，统计估计与政策决定都应可分开复核，并明确缺失和多重依赖。

本篇不能证明什么

一个本地 Bootstrap 和阈值 Gate 不构成完整统计审查，也不授权真实生产发布。它只展示从预声明 Trial、配对 Score 到可解释 Decision 的最小闭环；外部有效性、风险接受和组织审批仍在系统外。

[上一章](#) · [下一章](#)

07 | Eval-to-RL 与独立 Release

Eval: 改进闭环不能吃掉验收集

上一章 · 下一章

本篇要解决什么问题

评测发现失败样本后，最有价值的下一步往往是生成偏好对、验证器奖励或在线强化学习任务。但一旦同一批样本和 Scorer 被用于训练、选择 checkpoint，又继续用来声称发布质量，模型可能只学会通过已知门禁。Eval-to-RL 的关键不是把分数接到训练 API，而是保持训练 reward、checkpoint 选择和独立 Release Eval 三种决定的证据隔离。

学完你能解释什么

- Evaluator、RewardAdapter、DPO/GRPO/RFT 和独立发布评测怎样形成完整链路；
- 为什么并非所有 Score 都能直接变成 reward；
- 开发集、训练反馈集、选择集和独立留出集怎样分工；
- 为什么改进循环可以快，而发布 Gate 必须保持独立。

贯穿案例

shipping 的金额 100 失败可进入训练难例：对于 DPO，可构造“免运费”正确输出优于“收 10 元”错误输出的偏好对；对于 GRPO/RFT，可用确定性规则产生 0/1 reward。但发布评测不能只保留 99、100、101 这三个已被优化器反复看到的样本。应另外冻结未参与训练的边界组合、数据类型和异常路径，让 Candidate 在新运行中独立接受同一业务不变量检查。

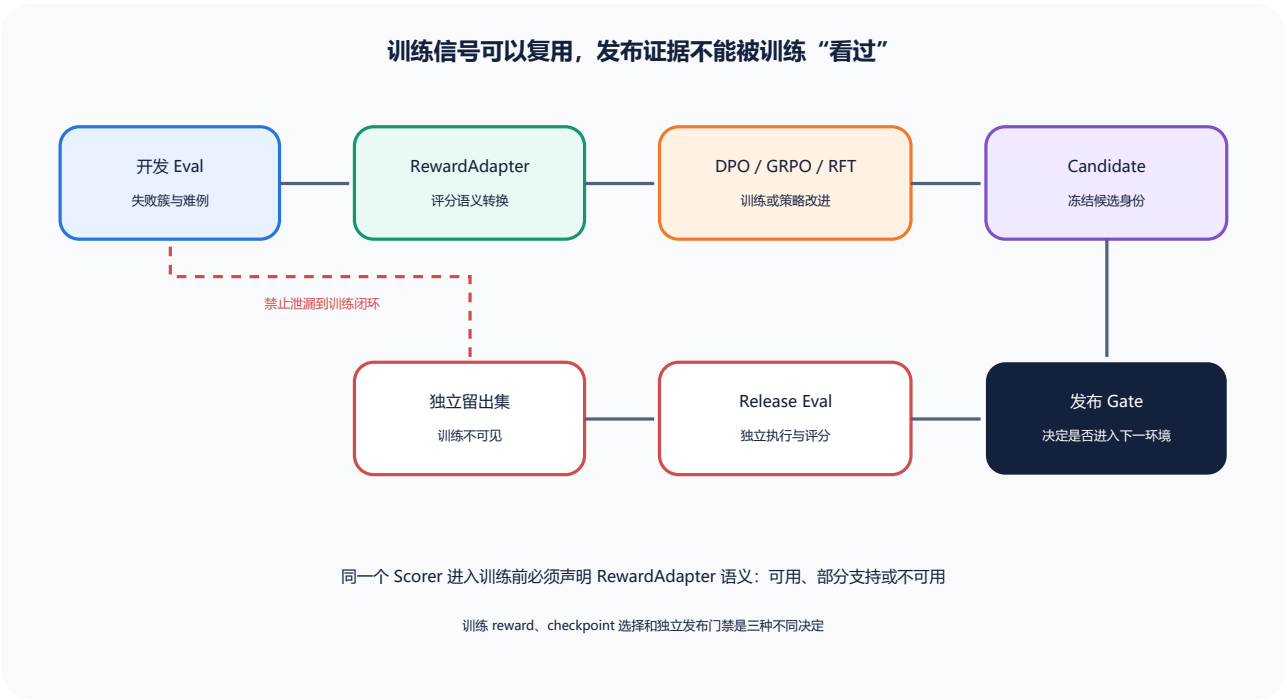
核心概念与边界

Evaluator 产生带血缘的 Score 和失败簇。**RewardAdapter** 明确把评测语义转换为训练接口：标量 reward、偏好对、验证器结果或不可用。**DPO** 消费偏好数据；**GRPO/RFT** 常消费可验证或模型生成轨迹上的奖励。

checkpoint choice eval 在训练过程中选择候选，允许影响优化路径。**independent release eval** 使用隔离留出集和独立运行，在候选冻结后决定是否进入下一环境。

RewardAdapter 必须声明能力合同。确定性测试通过可安全映射为验证器 reward；Judge 的 uncertain 不应硬压成 0/1；需要跨样本聚合的公平性 Metric 不能无解释地分摊到每条轨迹；带人工隐私数据的 Score 也可能禁止进入训练。不可表达时应写 unavailable，部分表达时写 partial。

机制图



调用链与状态变化

- 1 开发 Eval 保存 Trial、Observation、Score 和失败原因；Failure Miner 按错误机制聚类，而不是只挑最低分。
- 2 RewardAdapter 检查 Scorer 身份、证据许可、粒度和不确定状态，输出偏好、标量 reward 或拒绝转换。
- 3 DPO/GRPO/RFT 使用这些数据更新模型或策略。训练日志与 reward 版本被保存，但不成为发布证据。
- 4 Checkpoint Eval 在开发/选择 Split 上比较多个候选，选出一个冻结 Candidate 身份。
- 5 独立 Release Eval 在训练不可见的留出集和干净环境重跑 Candidate，对照 Baseline 与 Gate Policy。
- 6 只有独立 Gate 的证据进入发布报告；生产事件随后可以回流为下一轮开发数据，但不能改写历史 Gate。

关键数据结构

对象	需要的字段	主要风险
RewardAdapterContract	scorer_id、输出类型、状态映射、限制	语义错误转换
PreferencePair	prompt、chosen、rejected、来源 Score	偏好泄漏或伪标签
VerifierReward	Trial/Attempt/Bundle、reward、verifier 版本	奖励黑客
SplitManifest	entity group、用途、版本、授权	样本或实体泄漏
ReleaseEvalSpec	Candidate 身份、独立 Dataset、Gate	训练后调阈值

设计取舍

复用同一个 Scorer 能保持目标一致，却也会放大奖励黑客风险；因此 Release Eval 应增加独立切面、不同实现或人工抽查。完全隐藏留出集降低泄漏，但维护成本高且反馈慢；可采用开发集快速迭代、周期性刷新独立集。自动 reward 规模大，人工偏好能覆盖细腻语义，两者都需要来源与授权记录。

OpenAI Evals 的锁定 `eval.py` 展示样本 Eval 抽象，是**上游源码事实**。把 Score 经过 `RewardAdapter` 连接到 DPO/GRPO/RFT 是本仓库的**架构机制解释**；OpenAI Evals 该文件本身不提供这里描述的训练适配器。

失败语义

- Judge uncertain 被强制转成负 reward：Adapter 语义无效，应拒绝导出。
- 同一客户的近重复样本跨训练与留出集：实体泄漏，Release Eval invalid。
- 训练循环根据留出结果多次调参：留出集转化为开发集，必须更换独立集。
- checkpoint 在开发集提升、独立 Gate 失败：有效的负面发布结论，不得用训练 reward 覆盖。
- Reward 很高但真实终态不满足：奖励黑客，需要环境事实 Scorer 和新回归样本。

动手实验

从 shipping 的 `report.json` 选出 failed Score，沿 `observation_bundle_digest` 找到金额 100 的输出与 expected。写一张表，分别判断它能否转换为：DPO 偏好对、GRPO 标量 reward、独立发布 Score。再把一个 unscorable Score 放进同一张表。

预期输出与答案

明确错误且有确定性 Reference 的金额 100 样本可生成 `chosen=fee:0、rejected=fee:10`，也可映射为 0 reward；其来源血缘必须保留。它仍不能作为独立发布证据，因为已进入训练。unscorable 不能映射为失败 reward，应标记 unavailable 并修复观测链。独立发布 Score 必须来自冻结 Candidate 在未参与训练的 Dataset 上的新 Trial。

常见误解

“Eval 分数天然就是 reward” 忽略粒度和状态；“只要划分 train/test 就不会泄漏” 忽略同实体、近重复和反馈循环；“Release Eval 越晚越独立” 不成立，若结果持续指导调参，它已经是开发集；“RL 提升 reward 就证明产品更好” 把优化目标当外部有效性。

如何核对

从任一训练样本反查原 Score、Bundle 和 Scorer 版本，确认 `RewardAdapter` 的每个状态映射有测试。检查 `SplitManifest` 是否按实体分组，确认独立集没有进入训练、prompt 选择或阈值调节。Reference Harness 当前只提供 Eval 证据，不实现训练，这是有意的边界而非遗漏声明。

与其他 Harness 的关系

主流 Eval Harness 通常聚焦评测执行、评分或 CI；训练框架聚焦优化。二者之间最重要的是显式 Adapter，而不是共享一个数据文件。Agent Environment Harness 还能提供可验证终态 reward；LLM-as-Judge Harness 更需校准与人工仲裁。无论工具组合怎样，独立 Release Eval 都应保留自己的 Dataset、运行和 Gate 血缘。

本篇不能证明什么

本篇没有实现 DPO、GRPO 或 RFT 训练，也不声称某种算法必然提升真实质量。它给出的是证据隔离合同：训练信号可以源自 Eval，但用于发布的结论必须来自训练之外的独立评测。

[上一章](#) · [下一章](#)

lm-evaluation-harness 源码课程：Task 怎样变成批量模型请求

[上一节](#) · [下一节](#)

本篇要解决什么问题

lm-evaluation-harness 常被理解为“跑 benchmark 的命令行工具”，但真正值得学习的是它怎样把不同 Dataset、prompt 方式和 metric 统一成少数模型请求类型。只看 CLI 参数会错过三个关键问题：Task 何时把一条文档拆成多个请求；Model Adapter 为什么不用理解具体 benchmark；逐样本值又怎样重新组合并聚合成任务结果。

本课程以锁定提交 ffb2f7b0dfbb05a8095b04947a15cc0a70d54c66 为准，不追随浮动 main。我们把源码分成装配、请求构造、执行回填、样本处理和聚合五站，并指出它与本仓库 Trial/Attempt/Observation 语义不完全等价的地方。

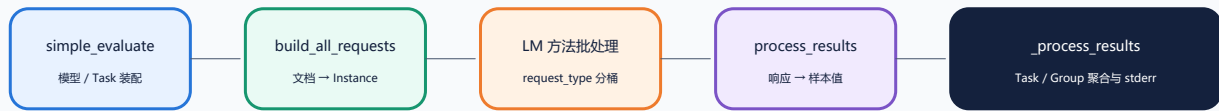
先建立源码地图

站点	锁定文件	主要责任
高层入口	evaluator.py	simple_evaluate 装配模型、Task、缓存与运行配置
Task 抽象	api/task.py	文档、上下文、Instance、过滤、逐样本 metric 与聚合函数
Model Adapter	api/model.py	loglikelihood、rolling likelihood、generation 与缓存代理
请求对象	api/instance.py	连接文档、请求参数、重复响应和过滤响应
聚合	evaluator_utils.py	Task/Group metric、stderr、配置和样本数整理

这些是**上游源码事实**。把它们映射成“Planner—Target Adapter—Scorer—Metric”是本课程的**机制解释**；上游并没有采用 Reference Harness 的全部对象或状态枚举。

完整调用链

从命令参数到任务指标：两层入口、四段数据变形



核中间对象：Instance

request_type 决定调用 LM.loglikelihood / loglikelihood_rolling / generate_until
 doc_id + idx 把多个请求重新组回原样本；repeats 控制同一请求的多次响应
 resps 保存原始响应，filtered_resps 保存过滤后交给 Task.process_results 的值

Task 同时负责“把文档变成请求”和“把响应变成样本 metric”；Model Adapter 只负责请求类型

- 1 CLI 最终调用 simple_evaluate。若 model 是字符串，Registry 创建 LM 子类；若启用缓存，再用 CachingLM 包装。
- 2 TaskManager.load(tasks) 返回 tasks、groups 等结构。入口覆盖 generation kwargs、few-shot 数和随机种子，并可运行 Task 完整性测试。
- 3 simple_evaluate 调用较低层 evaluate(lm, task_dict, ...)。后者为每个 Task 调用 build_all_requests，把评测文档变成 Instance。
- 4 evaluate 按 Instance.request_type 分桶，复制 repeats，必要时为分布式 rank 补齐，然后通过 getattr(lm, reqtype) 批量调用 Model Adapter。
- 5 响应按顺序追加到 Instance.resps。Task 运行 filter 后，源码按 doc_id 重新分组、按 idx 排序，把过滤响应交回 task.process_results。
- 6 每个文档得到的 metric 值进入 raw_metrics[(metric, filter)]；_process_results 先做 Task 聚合和 stderr，再自底向上做 Group 聚合。
- 7 高层入口补充模型、batch、device、seed、git hash、日期、环境与 tokenizer 信息，返回结果字典。

关键数据结构

Instance 是理解全链路的钥匙。它不是 Dataset Sample，也不是一次独立进程运行；它是 Model Adapter 的一条请求：

```

Instance
request_type  loglikelihood / loglikelihood_rolling /
               generate_until / multiple_choice
doc           原 Task 文档
arguments     交给 LM 方法的参数
task_name, doc_id, idx
repeats       同一请求需要几次响应
resps         原始响应
filtered_resps filter 后交给 process_results 的响应
  
```

一个多项选择文档可生成多个 loglikelihood Instance，它们共享 doc_id、使用不同 idx。于是“请求数”可能远大于“样本数”。上游结果还保存 doc/prompt/target hash，便于核对 sample log，但其血缘对象与本仓库

Observation Bundle 不是——对应。

实现取舍与失败语义

把所有模型约束到少数请求方法，使 Task 与后端解耦，便于批处理和缓存；代价是复杂 Agent 轨迹、工具副作用和环境终态不自然地落在这些接口里。Task 同时负责请求构造、filter、逐样本 metric 和 aggregation，扩展集中但责任较重。自定义 metric 没有 aggregation 时，源码会警告并 fallback 到 mean；这只是容错，不证明 mean 符合该 metric 的统计语义。

Task 被标为 unsafe code 且未显式确认时，evaluate 拒绝运行；多模态 Task 与非多模态 LM 不兼容时也提前失败。分布式 rank 请求数不等时会补齐调用。Model 请求错误如何恢复主要取决于具体 Adapter；核心 evaluate 的 repeats 是采样/重复响应机制，不应直接当作基础设施 Attempt。

动手实验

不下载模型也能验证对象变形。打开锁定源码的 tests/test_aggregation_pipeline.py，找到一个含 4 个 raw acc 值的测试，手工计算 Task 和单 Task Group 的结果。然后在本仓库运行：

```
python scripts/sources.py verify
python -m pytest tests/test_harness_course_docs.py -q
```

第二步不是运行上游模型，而是确认课程每条源码链接都落在锁定范围、正文和图示满足学习合同。

预期输出与答案

[1, 0, 1, 0] 经过 mean 得到 0.5；单 Task Group 仍为 0.5，sample_len 为 4。若一个文档有四个候选选项，它通常形成四个 loglikelihood Instance，但最后只产生一组文档级 metric。请求数不能直接作为准确率分母。

sources.py verify 应报告 8 个来源锁文件有效；课程测试应通过。若上游链接使用 main 而不是 40 位 commit，或路径不在锁定 scope，测试应失败。

如何核对

从 evaluator.py 搜索 build_all_requests，继续追到 getattr(lm, reqtype)、req.resps.append、process_results 和 _process_results。再读 api/instance.py，验证 doc_id、idx、repeats、resps 的角色。不要只引用 README 的概念描述。

本篇不能证明什么

锁定调用链不能证明某个 benchmark 代表真实业务、某个模型排名可靠，也不能证明上游适合 Agent 环境评测。它解释的是该提交如何组织 benchmark 型评测；数据有效性、Adapter 网络恢复和发布 Gate 需要另行审查。

[上一节](#) · [下一节](#)

01 | 入口与 Task 加载: simple_evaluate 实际装配了什么

上一节 · 下一节

本篇要解决什么问题

很多调用示例只展示 `simple_evaluate(model=..., tasks=...)`，容易让人以为它就是执行循环。源码中它更像装配层：解析模型、加缓存、加载 Task/Group、覆盖 Task 配置、记录实验身份，然后把真正执行交给 `evaluate`。分清两层入口，才能知道一个配置是在运行前固定，还是在样本循环中动态变化。

先建立源码地图

主要证据位于锁定 `evaluator.py`。 `simple_evaluate` 接收模型字符串或已初始化 LM、Task 名称、few-shot、batch、device、cache、seed、chat template 和日志选项。它调用外部 `TaskManager.load`，但本课程把 Manager 的内部扫描留到扩展节，先专注加载结果怎样进入核心循环。

Model Adapter 抽象在 `api/model.py`：LM 规定请求方法、rank/world_size 和分布式原语，CachingLM 代理特定请求方法并按参数哈希缓存结果。Task 抽象在 `api/task.py`。

完整调用链



- 1 `simple_evaluate` 设置 Python、NumPy、Torch 和 few-shot 随机种子。这些种子用途不同，不能只保存一个 seed 字段。
- 2 模型参数若是字符串，Registry 找到 Model 类并调用 `create_from_arg_string`；传入对象则必须是 LM 子类，否则抛出 `TypeError`。
- 3 开启请求缓存时，以 rank 区分 SQLite 文件，再用 CachingLM 包住真实模型。非确定性 generation 参数会影响是否缓存。

- 4 若调用者未传 TaskManager, 入口创建一个; load(tasks) 返回任务、组和层级信息。
- 5 对每个 Task, 入口覆盖 generation kwargs、predict-only metric、num_fewshot 和 few-shot seed。Task 显式配置 0-shot 时不会被命令参数强行改成别的值。
- 6 check_integrity 为真时执行 Task 测试; EvaluationTracker 存模型来源、参数、system instruction 和 chat template。
- 7 调用 evaluate 后, 只有主 rank 构建最终结果, 并补充实际模型信息、batch、device、各类 seed、git hash、日期、环境和 tokenizer 信息。

关键数据结构

加载结果不是简单 dict[str, Task], 而是至少含 tasks 与 groups 的结构。Task 配置包含 output_type、few-shot、generation_kwargs、metric 与 filter 等。高层结果的 config 同时保存逻辑模型名和运行参数; 若 LM 暴露 get_model_info, 还会合并更具体的模型信息。

从可复现角度看, 字符串 model="hf" 只是 Adapter 类型, model_args 才可能包含 checkpoint; 服务端 API 仍可能把逻辑名字解析到不同实际版本。因此本仓库会把这种配置视为 Target Identity 的一部分, 而不把它等同于完整实际身份。

实现取舍与失败语义

高层入口集中处理兼容性和用户便利, 使 evaluate 能假定收到已初始化 LM 与已加载 Task。代价是配置覆盖发生在对象加载后, 读者必须追踪最终 Task config, 不能只看 YAML。predict-only 通过覆盖 metric 为 bypass 是源码自己标注为较 hacky 的做法; 这提醒我们“只生成输出”和“有可解释 Score”是两种运行模式。

模型类型不满足 LM 协议会在运行前失败; Task 完整性测试失败也应阻断。只有 rank 0 返回结果并写文件, 避免多进程重复输出。缓存文件按 rank 分开解决并发写入, 却不自动证明不同机器、tokenizer 或浮动模型身份的缓存可互换。

动手实验

阅读 simple_evaluate 的参数表, 为 shipping Reference Harness 写一张映射表: model/model_args 对应 TargetSpec, tasks 对应 Task/Dataset 选择, limit 对应计划抽样, bootstrap_iters 对应统计配置, use_cache 对应执行优化。对每一项标注“完全对应、部分对应、不等价”。

然后执行:

```
python scripts/sources.py links
```

确认输出中的 evaluator.py 链接包含锁定 commit, 而不是 main。

预期输出与答案

model/model_args 只能**部分对应** TargetSpec, 因为实际服务身份可能需要运行后调和; tasks 也只部分对应 Task/Dataset, 因为 Task 对象同时含 prompt 和 metric; limit 不是随意截断结果, 而应在运行前进入 Trial/Sample 计划; bootstrap_iters 是聚合不确定性参数; use_cache 是执行优化, 不应改变逻辑输出。

链接命令应列出 evaluator.py、api/task.py、api/model.py、api/instance.py 与 evaluator_utils.py 的永久 URL。

如何核对

在 `evaluator.py` 中按顺序定位 `get_model(...).create_from_arg_string`、`CachingLM`、`task_manager.load`、`Task` 配置覆盖、`evaluate()` 和结果 `config`。再到 `api/model.py` 核对缓存仅代理哪些请求类型以及 `generation` 采样为何可能禁用缓存。

本篇不能证明什么

保存入口参数不能证明模型服务端实际版本、`Dataset` 文件内容或网络环境完全相同。它提供了强于命令行截图的运行身份，却仍需要内容摘要、实际身份返回和 `Artifact` 血缘补足。

[上一节](#) · [下一节](#)

02 | 请求执行：Instance 怎样分桶、批量调用并回填

上一节 · 下一节

本篇要解决什么问题

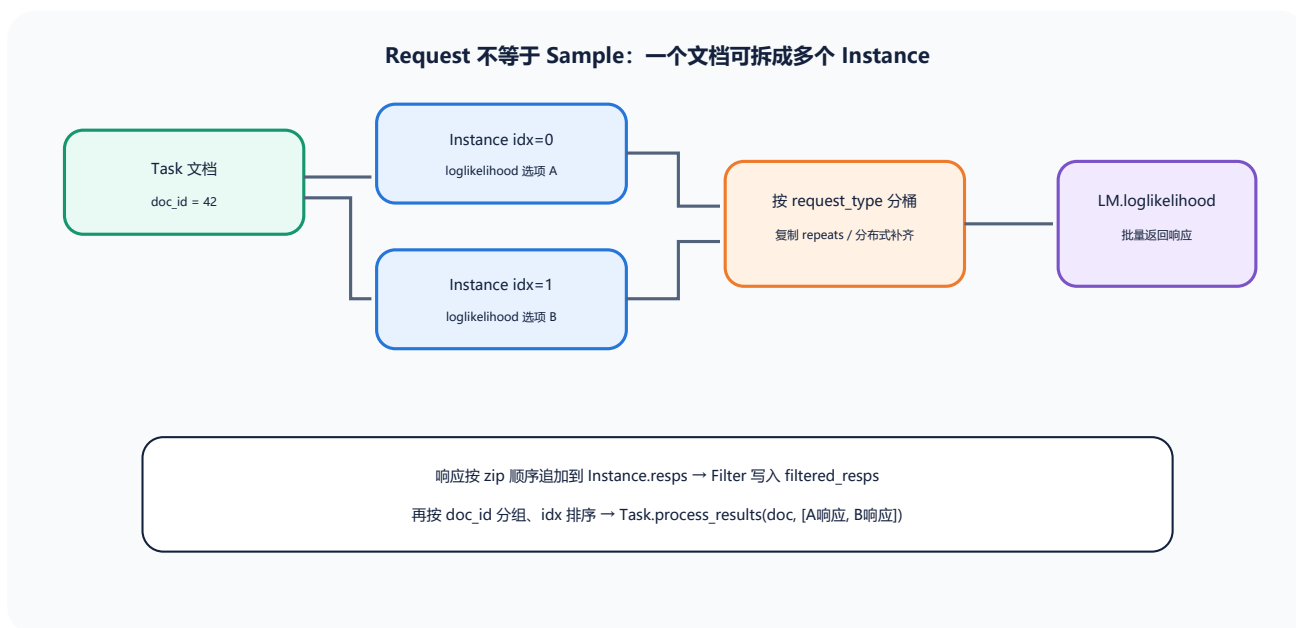
一条 benchmark 文档并不一定对应一次模型调用。多项选择题可能为每个选项构造一条 loglikelihood 请求，生成题可能只有一条 generate_until，请求还可能因 repeats 被复制。若把“模型请求数”“文档数”和“统计样本数”混在一起，就会误算成本、分母和重试语义。本节沿 evaluate 的核心循环追踪 Instance 的完整生命周期。

先建立源码地图

调度循环位于锁定 evaluator.py，Task 构造逻辑与各 output_type 的处理位于 api/task.py，Instance 字段位于 api/instance.py，Model 请求协议位于 api/model.py。

OutputType 包含 loglikelihood、loglikelihood_rolling、generate_until 和 multiple_choice。源码注释说明 multiple_choice 在调度时会分派成若干 loglikelihood 请求，因此 output_type 与最终 LM 方法名也不总是一字不差。

完整调用链



- 1 evaluate 为每个 Task 计算 limit 或指定 sample indices，再调用 task.build_all_requests，传入 rank、world_size、cache、system instruction 和 chat template 等上下文。
- 2 Task 遍历评测文档、构造上下文并创建 Instance。metadata 在 __post_init__ 中拆为 task_name、doc_id 和 repeats；idx 表示同一文档内请求顺序。
- 3 调度器遍历 task.instances，按 request_type 放入全局 requests 分桶。分布式模式先收集各 rank 数量，为较短 rank 计算 padding_requests。
- 4 每个分桶内按照 req.repeats 复制对象；分布式补齐再增加伪请求，使各 rank 前向批次数一致。

- 5 `getattr(lm, reqtype)(cloned_reqs)` 调用对应 Model Adapter 方法。返回列表与 `cloned_reqs` 用严格 zip 对齐，每个响应追加到原 Instance 的 `resps`。
- 6 Task 执行 filter，生成 `filtered_resps`。调度器按 `doc_id` 分组、按 `idx` 排序，恢复“同一文档的多条请求”顺序。
- 7 `task.process_results(doc, filtered responses)` 把多个响应变为文档级 metric。可选 sample log 同时记录 arguments、原始响应、过滤响应和 `doc/prompt/target hash`。

关键数据结构

下面三种基数必须分别记录：

基数	由什么决定	用途
eval docs	Task Dataset 与 limit/samples	文档级结果的候选分母
Instance	每个文档的请求构造	模型调用与 batching
cloned requests	Instance.repeats 与分布式 padding	实际 Adapter 调用数量

`resps` 是所有重复返回，`filtered_resps` 是 filter pipeline 的命名输出。`process_results` 接收到的是某个 filter_key 下、同一 doc 的过滤响应列表。于是 filter 不是报告格式化，而是进入评分前的数据变换；改变 filter 可能改变 metric。

实现取舍与失败语义

按 request_type 全局分桶能让不同 Task 的同类请求共同批处理，充分利用模型吞吐；代价是文档顺序被暂时打散，必须依靠 `doc_id/idx` 正确重组。把 repeats 实现为同一个 Instance 引用的复制，节省对象构造，但要求响应追加顺序绝对稳定。严格 zip 可以检测响应数量不一致，却不能检测 Adapter 返回顺序错误。

分布式 padding 是计算同步需要，伪请求不应进入最终文档 metric。chat template 与 tokenizer identity 参与请求构造缓存键，否则旧请求可能在新模板下被错误复用。unsafe Task 与多模态不兼容会在构造前阻断。具体 LM 网络超时、速率限制和内部 retry 由 Adapter 承担，核心 Instance 没有本仓库 Attempt 的显式失败状态。

动手实验

设想一个四选一文档，Task 为四个选项各构造 loglikelihood Instance，metadata 的 `doc_id` 都是 7，`idx` 为 0..3，`repeats=1`。写出 requests 分桶和 LM 返回 `[(-1.2, true), (-0.4, true), (-2.1, true), (-3.0, true)]` 后的 `resps`。再假设 `repeats=2`，回答文档级统计单位是否变成两个。

你也可以直接运行本仓库的静态课程合同：

```
python -m pytest tests/test_harness_course_docs.py -q
```

预期输出与答案

四条 Instance 都进入 loglikelihood 分桶。响应依次进入各自 `resps`，按 `doc_id=7` 分组、`idx` 排序后，`process_results` 能选择最大 loglikelihood 的选项 B。`repeats=2` 时会有八次响应，但是否形成两个统计观测取决于 Task 如何处理重复；不能仅因 cloned request 增加就把文档分母改成 2。

课程测试应确认本节有图、完整章节、锁定链接和上下导航。它不执行上游模型，所以不会下载 checkpoint 或访问模型 API。

如何核对

在 `evaluator.py` 依次搜索 `requests = defaultdict`、`build_all_requests`、`cloned_reqs`、`getattr(lm, reqtype)`、`req.resps.append`、`instances_by_doc_id` 和 `process_results`。再读 `api/instance.py`，确认 `args` 属性如何把非 `tuple` 参数包装成 `tuple`。

本篇不能证明什么

源码顺序与哈希日志不能证明 Model Adapter 没有服务端重试、响应来自声明模型，或每个 benchmark 文档统计独立。它只解释 Harness 核心怎样组织请求；真实身份、相关性和基础设施 Attempt 仍需外层证据。

[上一节](#) · [下一节](#)

03 | 评分、聚合与测试：从 process_results 到 Group 结果

上一节 · 下一节

本篇要解决什么问题

模型响应回填后还没有“准确率”。Task 需要把同一文档的若干响应解释成逐样本 metric，再为每个 metric 选择聚合函数，计算标准误，并可能把多个 Task 聚合为 Group。最容易被忽略的问题是：自定义 metric 没写 aggregation 会发生什么？有效样本数怎样计算？Group 为什么必须自底向上处理？测试又锁住了哪些语义？

先建立源码地图

逐样本处理和 aggregation 配置位于锁定 api/task.py。核心循环把结果加入 raw_metrics 的代码位于 evaluator.py。Task 与 Group 聚合、stderr、结果容器位于 evaluator_utils.py。

源码中的 process_results 名字容易误导：它处理的是一个 doc 的结果，不是整次运行报告。返回字典里的 value 才逐条积累，随后 aggregation 才产生 Task metric。

完整调用链



- 1 filter 完成后，调度器为每个 doc 调 task.process_results(doc, responses)，得到 {metric_name: sample_value}。
- 2 每个值追加到 raw_metrics[(metric, filter_key)]。若启用 log_samples，同一记录保存 doc、target、arguments、resps、filtered_resps 与三个 hash。
- 3 _process_results 调 _collect_results；后者对每个 Task 调 _compute_task_aggregations。
- 4 对每个 metric/filter，代码查询 task.aggregation()[metric]。找不到时记录 warning 并 fallback 到 mean。

- 5 聚合函数作用于 items，结果键写成 metric,filter。若 bootstrap_iters > 0，再由 stderr_for_metric 选择可用标准误函数；样本不足或不支持时写 N/A。
- 6 _collect_results 保存 Task config、version、num_fewshot、higher_is_better、logged samples，以及 original/effective n_samples。
- 7 aggregate_groups 以 post-order 收集 Group，确保子组先于父组，再调用每个 Group 自己的 aggregate。

关键数据结构

```
eval_results_acc[task] = {
    task,
    raw_metrics: {(metric, filter): [sample values]},
    logged_samples: [...]
}

EvalAcc
metrics / configs / versions / num_fewshot
higher_is_better / samples / n_samples / groups
```

锁定源码中 sample_len 在遍历 raw_metrics 时被设置为当前 items 的长度，注释明确提示它当前反映最后一个 metric 的 count。如果不同 metric 因缺失产生不同长度，只看单个 sample_len 可能隐藏分母差异。这是可直接核对的实现边界，也是本仓库坚持每个 Metric 显式 denominator 的原因。

实现取舍与失败语义

Task 自带 aggregation 能让 metric 定义靠近任务语义，支持 mean 以外的函数；fallback mean 提高扩展容错，却可能把本应成对、加权或非线性的 metric 错聚合。Bootstrap stderr 提供基础不确定性，但重采样单位是 items；若多个 doc 来自同一实体或同一 prompt 模板，独立同分布假设需要额外审查。

Group 使用后序遍历处理嵌套层级，避免父组在子组结果尚未生成时聚合。higher_is_better 需要跨子任务传播，方向冲突不能偷偷选一个。有效样本与原始样本同时保存是优点，但它不是预声明 Trial Plan；limit、过滤和缺失仍需读者核对。

动手实验

手工计算以下两组 Task：T1 有 2 个样本，acc 为 [1, 0]；T2 有 4 个样本，acc 为 [1, 1, 1, 0]。Group 若 weight_by_size=true，求结果；若先对两个 Task metric 做不加权 mean，再求结果。解释为什么二者不同。

再查看上游 tests/test_aggregation_pipeline.py 的单 Task、双 Task 加权、嵌套 Group 和错误路径测试名称，列出你会为自定义 aggregation 新增的最小测试。

预期输出与答案

T1=0.5，T2=0.75。按样本数加权为 $(2 \times 0.5 + 4 \times 0.75) / 6 = 2/3$ ；Task 等权 mean 为 0.625。哪一个正确取决于 Group 声明的 estimand，不能看到结果后再选。自定义 aggregation 至少要测正常输入、空/单样本、不同 filter、stderr 支持与 Group 组合。

上游测试里的单 Task Group 应保持 Task metric；双 Task weighted Group 应按 sample_len 权重；嵌套 Group 必须验证子节点先产生结果。

如何核对

从 `evaluator_utils.py` 依次阅读 `_compute_task_aggregations`、`_collect_results`、`aggregate_groups` 和 `_process_results`。特别核对 `fallback mean`、`bootstrap 上限特例`、`sample_len TODO` 与 `post-order traversal`。再回到 `api/task.py` 查看 `ConfigurableTask` 怎样注册 `aggregation` 和 `higher_is_better`。

本篇不能证明什么

聚合测试通过不能证明 `benchmark` 的采样有效、`stderr` 覆盖真实不确定性，也不能把任务排行榜当发布 Gate。它证明的是锁定实现中逐样本值如何变成 `Task/Group` 输出；独立性、外部有效性和非补偿风险政策仍需外层设计。

[上一节](#) · [下一节](#)

Inspect AI 源码课程：把 Solver、Sandbox、Scorer 和 EvalLog 放进同一条链

[上一节](#) · [下一节](#)

本篇要解决什么问题

Inspect AI 不只是“另一套 benchmark runner”。它把 Task 定义为 Dataset、Solver、Scorer 与可选 Sandbox 的组合，能运行普通模型任务，也能运行带工具和状态的 Agent Eval。真正需要读懂的是：公共 eval 怎样解析 Task；eval_run 怎样为每项 Task 准备隔离环境与日志；task_run 怎样执行每个 Sample；Scorer 又怎样读取 TaskState 和 Target，把结果写回结构化 EvalLog。

本课程固定在提交 `ebf4815ee260afcc8c34ad9d66e6f8d98a89e905`。正文把上游直接可见的类和调用标为源码事实；把它们映射到 Trial、Attempt、Observation 和 Gate 时会明确这是机制解释，不假设字段——相同。

先建立源码地图

站点	锁定源码	责任
公共入口	<code>_eval/eval.py</code>	同步/异步 API、Task 解析、模型与运行上下文
Task 调度	<code>_eval/run.py</code>	TaskRunOptions、SandboxManager、并发与 Task retry
单 Task 执行	<code>_eval/task/run.py</code>	Sample 生命周期、Solver Plan、Scorer 与日志更新
Scorer 协议	<code>scorer/_scorer.py</code>	Scorer(state, target)、注册、重建规格与 Metric 元数据
日志模型	<code>log/_log.py</code>	EvalSpec、EvalSample、EvalPlan、EvalResults、EvalLog

完整调用链

Task 把 Dataset、Solver、Scorer 与 Sandbox 组合成一次可记录 Eval



单个 Sample 内部

创建/选择 Sandbox → 复制 files 与 setup → 初始化 TaskState → 依次运行 Solver Plan
 模型与工具事件写入样本事件流 → Scorer(state, target) 产生 Score → 保存 usage 与 error
 多 epoch 可由 Reducer 合并; Task 级 Metric 记录 scored_samples 与 unscored_samples

Task retry 是整项 Task 调度恢复; Sample error、score_on_error 和 cancel 类型必须分开理解

- 1 eval 作为同步包装进入 eval_async。后者建立运行上下文，解析模型、Task 来源、审批和日志位置。
- 2 eval_resolve_tasks 把 Registry 名称、文件或 Task 对象解析成 ResolvedTask；同一入口还支持 TaskSource 动态注入。
- 3 eval_run 创建 SandboxManager，为每个 ResolvedTask 合并调用参数与 Task 默认值，解析 Solver、Scorer、Reducer，并生成 TaskLogger 与 TaskRunOptions。
- 4 run_task_retry_attempts 调度多项 Task，按模型平衡并发。Task 出错或显式请求 retry 时可重新排队；已写出的失败日志仍保留，而不是被成功结果悄悄覆盖。
- 5 _run_task 调用 task_run。后者为 Dataset Sample 建立 TaskState，在 Sandbox 中运行 setup 和 Solver Plan，记录模型、工具、状态与错误事件。
- 6 Scorer 是异步 callable，接收最终 TaskState 与 Target。多个 Scorer 可分别产生 Score，跨 epoch 结果还可交给 Reducer。
- 7 EvalLog 汇总 EvalSpec、EvalConfig、EvalPlan、样本事件、Score、Metric、usage、stats 和 error，形成可查看与可重评分的运行证据。

关键数据结构

EvalSample 保存 id、input、target、metadata、sandbox、files、setup、messages、output、scores、events、model_usage 与 error。EvalPlan 保存 Solver steps。EvalScore 保存 Scorer 名、参数、Metric、scored_samples 和 unscored_samples。EvalLog 再把 spec、plan、results、samples、stats 和 status 组合起来。

这些对象比只含最终准确率的报告丰富，但并不自动等同内容寻址证据。日志可裁剪 explanation 和 metadata，样本也可能因配置不写入；因此下游 Scorer 或审计器仍需知道实际记录级别。

实现取舍与失败语义

把 Sandbox 与 Task 并列是一项关键设计：同一 Solver 可以在本地、Docker 或其他环境实现中运行，Agent 终态也能进入评分。代价是环境创建、文件复制、setup、cleanup 和网络策略成为运行协议的一部分。Scorer 必须是 async callable，便于 Judge 或外部检查并发，但其可复现性取决于模型和参数是否完整记录。

Task retry 与 Sample 评分失败不同。run_task_retry_attempts 针对整项 Task 的错误或 cancel/retry 请求重新调度；score_on_error 决定 Sample 执行异常后是否仍评分；普通 Score 不通过则是有效评测结果。把这三者都叫“失败重试”会破坏统计解释。

动手实验

从锁定 log/_log.py 列出 EvalSample 中属于四类的字段：输入身份、执行轨迹、评分结果、资源/错误。然后对 shipping Reference Harness 的 evidence.json 做同样分类，找出 Inspect AI 多记录了什么、本仓库多强调了什么。

运行：

```
python scripts/sources.py verify
python -m pytest tests/test_harness_course_docs.py -q
```

预期输出与答案

输入身份至少有 id/input/target/metadata；执行轨迹有 sandbox/files/setup/messages/output/events；评分有 scores；资源与错误有 model_usage、total_time 与 error。Inspect AI 的模型和 Sandbox 语义更丰富；Reference Harness 显式绑定 canonical Attempt、Artifact digest 和计划 Trial 分母。二者可以互补，不能只按字段数量判断优劣。

来源锁应验证 8 项；在本课程四篇未全部完成前，课程合同会保持失败，这正是防止只提交一篇概览冒充完整课程。

如何核对

从 _eval/eval.py 搜索 eval_async、eval_resolve_tasks 与 eval_run；到 _eval/run.py 搜索 TaskRunOptions、run_task_retry_attempts；最后到 _eval/task/run.py 追 task_run。每一跳都记录调用者、输入、状态变化和返回值。

本篇不能证明什么

结构化 EvalLog 不能证明 Sandbox 完全隔离、Judge 有效或 Dataset 代表线上任务。课程也不声称 Inspect AI 的 Task retry 等于本仓库 Attempt；只有明确失败分类和统计计划后才能建立可靠映射。

[上一节](#) · [下一节](#)

01 | Eval、Task 与 Solver: 从公共 API 到可执行 Plan

上一节 · 下一节

本篇要解决什么问题

Inspect AI 的 `eval()` 参数很多: 模型、Task、Sandbox、Solver、epochs、并发、预算、日志与 `retry` 都能在入口覆盖。若只把它看成一个大函数, 就看不出哪些值属于 Eval 级调度, 哪些会进入每个 Task, 哪些最终改变单个 Sample 的 Solver Plan。本节沿同步 API、异步上下文、Task 解析和 `TaskRunOptions` 四层拆开这条装配链。

先建立源码地图

公共同步与异步入口、模型和 Task 解析在锁定 `_eval/eval.py`。Task 级准备和调度在 `_eval/run.py`。Solver Plan 的解析与单 Task 主循环在 `_eval/task/run.py`。

源码直接显示 `eval_async` 同时只能有一个活动调用, 并通过 `anyio TaskGroup` 运行 `_eval_async_inner`。这属于锁定版本的**源码事实**, 不应推广为所有版本或所有部署模式的永久限制。

完整调用链



- 1 `eval` 处理同步显示与异常边界, 再调用 `eval_async`。后者规范化 checkpoint 与 control server 参数, 创建 TaskGroup。
- 2 `_eval_async_inner` 解析 `model_args/task_args`、成本表与日志配置, 生成 `run_id`, 并初始化模型、并发限制和运行 hook。
- 3 `eval_resolve_tasks` 在已初始化的模型/角色上下文中解析 Task。输入可以是 Registry 名称、文件、Task 对象或 TaskSource; 每个逻辑 Task 与实际 Model 组合成 ResolvedTask。

- 4 入口调用 `eval_run`。该函数创建 `SandboxManager`，并为每个 `ResolvedTask` 合并 Eval 参数与 Task 默认配置。调用级 Solver 可以覆盖 Task 自带 Solver，但覆盖结果必须进入日志规格。
- 5 `resolve_plan` 把单 Solver、Chain 或 Plan 统一为 Plan；Task 的 setup Solver 会被复制后前置，避免重复调用在原对象上不断叠加 setup。
- 6 Scorer 被转成可重建的 `ScorerSpec`，Reducer、审批、限制、日志与 `SampleSource` 一起进入 `TaskRunOptions`。
- 7 `run_task_retry_attempts` 以 Task × Model 为调度单位，在并发上限内尽量平衡不同 Model，最终把每项 Task 交给 `task_run`。

关键数据结构

`ResolvedTask` 表示 Registry/File/对象解析后的 Task 与来源；`TaskRunOptions` 则是一次 Task 执行所需的冻结装配结果，包含 Task、Model、Sandbox、EvalConfig、Solver、Scorer、Logger、SampleSource 和运行限制。Plan 是 Solver steps 的有序列表；每个 step 会被记录为 `EvalPlanStep`，便于日志重放“实际运行了哪些 Solver”。

Task 的 Dataset 与 Sample 是评测输入；Solver 改变 `TaskState`，包括 messages、output、tools 与 store；Scorer 在 Solver 结束后读取 state 与 Target。这个切分比把 prompt、模型调用和评分全塞进一个 callback 更容易审计，但 Task 对象仍是多项配置的聚合根。

实现取舍与失败语义

统一同步与异步入口降低使用门槛，anyio 便于并发 Task/Sample；全局单活动 `eval_async` 则限制同进程嵌套运行。调用参数可覆盖 Task 默认值带来实验便利，也意味着日志必须保存**解析后配置**，否则仅凭 Task 源码无法重建运行。

Solver 覆盖适合对同一 Dataset 比较不同策略，但它可能改变工具、消息和终止语义，不能只把 Solver 名当作普通超参数。Task 解析失败、模型角色缺失或 Sandbox 规格无效应在 Sample 前阻断；Solver 返回错误行为则是 Sample 级产品结果，不能自动解释为 Eval 基础设施错误。

动手实验

为同一个虚构退款 Dataset 设计两个 Solver：`direct_answer` 只生成一次，`tool_agent` 允许查询订单和调用退款工具。列出在调用级替换 Solver 时必须同步冻结的身份：Solver Registry 名与参数、工具集合、审批策略、Sandbox、Model、消息/turn/token/time/cost limits。

再从锁定源码查找 `resolve_plan`，解释 `Task.setup` 为什么要在复制后的 Plan 上前置，而不能直接修改复用的 Plan。

预期输出与答案

两种 Solver 不能只按最终文本比较：`tool_agent` 还会产生工具副作用和环境终态。至少应冻结 `SolverSpec`、Model、工具权限、审批和各项预算。`resolve_plan` 复制的原因是同一 Task/Plan 可能被多次 eval；若原地 prepend setup，第二次运行会重复执行 setup，实际 Plan 与声明不一致。

课程测试应继续保持红色，直到本课程另外两篇都存在并满足内容合同；这防止一张全局图取代 Sample 和 Scorer 的细节。

如何核对

在 `_eval/eval.py` 定位 `eval_async`、`_eval_async_inner`、`eval_resolve_tasks` 与 `eval_run`。在 `_eval/run.py` 查看 `TaskRunOptions` 的构造。最后在 `_eval/task/run.py` 核对 `resolve_plan` 对 `Solver`、`Chain`、`Plan` 和 `setup` 的分支。

本篇不能证明什么

完整记录 `Plan` 不证明 `Solver` 安全、工具权限最小或任务有效。它只能证明锁定实现怎样把声明解析成执行计划；实际模型身份、`Sandbox` 隔离和 `Scorer` 有效性需要后续证据。

[上一节](#) · [下一节](#)

02 | Sandbox 与 Sample: 一次 Agent Eval 怎样真正运行

上一节 · 下一节

本篇要解决什么问题

对普通问答, Sample 似乎只是 input 到 output; 对 Agent Eval, Sample 还可能复制文件、执行 setup、创建 Sandbox、暴露工具、产生多轮消息、达到 token/time/cost limit, 并在清理前检查终态。只看最终 output 会遗漏大量产品行为。本节追踪 task_run 如何把 Dataset Sample 变成 TaskState, 再如何在隔离上下文中运行 Solver 和保存 EvalSample。

先建立源码地图

Sample 生命周期集中在锁定 `_eval/task/run.py`。Task 级 `SandboxManager` 与并发准备在 `_eval/run.py`。日志侧 `EvalSample`、`SandboxEnvironmentSpec`、`EvalSampleLimit` 与事件容器在 `log/_log.py`。

这三个文件共同说明 Environment 不是 Target 的一个字符串参数, 而是具有启动、连接、限制、清理和记录语义的执行对象。

完整调用链



- 1 `task_run` 解析 Plan、Scorer 名称、epoch 与 Sandbox limits, 创建进度和结果容器, 并从 Dataset/SampleSource 取得待运行 Sample。
- 2 `create_sample_state` 把 Sample 输入、目标、Model、metadata 和初始消息装入 TaskState; 每个新运行具有独立 UUID, 避免 sample retry 复用错误状态。
- 3 `task_run_sample` 初始化 scoring context 和 Sandbox context。Sample 自己的 sandbox 可以覆盖 Task 默认; files、setup 与连接信息在环境初始化阶段处理。

- 4 进入 Sandbox 后, Solver Plan 依次运行。Generate 函数更新 TaskState, 模型调用、工具事件、store、消息和 output 进入 transcript/Event。
- 5 消息数、token、turn、wall time、working time 与 cost 等限制在运行期间检查。达到限制可形成 EvalSampleLimit, 并根据策略停止、评分或标错。
- 6 Solver 完成后运行 Scorer; 随后 Task cleanup 仍在 Sandbox context 内执行, 以便检查或清理环境。
- 7 make_eval_sample 从 state、events、scores、usage、error、limit 与 sandbox 信息构造 EvalSample。只有在清理和日志阶段完成后, Sample 才进入 Task 级聚合。

关键数据结构

TaskState 是运行中的可变视图, 包含 model、sample_id、epoch、input、messages、output、tools、store、metadata 与 scores。EvalSample 是日志中的持久化快照, 除输入输出外还记录 target、sandbox、files、setup、events、scores、model_usage、error、limit 和时间摘要。

SandboxEnvironmentSpec 说明环境类型与配置, 但真正的环境证据还包括连接、文件、命令结果和终态。

EvalSampleLimit 要区分是 token、message、turn、time、working time 还是 cost 限制, 因为这些停止原因对产品解释不同。

实现取舍与失败语义

每 Sample Sandbox 隔离最容易复现和清理, 却会增加启动成本; 复用环境更快, 但必须证明 reset 消除了跨 Sample 状态。Inspect AI 允许 Task/Sample 级 Sandbox 和并发上限, 给实现者灵活性; 教材将其视为能力, 不把“用了容器”直接等同于无污染。

retry_on_error 是 Sample 级错误重试, 递归重建新 TaskState 并保留 error_retries。它与 Solver 内部自我修正、Task 级 retry、普通 Score 失败不同。score_on_error 只在重试耗尽后允许错误 Sample 进入 Scorer; 这能评估部分完成, 但错误仍计入 fail_on_error, 不应被 Score 掩盖。操作员 cancel 的 score/error/abort/retry 也具有不同终止语义。

动手实验

设计一个“删除指定临时文件”的 Agent Sample: Sandbox 初态有 target.txt 与 keep.txt, Agent 只能调用 shell 工具, Scorer 检查终态。列出必须记录的观察: 初态摘要、每次工具调用、退出码、最终目录清单、目标文件是否缺失、保留文件是否仍在、Sandbox reset 结果。

再回答: Agent 最终文本写“已删除”但 target.txt 仍存在时, TaskState.output、EvalSample error、Score 分别应是什么。

预期输出与答案

最终文本只是 output; 若执行过程无异常, EvalSample 可以没有 error, 但环境事实 Scorer 应给 failed。不能因为 Agent 自述成功而通过。若 Sandbox 无法创建, Sample 是基础设施错误, 可按 retry_on_error 恢复; 若 Agent 执行 rm 返回权限错误, 这是被测行为证据, 是否产品失败由 Task/Scorer 契约决定, 不能默认当平台重试。

上述观察中, 终态目录与文件摘要应作为 Artifact 或结构化 Event 进入评分。reset 失败会污染后续 Sample, 应阻断后续运行, 而不是继续累积分数。

如何核对

在 `_eval/task/run.py` 搜索 `task_run_sample`、`sandboxenv_context`、`TaskState`(、`span("solvers")`)、`Scorer` 循环、`make_eval_sample` 和递归 `retry`。再到 `log/_log.py` 核对 `EvalSample` 字段，确认哪些观察能持久保存、哪些只存在运行内存。

本篇不能证明什么

有 `Sandbox` 与事件日志不证明环境无逃逸、`reset` 完整或所有副作用都被观察。真正的 Agent 发布评测仍需威胁模型、最小权限、外部服务隔离和独立终态断言。

[上一节](#) · [下一节](#)

03 | Scorer、EvalLog 与 Retry：怎样保留可重评分的运行事实

上一节 · 下一节

本篇要解决什么问题

Inspect AI 能保存完整 EvalLog，也支持 Sample retry、Task retry、score_on_error、多个 Scorer 和 epoch Reducer。功能丰富也带来解释风险：一次失败后重试的结果怎样进入日志？Scorer 身份能否重建？未评分样本是否从分母消失？Task retry 会不会用成功尝试覆盖失败历史？本节把评分与恢复分别追到持久化对象。

先建立源码地图

Scorer Protocol、Registry 包装、ScorerSpec 与 Metric 元数据在锁定 `scorer/_scorer.py`。Sample Scorer 调用、Reducer 和 Task 结果聚合在 `_eval/task/run.py`。Task 级重试调度位于 `_eval/run.py`，日志 Schema 位于 `log/_log.py`。

完整调用链



- 1 `eval_run` 把 Task 的 Scorer 转为 ScorerSpec，保存 Registry 名、构造参数、metadata 和 Metric 规格；未通过 `@scorer` 创建的对象无法可靠记录为可重建规格。
- 2 `task_run` 为多个 Scorer 生成唯一名字。每个 Sample 完成 Solver 后，依次执行 `await scorer(state, Target(sample.target))`。
- 3 Score 写入 `TaskState.scores`、`ScoreEvent` 和 `SampleScore`；Scorer 若直接篡改同名 `state.scores`，运行会检测冲突。
- 4 多 epoch 的同一 Sample 可由 Reducer 合并；Task 结果明确保存每个 Scorer 的 `scored_samples` 与 `unscored_samples`，而不是只给一个平均值。

- 5 TaskLogger 把 EvalSpec、Plan、Samples、Results、Stats 和 Error 写成 EvalLog。log_header_only 或关闭 log_samples 会改变可审计深度，必须进入运行配置。
- 6 Sample error 先消费 retry_on_error；Task error 或显式 cancel/retry 由 run_task_retry_attempts 重新排队。Task retry 新建日志 entry，并从失败日志构造 SampleSource 复用已完成 Sample。
- 7 调度器按原 index 返回最新 Task 结果，但失败日志位置仍存在。外层审计必须把多条 entry 视为恢复历史，不能只看最后一条。

关键数据结构

ScorerSpec(scorer, args, metadata, metrics) 记录如何重建评分器。EvalSample.scores 是 scorer-name 到 Score 的映射。EvalScore 保存 name/scorer、参数、Metric、scored_samples、unscored_samples 和 metadata。EvalResults.scores 汇总各 Scorer，EvalLog 再保存 status、eval spec、plan、samples、results、stats、error 与 location。

Reference Harness 的 ScoreRecord 强制绑定 canonical Attempt 与 Observation digest；Inspect AI 更强调 TaskState、Sample events 和可重建 Scorer。做 Adapter 时应把缺少的严格 digest/canonical 语义标为 partial，而不是凭字段相似声称完全对应。

实现取舍与失败语义

Registry Scorer 便于日志重评分和共享，但动态闭包或本地对象若无法记录构造参数，会降低可复现性。多个 Scorer 与 Reducer 支持丰富评测，却要求每个 Metric 明确分母和 reducer 语义。scored_samples/unscored_samples 是重要信号；发布 Gate 不应只读取已评分子集的平均值。

Sample retry 重建状态，适合瞬时错误；若模型答错却没有异常，不应进入 retry_on_error。Task retry 复用已完成 Sample 可节省成本，但前提是 Task/Model/Sandbox/Scorer 身份没有改变，失败日志和新日志关联可追踪。源码注释明确区分 retry、abort、score/error graceful resolution 与外部取消；只有错误或显式 retry 且预算未耗尽时重新排队。

动手实验

构造 100 个计划 Sample 的思想实验：90 个有 Score，其中 80 passed、10 failed；5 个 Solver error 且 score_on_error=false；5 个 Scorer 返回 None。分别计算“只在 scored samples 上的通过率”和“以计划 Sample 为分母的通过率”，并决定 Gate 应该 passed、failed 还是 inconclusive。

再比较两种恢复：Sample 23 第一次 API 超时后 retry 成功；Task 日志写入阶段失败后 Task retry 复用已完成 Sample。画出每种情况下应该保留的错误、重试和最终 Score 关系。

预期输出与答案

只看已评分子集为 80/90≈88.9%；计划分母为 80/100=80%。若政策要求 85% 且关键缺失不可忽略，Gate 不应以 88.9% passed，而应因 10 个未评分样本 inconclusive，或按预声明的保守政策 failed。选择必须在运行前声明。

Sample retry 应保留第一次 error_retries，并让新状态的 Score 进入结果；Task retry 应保留失败 EvalLog，新 entry 记录复用 SampleSource 和新的 Task 尝试。两者都不能伪装成“从未失败过”。

如何核对

在 `scorer/_scorer.py` 查看 Scorer Protocol、装饰器和 `as_scorer_spec`。在 `_eval/task/run.py` 追 Scorer 循环、ScoreEvent、Reducer 与 unscored count。在 `_eval/run.py` 阅读 `run_task_retry_attempts` 对 `cancel_type` 和 `result.status` 的分支。

本篇不能证明什么

日志字段齐全不能证明 Scorer 校准、Judge 无偏或 Task retry 在外部服务变化后仍可比较。它只提供恢复与评分的可见结构；独立发布 Gate 仍需固定身份、分母和缺失政策。

[上一节](#) · [下一节](#)

OpenAI Evals 源码课程：Registry 怎样驱动 Eval 与事件记录

[上一节](#) · [下一节](#)

本篇要解决什么问题

OpenAI Evals 的经典实现围绕 Registry、EvalSpec、CompletionFn 和 Recorder 展开。命令行接收几个字符串，运行时却要把它们解析为 Eval 类、数据路径、模型调用对象和日志后端。新人若只看 YAML，会误以为配置本身就是执行；若只看某个 Eval 子类，又看不到运行身份和事件怎样统一记录。本课程追踪 oaieval.run 从配置键到 final report 的全过程。

锁定版本为 8eac7a7de5215c907fbddc30efdaf316913eccdd。这套源码包含较早的 CompletionFn/Eval Registry 设计，也有 SolverEval 等扩展。课程将它作为配置驱动 Eval Harness 的重要实现研究，不将其公开接口推断成 OpenAI 当前所有内部评测系统。

先建立源码地图

站点	锁定文件	责任
CLI	cli/oaieval.py	参数、RunSpec、Eval/CompletionFn/Recorder 装配
Registry	registry.py	加载 YAML、解引用 spec、实例化类
Eval	eval.py	Sample 遍历、CompletionFn 使用与聚合返回
Completion API	api.py	CompletionFn Protocol 与匹配记录 helper
Recorder	record.py	Sample 事件、Local/HTTP/DB 后端与 final report

完整调用链

Registry 把字符串配置解析成 Eval、CompletionFn 与数据路径



每个 sample: `as_default_recorder(sample_id) → 调 CompletionFn → record_sampling / match / metrics / error`

Eval.run 返回 dict → `Recorder.record_final_report`

事件丰富不等于 Trial/Attempt 状态完整; Registry 路径和解析后参数必须一起保存

- 1 `oaieval.run` 创建或接收 Registry, 追加自定义 registry paths, 再通过 eval key 取得 EvalSpec。
- 2 CLI 合并 Eval 参数和 completion_args。CompletionFn 字符串按逗号拆分, 由 `registry.make_completion_fn` 解析为模型快捷实现、Registry CompletionFn 或 Solver。
- 3 入口构造 RunSpec, 选择 Dummy、Local、HTTP 或数据库 Recorder, 并通过 Registry 取得 Eval class。
- 4 Eval class 用 completion_fn_instances、EvalSpec args、eval_registry_path 和 Registry 实例化。数据相对路径会以该 Registry 的 data 目录为基准解析。
- 5 `eval.run(recorder)` 遍历样本。每个 Sample 进入 `recorder.as_default_recorder(sample_id)` 上下文, Eval/CompletionFn 通过 helper 写 sampling、match、function_call、metrics 或 error 事件。
- 6 Eval 返回聚合 dict; CLI 可从 sampling 事件补充 token usage, 再调用 `record_final_report`。

关键数据结构

EvalSpec 保存 key、class、args 与 registry_path; CompletionFnSpec 保存类和参数; RunSpec 把 completion_fns、EvalSpec 和 run_id 固定为运行头。Event 至少把 run_id/sample_id/type/data/time 关联起来。CompletionFn Protocol 接收 prompt 并返回 CompletionResult, Eval 决定怎样把结果转成 match 或其他 metric。

这里的 Sample event 与 Reference Harness TraceEvent 相似, 但没有强制 parent/sequence、canonical Attempt 或 Observation Bundle digest。课程会把这一能力映射标为 partial, 而不是因为都是 JSONL 就声称等价。

实现取舍与失败语义

Registry 降低新增 Eval/模型配置的代码耦合, 也让同一类可以由不同 args 实例化; 代价是运行身份分散在 YAML、路径优先级、解引用链和 CLI override。Recorder 使用 ContextVar 提供隐式 sample_id, Eval 代码简洁, 但异步/线程边界和缺失上下文必须谨慎处理。

HTTP Recorder 失败可回落 LocalRecorder, 保护证据不因远端日志服务故障完全丢失; 这属于记录层恢复, 不代表产品 Sample 可以重答。CompletionFn 的网络 retry 与 Eval 的 Sample 语义也未在核心对象里形成显式

Trial/Attempt 分层。

动手实验

写一份运行身份清单：Eval key、Registry 路径与 commit、解析后的 Eval class/args、CompletionFn key/class/args、数据文件摘要、Recorder 类型与路径。再指出仅保存 CLI 字符串会丢掉哪些信息。

执行：

```
python scripts/sources.py verify
python -m pytest tests/test_harness_course_docs.py -q
```

预期输出与答案

仅保存 oaieval model eval-name 会丢 Registry 路径优先级、spec 解引用、CLI override、模型实际版本、数据内容摘要和 Recorder fallback。可复现记录必须保存解析后 RunSpec 与实际资源版本。

课程门禁只有在四篇正文、图示、锁定链接和上下导航完整时通过；它不需要 OpenAI API key，也不运行外部模型。

如何核对

从 cli/oaieval.py 顺序追 get_eval、make_completion_fn、RunSpec、build_recorder、get_class、eval.run 和 record_final_report。再到 registry.py 核对 Registry 怎样加载不同资源目录和拒绝重复 key。

本篇不能证明什么

Registry 与 Recorder 完整不能证明 Eval 数据有效、CompletionFn 是声明模型、事件足以审计 Agent 副作用，或 final report 可直接作为发布 Gate。它解释配置驱动机制，不扩大为生产质量证明。

上一节 · 下一节

01 | Registry 与 EvalSpec：字符串怎样变成真实对象

上一节 · 下一节

本篇要解决什么问题

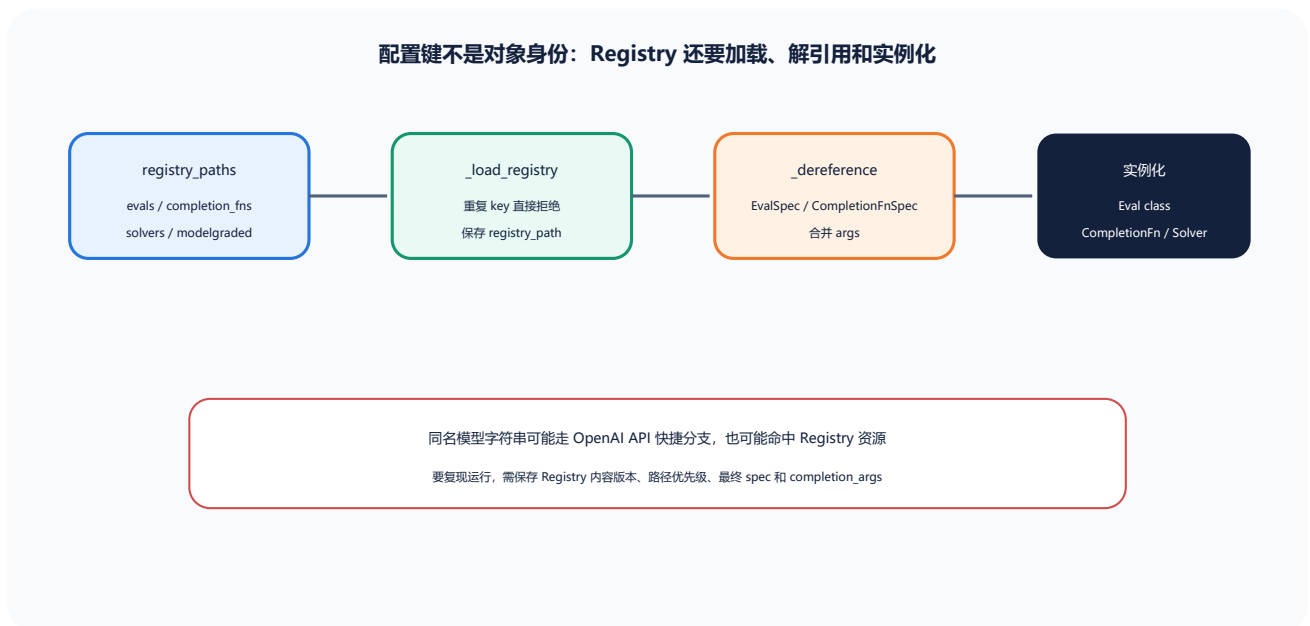
同一个 eval-name 在不同 Registry 路径可能指向不同 class、args 和数据；同一个模型字符串既可能命中 API 模型快捷分支，也可能是 Registry CompletionFn 或 Solver。若不理解加载顺序和解引用，运行命令看似相同，实际执行对象却可能不同。本节专门拆解 Registry 的身份解析。

先建立源码地图

核心实现位于锁定 registry.py。CLI 何时添加 registry paths 和覆盖 args 位于 cli/oaieval.py。Eval 怎样使用 eval_registry_path 解析数据位于 eval.py。

Registry 资源类型包括 completion_fns、solvers、eval_sets、evals 和 modelgraded。每次属性访问可从配置路径加载对应目录；加载时保存资源来源路径，并对重复条目执行断言。

完整调用链



- 1 Registry 初始化默认路径；CLI --registry_path 可追加更多目录。
- 2 _load_resources 扫描某一资源类型的 YAML，_load_registry 合并条目。重复 name 立即失败，避免静默覆盖。
- 3 get_eval(name) 通过 _dereference 把原始字典解析为 EvalSpec，并把 registry_path 留在 spec 中。
- 4 CLI 把 extra_eval_params 合入 EvalSpec.args。此时 spec 已不同于仓库静态 YAML，必须记录解析后值。
- 5 make_completion_fn 先识别 dummy 或已知 OpenAI 模型名，再查询 CompletionFn/Solver Registry；最终加载类、合并 kwargs，并验证实例满足 CompletionFn。

- 6 `get_class(eval_spec)` 取得 Eval class。实例化时传入 `eval_registry_path`，Eval 的数据路径通过 `_prefix_registry_path` 落到对应 Registry 的 data 目录。

关键数据结构

RawRegistry 是 name 到原始 spec 的字典。BaseEvalSpec/EvalSpec/CompletionFnSpec 把 key、class 名、args、metrics 和 registry_path 结构化。逻辑 key 便于引用，class 决定代码，args 决定行为，registry_path 决定数据相对路径；四者缺一都不能构成完整身份。

RunSpec 又保存 run_id、completion_fns 和 eval_spec，供 Recorder 每个 Event 引用。它比单条 EvalSpec 更接近一次运行身份，但数据文件内容、代码 commit 和服务端实际模型仍需外部摘要补足。

实现取舍与失败语义

多 Registry 路径支持私有 Eval 覆盖和扩展，但当前重复 key 选择失败而非后者覆盖，这是更安全的显式冲突策略。每次加载资源简单直观，却意味着性能和确定性依赖文件系统状态；要离线复现，应该把解析结果和来源版本一起冻结。

CLI override 提高实验速度，却会让静态 YAML 与实际运行不一致。CompletionFn 模型快捷分支方便常见 API，却让“字符串解析规则”也成为版本化身份的一部分。找不到 key、class 载入失败或实例不满足 Protocol 应在运行前阻断，而不是生成空结果。

动手实验

假设两个 registry paths 都定义 shipping.eval，一个 class 为规则 Eval，一个 class 为 ModelGraded Eval。回答加载器应静默选择哪个；再假设只有一个条目，但 CLI 把 max_samples 从 100 改成 10，报告应保存 YAML 值还是解析后值。

列出生成永久运行指纹所需字段，并与本仓库 canonical_digest 的输入思路比较。

预期输出与答案

锁定实现会拒绝重复 key，不应静默选前或选后。报告必须保存解析后的 10，同时保留 override 来源，静态 YAML 的 100 只能解释默认值。指纹至少应覆盖 Registry commit/路径、EvalSpec class/args、CompletionFn class/args、数据摘要和代码版本。

若仅哈希逻辑 key，两个不同 Registry 的同名 Eval 会碰撞；若只哈希 YAML 又忽略 CLI override，也无法代表实际运行。

如何核对

在 registry.py 搜索 `_load_registry`、`duplicate entry`、`_dereference`、`make_completion_fn` 与 `_evals`。在 cli/oaieval.py 核对 `extra_eval_params` 和 `completion_args` 的合并时点。

本篇不能证明什么

确定解析对象不能证明 YAML 内容正确、数据授权充分或 class 无副作用。Registry 解决配置发现与实例化，不替代 Dataset 治理、Sandbox 隔离或 Scorer 有效性。

上一节 · 下一节

02 | CompletionFn 与 Sample：谁负责调用，谁负责判断

上一节 · 下一节

本篇要解决什么问题

CompletionFn 名字像“完成一次评测”，实际只负责给定 prompt 产生 CompletionResult；怎样遍历数据、构造 prompt、比较 Reference 和聚合 metric 由 Eval 决定。分清二者可以避免把模型重试、Sample 重复和 Scorer 判断混成一个 callback，也能解释 SolverEval 为什么是另一种执行边界。

先建立源码地图

CompletionFn Protocol、CompletionResult 与匹配 helper 位于锁定 api.py。Eval ABC、Sample 迭代 helper、CompletionFn 包装和 SolverEval 位于 eval.py。CompletionFn 的解析入口位于 registry.py。

Eval 接收 CompletionFn 列表，completion_fn property 只是方便访问单项；SolverEval 则要求恰好一个 completion_fn，并将它包装成 Solver。这是两种扩展语义，不应按名称假设完全相同。

完整调用链



- 1 Registry 按模型名或 spec 创建 CompletionFn。对象只需满足协议，不需要知道 Eval key 或最终 Metric。
- 2 Eval class 从 Registry 相对 data 路径加载 JSONL Sample，可通过 `_index_samples` 给每条记录稳定运行内 index，并受 `max_samples` 限制。
- 3 Eval.run 为每条 Sample 设置 `recorder.as_default_recorder(sample_id)`，可先记录 `raw_sample`。
- 4 Eval 根据 doc 构造 prompt，调用 CompletionFn，取得 CompletionResult。具体实现可能记录 sampling 事件。

- 5 Eval 解析 sampled response, 并调用 record_match、record_metrics 或自定义事件。record_and_check_match 同时记录 expected、picked、sampled 和 options。
- 6 Eval 累计局部结果, 最后返回 dict。CLI 不理解每个 metric 的语义, 只把 dict 交给 Recorder final report。
- 7 SolverEval 把单一 CompletionFn 作为 Solver 使用, Eval 代码与 Solver 交互; 这允许多步逻辑, 但仍需要具体 Eval 定义其状态和评分。

关键数据结构

CompletionFn(prompt, **kwargs) -> CompletionResult 是模型调用边界。CompletionResult 暴露 completion 或相关响应信息。Sample 通常是 JSONL dict, 结构由具体 Eval class 解释; 这比统一 Sample 模型灵活, 但跨 Eval 复用和 schema 检查较弱。

Recorder Event 将 sample_id 与 sampling/match 等 data 绑定, 却没有统一的 Trial ID、Target resolved identity 或 Attempt ordinal。若要接入本仓库, 应由 Adapter 补充计划 Trial 和基础设施恢复记录, 不能把多次 sampling event 直接当成多个 Trial。

实现取舍与失败语义

极小 CompletionFn Protocol 让多种模型和 Solver 接入简单, 也使 Eval class 拥有较大自由度; 代价是 batching、重试、缓存、身份调和和错误分类可能分散在不同实现。Eval 子类直接写事件方便快速开发, 却难以静态证明每条 Score 都绑定相同证据字段。

模型 API 超时可能在 CompletionFn 内重试, 核心接口没有显式 Attempt; CompletionFn 正常返回错误答案是产品结果, 应由 Eval 的 match 或 scorer 判错。Sample schema 错误、Reference 缺失或 Eval class 异常应记录 error, 并明确是否进入最终分母。Solver 自我尝试属于被测算法, 不应由外层当成基础设施恢复。

动手实验

用伪代码写一个确定性 CompletionFn: 输入金额, 返回 shipping fee。再写 Eval: 加载 99/100/101, 调用 CompletionFn, 记录 expected/picked/match, 返回 accuracy。标注哪些代码属于 Target Adapter, 哪些属于 Scorer, 哪些属于 Metric。

把金额 100 的 buggy 返回视为正常 CompletionResult; 说明为什么不应由 CompletionFn 外层重试到 fixed 答案。

预期输出与答案

CompletionFn 是 Target Adapter; expected/picked 比较是 Scorer; 三条 match 的 mean 是 Metric。buggy 结果应产生 match=false, 仍是一次完成的 Sample。若外层再次调用直到 match=true, Eval 就泄露 Reference 并改变被测策略, 分数失去意义。

正确报告还应保留三条 Sample 事件和固定分母 3。若某条因基础设施错误无结果, 应显式 unscored/blocked, 而不是从 mean 列表删除。

如何核对

阅读 api.py 的 CompletionFn、DummyCompletionFn 与 record_and_check_match。再阅读 eval.py 的 Eval/SolverEval、Sample 迭代和 default recorder context, 核对职责确实分开。

本篇不能证明什么

协议分层不能证明具体 Eval 子类正确处理缺失、并发或网络重试，也不能证明模型响应来自声明版本。它给出扩展接缝，真正证据质量取决于实现和运行记录。

[上一节](#) · [下一节](#)

03 | Recorder 与 Metric

边界：事件很多，推断仍要另建合同

上一节 · 下一节

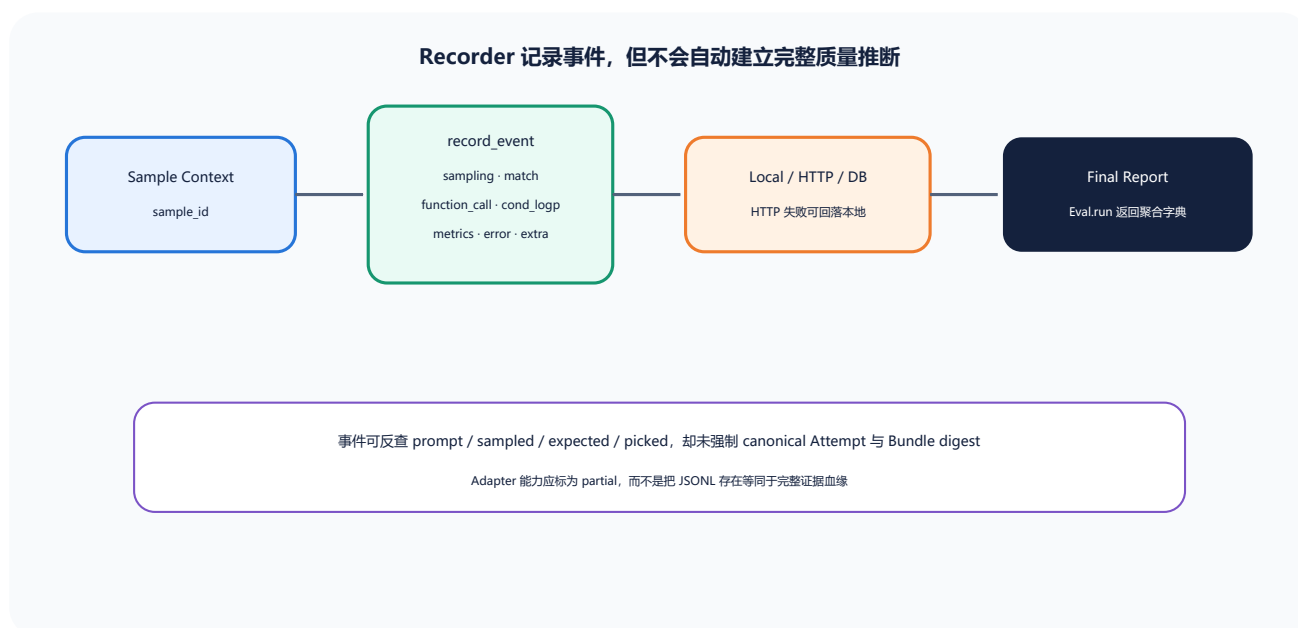
本篇要解决什么问题

OpenAI Evals 的 Recorder 可以记录 sampling、match、function_call、cond_logp、metrics、error 和任意 extra，LocalRecorder 还能写 JSONL，HTTP 失败时回落本地。但“记录到了事件”不自动回答：事件是否有因果顺序、哪次执行是 canonical、未评分样本多少、最终 accuracy 的分母是什么、报告能否作发布决定。本节划清记录层和推断层。

先建立源码地图

RecorderBase、Event、default recorder ContextVar、事件 helper 和 Local/HTTP/数据库实现位于锁定 record.py。CLI 选择后端、补 token usage 和写 final report 位于 cli/oaieval.py。match helper 位于 api.py。

完整调用链



- 1 CLI 根据 dry-run、record_path、remote URL 等选择 Recorder。每个 Recorder 持有 RunSpec。
- 2 Eval 进入 as_default_recorder(sample_id)，ContextVar 暂存当前 Recorder 与 sample_id；离开上下文后恢复旧值。
- 3 CompletionFn/Eval 调用全局 helper，最终进入 record_event(type, data, sample_id)。没有显式或上下文 sample_id 时抛错，防止事件成为孤儿。
- 4 LocalRecorder 把 Event 写入本地 JSONL；HTTP Recorder 发送远端，异常时写 local fallback 并提示。
- 5 Eval.run 返回 final report。CLI 从 sampling events 可累计 token usage，再单独调用 record_final_report。

- 6 消费者可按 type 查询 events，但 Metric 的聚合公式主要由 Eval 返回 dict 决定，Recorder 不验证分母或统计假设。

关键数据结构

Event 连接 run、sample、type 和 data。sampling data 可含 prompt/sampled/usage；match 可含 correct/expected/picked/sampled/options；function_call 可含 name/arguments/return；error 保存消息和异常；metrics 是 Eval 自由提供的键值。

RunSpec 负责运行头，final report 负责聚合输出。两者之间缺少强制的 ScoreRecord/MetricEstimate 引用链：final report 不一定列出参与每个 Metric 的 Event ID。做证据 Adapter 时需要保留原 Event，同时生成显式 Observation 与 Score 血缘。

实现取舍与失败语义

通用 Event API 让实验快速记录任意事实；代价是 type/data schema 由调用者约定，跨 Eval 语义可能不一致。ContextVar 比全局变量更适合并发上下文，但跨线程/异步任务传播仍需测试。Local fallback 是好的持久性策略，却会产生远端与本地两个位置，后续汇总必须去重并标记实际 sink。

记录服务失败属于 Harness 基础设施故障，不应改变模型 Score。Sampling event 存在但 match 缺失可能是 unscored，不应默认为失败或从分母消失。Final report 若只含平均值而没有 Sample 事件清单，证据能力为 partial。Recorder 也没有发布 Gate；任何阈值决策都应在独立政策层实现。

动手实验

给定事件：100 个 raw_sample、90 个 sampling、80 个 match=true、10 个 match=false、5 个 error、5 个只有 sampling 没有 match。分别设计：事件完整性报告、Score 状态映射、Metric denominator 和 Gate 输入。说明 HTTP fallback 重复上传时怎样去重。

再把这些字段映射到 Reference Harness 的 TraceEvent、ObservationBundle、ScoreRecord 与 MetricEstimate，标出 unavailable/partial。

预期输出与答案

计划 Sample 分母为 100。80 passed、10 failed、5 error（需按错误类别 blocked/invalid）、5 unscored。仅在 90 个 match 上算 88.9%不能直接通过；质量决定应暴露 10% 缺失。去重至少使用 run_id + sample_id + event identity/内容摘要，不能按事件文本相同删除。

Event 到 TraceEvent 为 partial，因为父子/sequence 未必存在；sampling/match 可组成 Observation，但缺 canonical Attempt digest；match 可转 Score；final report metric 若没有 score_ids，只能 partial；Gate 在核心 Recorder 中 unavailable。

如何核对

在 record.py 阅读 as_default_recorder、record_event、LocalRecorder、HttpRecorder fallback 与 helper。再在 cli/oaieval.py 核对 final report 与 token usage 是入口后处理，而不是 Recorder 自动推断。

本篇不能证明什么

JSONL 可读和 fallback 成功不能证明证据完整、事件未被改写或 Metric 统计有效。Recorder 提供观察存储；质量结论仍需要身份、血缘、分母、不确定性和 Gate Policy。

[上一节](#) · [下一节](#)

Promptfoo

源码课程：配置矩阵怎样变成可判定的评测运行

上一节 · 下一节

本篇要解决什么问题

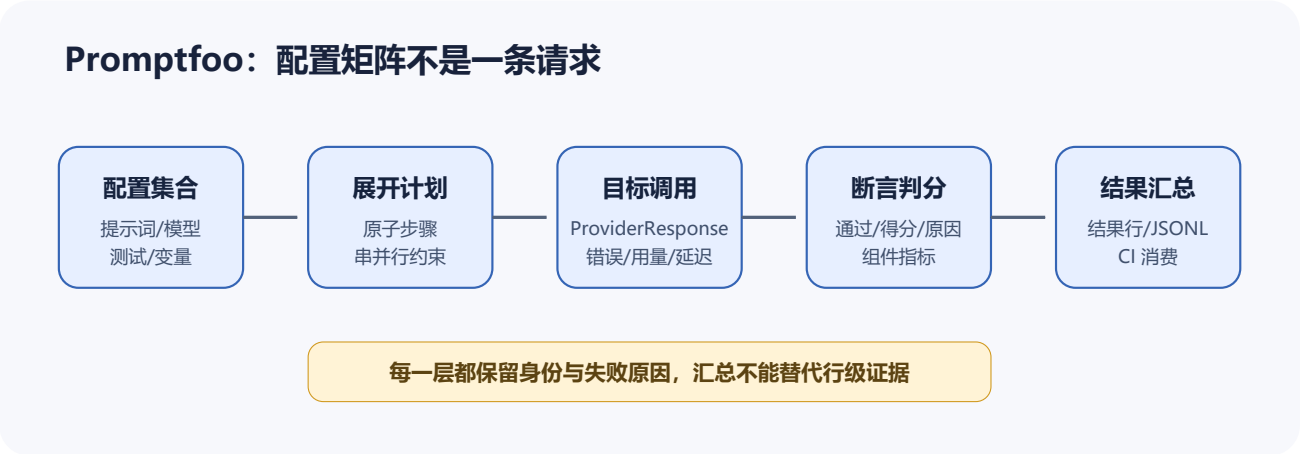
Promptfoo 常被介绍为“用 YAML 对比多个提示词和模型”，但这句话省略了 Eval Harness 最关键的部分：配置中的 prompt、provider、test、变量、重复次数与断言并不是一条请求，而是一张需要展开、调度、执行、判分和持久化的运行计划。本课程要回答：配置对象怎样被标准化为原子测试，Provider 怎样成为统一调用边界，运行器为什么有时必须放弃并发，断言怎样把响应转换为通过、得分和失败原因，以及 CI 看见的汇总与单条证据之间是什么关系。

锁定版本为 ce89186a22c59543f4f71a55d42442ff3f0e3654。课程只对该版本列出的五个源码文件作可核对解释。Promptfoo 还包含红队、缓存、Web 界面、远程服务与大量 Provider；它们不是本组入门课的覆盖承诺。

先建立源码地图

站点	锁定文件	本课关注的问题
总调度器	src/evaluator.ts	测试展开、运行步骤、并发、超时、判分和统计
运行时契约	src/evaluator/runtime.ts	Store、结果写入器和运行时依赖
Provider 装配	src/providers/index.ts	字符串、配置、函数和文件怎样解析为 ApiProvider
断言执行	src/assertions/index.ts	单断言、断言集合、模型判分和 trace-aware 判分
公共类型	src/types/index.ts	AtomicTestCase、EvaluateResult、GradingResult 与选项

完整调用链



- 1 配置层提供 prompts、providers、tests、scenarios、defaultTest 与运行选项；加载后的 TestSuite 仍不是执行队列。
- 2 Evaluator 合并默认测试和场景，把变量数组展开为组合，并为每个 test × vars × prompt × provider 生成 RunEvalOptions。重复运行和比较断言会继续改变步骤数与分组关系。
- 3 Provider 解析器把字符串 ID、内联对象、函数或外部文件统一为带 id() 与 callApi() 的 ApiProvider；运行步骤因此不必知道某家模型 SDK。
- 4 调度器检查 conversation 变量、storeOutputAs 和持久浏览器会话等跨步骤状态；这些特性存在时，并发会被降为 1。其余步骤用受限并发执行，并共享速率限制注册表。
- 5 单步渲染 prompt 和变量，调用活跃 Provider，得到 ProviderResponse；空响应、Provider 错误、超时和目标不可用分别进入不同结果分支。
- 6 runAssertions 对 assertion 或 assertion set 求值，形成 GradingResult；Evaluator 再把 success、score、namedScores、failureReason、tokenUsage 和 latency 写回 EvaluateResult。
- 7 Store 与 JSONL writer 持久化单行结果，末尾聚合每个 prompt 的通过数、失败数、错误数和得分，供报告或 CI 决策使用。

关键数据结构

AtomicTestCase 是已经可运行的测试语义，包含 vars、assert、provider override、metadata 等；RunEvalOptions 再把具体 prompt/provider/testIdx/promptIdx 与超时、缓存和信号装到一次执行。ProviderResponse 是目标调用边界，既可能有 output，也可能有 error、tokenUsage、latencyMs 与 metadata。EvaluateResult 是一行运行结果，保留原测试、响应、评分、成功状态和失败原因。GradingResult 则专注判分：pass、score、reason、namedScores、componentResults 和 assertion。

这些结构相互关联，却不能互换。Provider 有返回不等于断言通过；断言通过不等于整个 Trial 有统计代表性；prompt 汇总也不等于发布 Gate。Reference Harness 会用 Trial/Attempt 和独立 Gate 把这几层进一步拆开。

实现取舍与失败语义

配置矩阵让同一组测试可以快速横向比较多个模型和提示词，代价是“配置中的一行”与“结果中的一行”不再一一对应。排查数量异常时必须先计算展开规则。统一 Provider 接口降低接入成本，但字符串解析、插件加载和运行时配置意味着复现记录应保存解析后的 Provider 身份，而不只是原始短名。

并发降级是正确性选择：会话历史或前一步输出参与后一步输入时，乱序会改变样本本身。Provider 重试、缓存命中和断言中的模型调用又是不同恢复层，不能把多次底层调用当成多个独立样本。持久化失败会被记录并尝试保留 JSONL 权威副本；这保护已有证据，但不能自动证明结果集无缺口。

动手实验

假设配置有 2 个 prompt、3 个 provider、4 个 test，其中一个 test 的变量 tone 有 2 个值。先不运行代码，写出最小步骤数；再加入 repeat: 2 重新计算。随后标出哪些字段属于配置身份、目标响应、评分结果和运行汇总。最后解释如果某个 test 使用会话变量，为什么把并发设为 8 也不应并发执行。

运行仓库的离线检查：

```
python scripts/sources.py verify
python -m pytest tests/test_harness_course_docs.py -q
```

预期输出与答案

若四个 test 都只有一个变量组合，基础步骤数是 $2 \times 3 \times 4 = 24$ ；若只有其中一个 test 因 tone 展开为两个原子测试，总数是 $2 \times 3 \times (3 + 2) = 30$ 。全部重复两次则为 60。真实配置还可能受 scenarios、过滤、已完成结果恢复、比较断言和 provider override 影响，因此这是用于理解的最小模型。

会话变量依赖前序结果，任意并发会让输入取到错误历史或不同历史，所以调度器应串行化。课程测试应在无模型密钥、无网络调用时通过；它验证课程结构和锁定链接，不声称执行了 Promptfoo 全套集成测试。

如何核对

先在 src/evaluator.ts 依次找 buildTestsFromSuite、generateVarCombinations、appendRunEvalOptionsForProvider、adjustConcurrencyForSerialFeatures、runConcurrentEvalSteps 和 runEval。再到 src/providers/index.ts 核对 Provider 的多种输入形式，最后在 src/assertions/index.ts 核对断言聚合。

本篇不能证明什么

本篇不能证明某个 Promptfoo 配置适合生产发布、模型判分可靠、缓存无污染、所有 Provider 行为一致，或 CI 中一次全绿具有统计显著性。它提供的是锁定版本的运行骨架和核对方法。

[上一节](#) · [下一节](#)

Promptfoo 配置与 Provider：先解析身份，再生成请求

[上一节](#) · [下一节](#)

本篇要解决什么问题

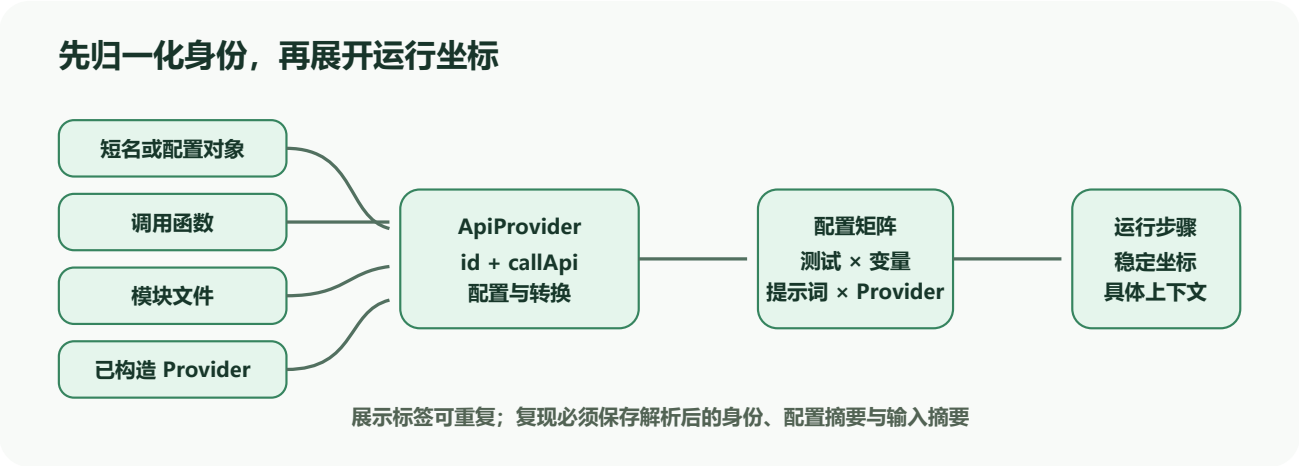
看到 `providers: [openai:gpt-4o-mini]` 时，新人容易把它理解成 Evaluator 直接拿字符串调用 SDK。实际实现必须解决更多问题：Provider 可以来自短名、带 config 的对象、JavaScript 函数、模块文件或多 Provider 配置；prompt 也可能有标签、原始模板和配置；测试还可以覆盖 Provider。若不先把这些形式归一化，执行层会充满类型分支，运行身份也无法准确记录。本篇追踪 Provider 解析和 `prompt/provider/test` 组合的边界。

锁定源码允许我们解释解析路径，不承诺列出 Promptfoo 支持的全部 Provider。某个短名在未来映射到哪个实现可能改变，因此教材以锁定 commit 和解析后的 `id()`、配置与输入为核对单位。

先建立源码地图

源码位置	作用	阅读问题
src/providers/index.ts	loadApiProvider、resolveProvider、loadApiProviders	输入形式怎样变成 ApiProvider
src/evaluator.ts	合并 prompt 配置、生成 RunEvalOptions	解析对象怎样进入运行矩阵
src/types/index.ts	TestSuite、RunEvalOptions、EvaluateResult 等契约	哪些字段跨越层级

完整调用链



- 1 loadApiProviders 接受单个或多个 Provider 引用。已经满足 ApiProvider 契约的对象可直接使用；函数被包装为 callApi；字符串和配置对象进入 loadApiProvider。
- 2 路径型配置可导出一个或多个 Provider。单 Provider 入口若读到多个 Provider 会明确报错，避免调用者误以为只选择了第一个。

- 3 解析阶段合并 basePath、环境、Provider options、label、transform、delay 与 inputs。带模板的动态配置保留到实际 callApi() 上下文再渲染，而不是在全局加载时过早求值。
- 4 Evaluator 先构建 tests，再对每个 test 计算变量组合；随后按 prompt 和 provider 逐层附加 RunEvalOptions。测试级 Provider override 可以替换套件默认 Provider。
- 5 createRunEvalOption 固定 testIdx、promptIdx、具体 ApiProvider、具体 Prompt、vars、timeout 和运行控制项。到这里，一个配置矩阵才变成可调度步骤。
- 6 单步运行时把 Provider 配置与 prompt config 合并为调用上下文，渲染模板后调用 provider.callApi(renderedPrompt, context, options)，并把返回值标准化为结果行。

关键数据结构

ApiProvider 的核心不是厂商名称，而是稳定身份与调用能力：id() 用于结果归属，callApi() 是目标边界，label/config/transform/delay/inputs 描述额外行为。Prompt 同时保留 raw、label 与 config，避免报告只显示长模板而无法区分候选。TestSuite 是未展开的配置集合，AtomicTestCase 是合并 default/scenario 后的测试，RunEvalOptions 才是某个 provider × prompt × test × vars 的执行单位。

复现清单至少应包含：锁定 Promptfoo commit、解析后的 Provider ID 与 label、非敏感配置摘要、prompt 原文摘要、变量值、测试断言、输入文件摘要和运行选项。环境变量中的密钥只记录名称或来源，不得写入证据包。

实现取舍与失败语义

多输入形式让用户可以从简单 YAML 平滑扩展到自定义函数，代价是配置解析本身成为可信计算的一部分。文件导出多个 Provider 时拒绝隐式挑选，是用显式失败换取可预测性。动态配置延迟到调用时渲染，支持 per-test 变量，却意味着“Provider 在加载时看起来相同”不代表每个测试实际请求相同。

Provider 解析失败属于装配错误，应在目标调用前暴露；callApi 返回 error 属于目标执行结果；transform 或 prompt 渲染失败属于输入/适配层错误。三者若都压成 success: false，使用者会失去修复方向。缓存命中是一次执行的来源属性，也不等于新增独立观察。

动手实验

设计三个逻辑相同的 Provider：短名、内联函数和模块文件。为每个写出应该记录的 identity 字段，并说明为什么只记录展示 label 不足以复现。再构造一个 test 级 Provider override，画出它与 suite 默认 Provider 的优先级。最后给 prompt 配置加入 temperature，说明它应在哪一层合并、怎样进入审计信息。

离线核对命令：

```
python scripts/sources.py verify
python -m pytest tests/test_harness_course_docs.py -q
```

预期输出与答案

短名至少记录解析后的实现 ID 和配置；函数记录模块或代码摘要与稳定 label；文件记录规范化相对路径、文件摘要和导出对象 ID。展示 label 可以重复或被用户修改，不能单独充当身份。测试级 override 只影响该测试生成的步骤，其他测试仍使用 suite providers。prompt config 应在创建调用上下文时与 Provider/prompt 配置按明确优先级合并，并随结果或运行清单保存非敏感快照。

若一个文件导出多个 Provider，却经单 Provider API 加载，预期是明确错误，而不是默默选择。若配置包含每测试模板变量，预期在运行时根据当前 vars 渲染。

如何核对

在 `src/providers/index.ts` 从 `loadApiProviders` 追到 `loadApiProvider`，观察函数包装、对象直通、文件配置和多导出拒绝逻辑；再在 `src/evaluator.ts` 追 `appendRunEvalOptionsForTestCase`、`appendRunEvalOptionsForVars`、`appendRunEvalOptionsForProvider` 和 `createRunEvalOption`。

本篇不能证明什么

Provider 成功加载不能证明凭据有效、模型版本固定、外部 API 可用或请求被服务端按声明参数执行。配置摘要也不能替代完整供应链锁定；自定义函数的安全性与沙箱边界需要单独审计。

[上一节](#) · [下一节](#)

Promptfoo 测试运行时：展开、串并行与结果落盘

上一节 · 下一节

本篇要解决什么问题

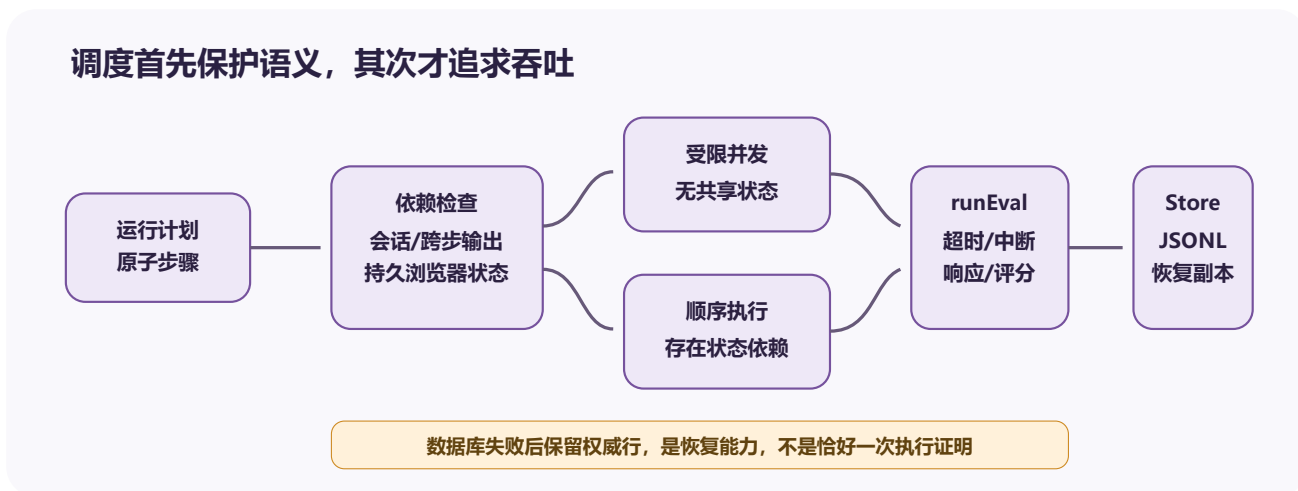
评测性能问题常被简单归因于“并发太低”，结果正确性问题又常被归因于“模型随机”。Promptfoo 的 Evaluator 表明，两者都要先理解运行步骤怎样生成和调度：变量数组会扩大测试数，resume 会过滤已完成步骤，会话历史和跨步骤输出会强制串行，超时可以发生在单步或全局，持久化失败还会触发恢复副本。本篇解释一次原子运行从队列到结果存储的生命周期。

重点不是记住每个私有函数，而是建立四个检查点：计划是否正确展开、调度是否保持依赖顺序、错误是否归到正确层、持久化是否保留权威结果。

先建立源码地图

源码位置	责任	需要核对的行为
src/evaluator.ts	Evaluator、runEval、串并行调度	计划、超时、速率限制、统计
src/evaluator/runtime.ts	Store 与 writer 契约	结果怎样追加、读取和恢复
src/types/index.ts	运行选项与结果类型	timeout/cache/abort 和结果字段

完整调用链



- 1 Evaluator 合并 default tests 与 scenarios，准备变量并应用 input transform，得到 AtomicTestCase 列表。
- 2 变量数组经 generateVarCombinations 展开，随后按 Provider 和 Prompt 生成 RunEvalOptions。resume 模式会读取已有结果并过滤已完成坐标。
- 3 adjustConcurrencyForSerialFeatures 检查 conversation 变量、storeOutputAs 和浏览器持久会话。任何一步依赖前一步状态时，最大并发被改成 1；其余步骤进入 forEachOfLimit。
- 4 调度器为每步检查 abort/global duration，并给单步包裹 timeout；共享 RateLimitRegistry 可以根据 Provider 的限流反馈调整并发。

- 5 `runEval` 创建状态，渲染 prompt，构造 Provider context，调用 Provider，再处理 trace、delay、transform、空响应与 grading。
- 6 `EvaluateResult` 先更新 prompt metrics 和全局 stats，然后追加到 Store 与 JSONL writer。数据库追加失败会登记 persistence failure；之后在内存保留权威副本，供终局合并覆盖陈旧行。
- 7 中断时保存已完成进度；目标被判定不可用时结束剩余步骤并保存当前结果。最终 Store 汇集结果、prompts、vars 与统计。

关键数据结构

运行坐标由 `testIdx + promptIdx + provider + vars` 共同确定，不能只用数组序号。`EvalProcessingContext` 持有 concurrency、共享变量集合、目标不可用状态等本次运行状态。`EvaluateResult` 同时保存 `testCase`、`prompt`、`provider`、`response`、`gradingResult`、`success`、`score`、`namedScores`、`failureReason`、`latency` 与 `tokenUsage`。Store 提供 `appendResult`、`readResults`、`hasResultPersistenceFailure`、`recordFinalResult` 等接口，使 Evaluator 不绑定某个数据库。

Prompt metrics 是对结果行的派生汇总。它们适合报告，但不能替代行级证据：总通过数无法回答哪条输入失败、是否缓存命中、哪个 Provider 返回错误，或一次比较断言怎样改变最终得分。

实现取舍与失败语义

受限并发提升吞吐，串行特性检测保护有状态语义；难点是任何未声明的共享状态都可能绕过检测。自适应限流减少持续 429，但重排等待时间可能影响时间相关指标，因此 latency 的解释要区分排队与 Provider 调用。单步 timeout 生成一条失败结果，全局 max duration 则可能让剩余步骤从未产生结果，这两种缺口不能只看失败率区分。

持久化采用“数据库 + 流式 JSONL + 失败后内存权威副本”的韧性策略。它降低进程中后段丢结果风险，但恢复合并仍需稳定坐标与去重规则。中断保存是运行恢复能力，不应把同一逻辑测试恢复后的多次底层调用统计成多个独立 Trial。

动手实验

构造六个 `RunEvalOptions`：前三个无状态，第四个写 `storeOutputAs`，第五个读取该值，第六个使用 `conversation` 变量。分别给出并发 3 和并发 1 时允许的执行顺序，指出哪一种保持语义。再模拟第三条结果数据库写失败但 JSONL 成功，写出终局恢复时需要的合并键和优先级。最后区分 Provider 500、断言失败、单步超时、全局到时和用户中断应产生的证据。

```
python scripts/sources.py verify
python -m pytest tests/test_harness_course_docs.py -q
```

预期输出与答案

前三个无状态步骤可受限并发；出现跨步骤存取或会话历史后，相关序列必须保持顺序，当前实现会把整次运行并发降为 1，以简单规则换正确性。恢复键至少要包含稳定运行身份与 `test/prompt/provider/vars` 坐标；当数据库已报告持久化失败，最终内存/流式权威行应覆盖数据库陈旧行，但重复行必须去重。

Provider 500 是目标错误；断言失败是有效响应未满足判据；单步超时应有对应结果行；全局到时意味着已完成行之外存在未运行步骤；用户中断还应记录终止原因。把后四种都显示成红色可以，但机器可读原因必须不同。

如何核对

在 `src/evaluator.ts` 核对 `adjustConcurrencyForSerialFeatures`、`runSerialEvalSteps`、`runConcurrentEvalSteps`、`processEvalStepWithTimeout`、`persistEvalRow` 与 `saveInterruptedEval`；在 `src/evaluator/runtime.ts` 核对 `Store` 的读取、追加和 `persistence failure` 契约。

本篇不能证明什么

存在 `resume`、`JSONL` 和自适应限流不能证明恰好一次执行、跨进程事务完整、远程 `Provider` 幂等或中断后统计无偏。课程也没有对真实 API 做故障注入；这些属于部署环境中的专项验证。

[上一节](#) · [下一节](#)

Promptfoo 断言与 CI：从响应判分到门禁还差哪一步

上一节 · 下一节

本篇要解决什么问题

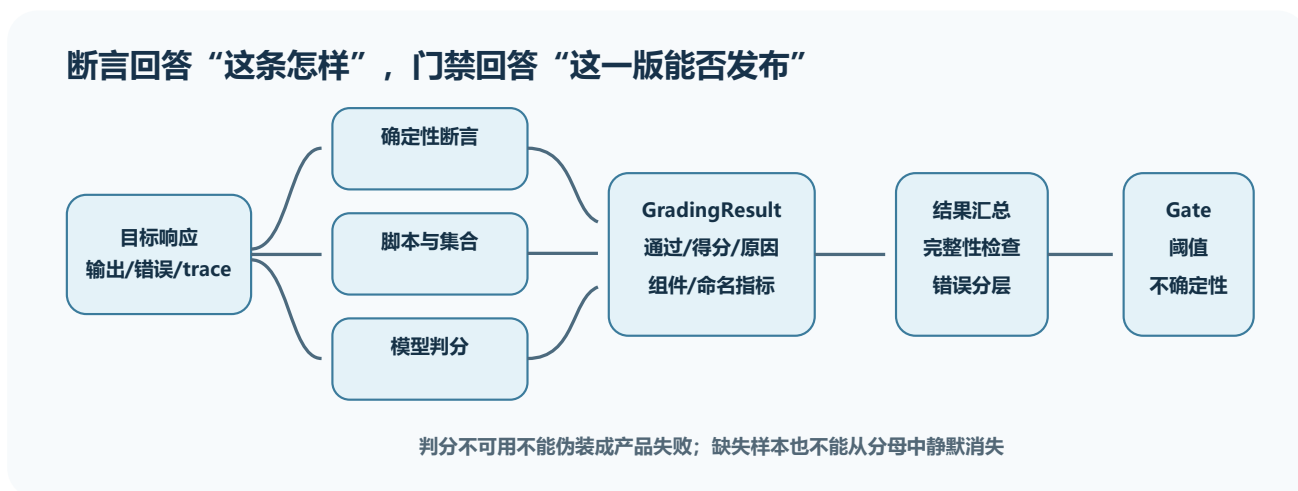
一个响应可以同时接受 contains、JSON schema、自定义脚本和模型判分；断言还可能有权重、阈值、命名指标、否定前缀与嵌套集合。于是“pass”并不是 Provider 的属性，而是某组判据在某次响应上的聚合结论。本篇追踪 runAssertions 怎样产生 GradingResult、Evaluator 怎样把它并入结果和 metrics，并解释为什么这些统计还不是一个完整发布 Gate。

读完应能回答：单断言失败和断言执行错误如何区分；模型判分为什么可能额外消耗 token；断言集合怎样保留组件结果；比较型断言为什么要延迟到多行结果齐备；CI 阈值应该读取哪些稳定字段。

先建立源码地图

源码位置	责任	阅读焦点
src/assertions/index.ts	断言分派、求值、聚合与比较	pass/score/reason 怎样生成
src/types/index.ts	Assertion、AssertionSet、GradingResult	判分契约保存什么
src/evaluator.ts	grading、comparison、metrics 和统计	单行结论怎样汇总

完整调用链



- 1 ProviderResponse 进入 gradeRunEvalResponse；无断言时返回专门的 no-assert 结果，而不是伪造一个命中的断言。
- 2 runAssertions 将普通 assertion 与 assertion set 展开为待执行项，为每个集合建立子聚合器。若存在 trace-aware assertion，会用 traceId 加载相关 trace 数据。
- 3 单项进入 runAssertionInternal：模板值先按当前 vars 渲染；脚本型值可能加载外部函数；base type 与 not-反向语义被标准化；最终 handler 接收 AssertionParams。

- 4 模型判分型断言可通过 grading Provider 发起额外调用。它返回的 tokenUsage 属于评分成本，不应混入目标模型成本后丢失来源。
- 5 每个结果写入主 AssertionsResult；属于集合的结果同时写入子聚合器。结束后集合按 threshold/权重形成组件结论，并保留 componentResults。
- 6 Evaluator 把 GradingResult 应用于 EvaluateResult：设置 success、score、namedScores、reason 和 failureReason，同时累计 assertion token、通过数、失败数。
- 7 select-best、max-score 等比较断言必须等待同组候选结果齐备后执行；其结论可能回写某行成功状态与 prompt metrics。
- 8 CI 可消费结果或汇总并按配置退出非零，但真正发布 Gate 还需要预注册数据版本、统计规则、不确定性处理和缺失结果策略。

关键数据结构

Assertion 包含 type、value、threshold、weight、metric、provider 等；AssertionSet 用嵌套 assert 数组表达组合判据。AssertionParams 把 assertion、标准化 baseType、inverse、output、renderedValue、test、provider、ProviderResponse 和上下文交给 handler。GradingResult 至少含 pass、score、reason，并可含 namedScores、tokensUsed、componentResults、assertion 与 metadata。

EvaluateResult.success 是整条结果的最终布尔结论，failureReason 区分目标错误和断言失败，namedScores 支持多个指标。PromptMetrics 再汇总 testPassCount、testFailCount、assertPassCount、assertFailCount 与 score。若只导出 success，会丢掉“为何失败”和“哪个判据变化”的诊断证据。

实现取舍与失败语义

统一断言分派使大量判据共享模板渲染、反向语义与错误处理；动态脚本和模型判分提高表达力，也引入外部代码安全、Judge 偏差和额外网络失败。集合聚合保留 componentResults，优于只返回总分；但阈值和权重仍是产品政策，必须有版本与理由。

断言 false 是被测对象不满足契约；断言 handler 抛错是评测系统无法得出结论；评分 Provider 不可用是 Judge 层故障；trace 缺失可能让 trace-aware 判据不可判。发布规则不应把后三者当普通负样本计入失败率，否则基础设施故障会伪装成产品质量下降。反过来，也不能忽略它们后用较小分母宣布通过。

动手实验

给同一响应设计三项断言：格式正确权重 1、事实一致权重 2、安全约束权重 3。分别计算全部通过、仅事实失败、Judge 超时三种情况下应保存的组件结果和总体状态。再设计一个候选 A/B 的 select-best，说明为什么它不能在 A 返回后立即判分。最后写一份 CI Gate 输入契约，至少包括结果集完整性、错误率、核心指标、最小样本数和置信区间策略。

```
python scripts/sources.py verify
python -m pytest tests/test_harness_course_docs.py -q
```

预期输出与答案

全部通过时总体 pass，组件仍应逐项保存；事实失败时总体规则按配置阈值计算，并保留事实项 reason，而不是只写总分；Judge 超时时应标为评分不可用或评测错误，不能等同事实断言为 false。A/B 比较必须拿到同组输出才能形成相对结论。

合格 Gate 至少先验证预期坐标全部出现且无不可解释重复，再把产品失败与 harness error 分开；随后计算预注册核心指标及不确定性，最后按明确阈值给 pass/fail/inconclusive。Promptfoo 的 assertions 和 CI 退出码提供重要构件，但课程不把它们自动等同完整发布制度。

如何核对

从 src/assertions/index.ts 的 runAssertions 追到 runAssertion、runAssertionInternal 和 handler 映射，观察集合、并发与 trace-aware 分支；再在 src/evaluator.ts 查 gradeRunEvalResponse、applyGradingResult、comparison merging 与 metrics 更新。

本篇不能证明什么

断言实现丰富不能证明 Judge 与人类一致、指标与线上业务相关、阈值合理或结果具有统计显著性。CI 退出码也不是发布授权；真实组织仍需独立数据治理、复核、例外流程和回滚机制。

[上一节](#) · [下一节](#)

DeepEval 源码课程：从测试用例到 MetricData 的评测生命周期

[上一节](#) · [下一节](#)

本篇要解决什么问题

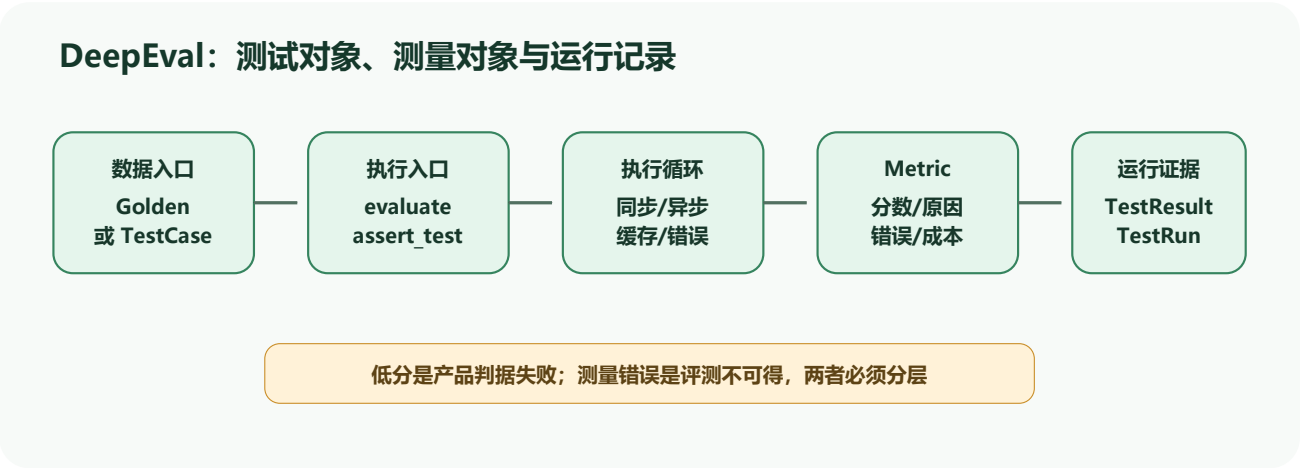
DeepEval 经常以“像 pytest 一样测试大模型应用”出现，但源码中的核心并不是一个 `assert score >= threshold`。输入可能是已经包含 `actual_output` 的 `LLMTestCase`，也可能是等待用户代码执行的 `Golden`；指标可能同步、异步、面向单轮、对话或 `trace`；缓存、忽略错误、并发和测试运行管理器又会改变执行与记录方式。本课程追踪这些对象怎样汇合为 `TestResult`、`MetricData` 与一次 `TestRun`。

锁定版本为 `a2e0d4cfd3118352d321c1c84bdeba17d4a201bc`。DeepEval 项目还包含大量具体指标、追踪、平台集成和 `Agentic` 入口，本课程只解释锁定范围内可直接核对的执行骨架。我们会把它映射到统一的 `Dataset` → `Sample` → `Target` → `Scorer` → `Result` 模型，同时保留 DeepEval 自己的 `Golden`、`TestCase`、`Metric` 命名。

先建立源码地图

站点	锁定文件	责任
公开入口	<code>deepeval/evaluate/evaluate.py</code>	<code>assert_test</code> 、 <code>evaluate</code> 、配置与 <code>TestRun</code> 收尾
执行循环	<code>deepeval/evaluate/execute/loop.py</code>	测试、 <code>trace</code> 、 <code>metric</code> 的同步/异步执行与错误处理
指标基类	<code>deepeval/metrics/base_metric.py</code>	<code>score</code> 、 <code>threshold</code> 、 <code>success</code> 、 <code>reason</code> 与测量接口
数据集	<code>deepeval/dataset/dataset.py</code>	<code>Golden/TestCase</code> 集合、身份和 <code>agentic iterator</code>
单轮用例	<code>deepeval/test_case/llm_test_case.py</code>	<code>input</code> 、 <code>actual_output</code> 、 <code>expected_output</code> 、 <code>context</code> 、 <code>tools</code> 等观测

完整调用链



- 1 调用者选择两种边界：把已执行结果封装成 `LLMTestCase` 交给 `evaluate`，或遍历 `Dataset` 的 `Golden`，在用户代码产生输出和 `trace` 后由 `agentic loop` 构造 `TestCase`。
- 2 `evaluate` 验证输入与 `metrics`，配置 `AsyncConfig`、`DisplayConfig`、`CacheConfig`、`ErrorConfig`，并重置或取得全局 `TestRunManager`。
- 3 同步路径调用 `execute_test_cases`，异步路径通过事件循环调用 `a_execute_test_cases`；指标对象会按运行模式初始化并在每个测试上测量。
- 4 指标的 `measure/a_measure` 写入 `score`、`reason`、`error` 等状态；基类 `is_successful` 按 `threshold`、`strict_mode` 和 `error` 得出成功语义。
- 5 执行器把指标状态转换为 `MetricData`，附着到 `TestResult/API test case`，更新 `TestRun`。缓存可复用先前评分，`ignore_errors` 决定错误是终止还是保留为结果。
- 6 `evaluate` 输出控制台/文件报告，写入运行时长和超参数，保存临时 `TestRun`，并根据 CLI 与普通调用模式选择由谁完成 `wrap-up`。
- 7 `assert_test` 复用执行器，但在任何有阈值的指标失败或出错时抛出 `AssertionError`，适合测试框架门禁，不等同统计发布判定。

关键数据结构

`Golden` 更接近待执行样本：保存输入、期望与上下文，但通常没有被测应用的 `actual_output`。`LLMTestCase` 是可评分观测，包含 `input`、`actual_output`、`expected_output`、`context`、`retrieval_context`、`tools_called`、`expected_tools`、`metadata` 等。`BaseMetric` 是有状态 `Scorer` 实例，测量后持有 `score`、`threshold`、`reason`、`error`、`success`、`verbose_logs` 和 `evaluation_cost`。`MetricData` 是把这些状态冻结到结果中的记录。`EvaluationResult` 汇总 `TestResults`、`confident link` 与 `run identity`。

同一个 `Metric` 实例在多测试和并发中如何重置状态非常关键。对象字段便于实现具体指标，却要求执行器在每次运行前清理 `error` 等字段，并避免跨协程污染。结果证据应保存 `metric` 名称、实现/模型版本、`threshold`、`strict mode` 和单次输出，而不是只保存最终布尔值。

实现取舍与失败语义

允许用户直接提交 `TestCase`，使 `DeepEval` 可以评测任何外部系统输出；`agentic iterator` 又能把被测代码执行纳入 `trace`。两者的证据强度不同：前者信任调用者填入 `actual_output`，后者可以关联运行 `trace`，但仍需锁定用户代码与环境。

`Metric` 失败表示有效测量低于阈值，`metric.error` 表示没有得到可信测量。`ignore_errors=True` 让批量运行继续，却不应把错误项从分母静默移除。缓存可节省 `Judge` 调用，前提是 `key` 覆盖测试、指标配置、模型和依赖版本。异步并发提高吞吐，但会暴露有状态 `Metric`、事件循环和全局 `TestRun` 管理的隔离要求。

动手实验

为问答系统设计一个 `Golden` 和对应 `LLMTestCase`，分别填写 `input`、`actual_output`、`expected_output`、`context` 与 `retrieval_context`，说明每个字段由谁产生。再设计两个 `Metric`：确定性格式检查和模型 `Judge`；列出 `score`、`threshold`、`reason`、`error`、`cost` 应怎样记录。最后比较 `evaluate(..., ignore_errors=True)` 与 `assert_test` 在 `Judge` 超时时应产生的外部行为。

```
python scripts/sources.py verify
python -m pytest tests/test_harness_course_docs.py -q
```

预期输出与答案

Golden 的 `input/expected/context` 属于数据集定义, `actual_output` 属于 Target 执行, `retrieval_context` 是本次系统观测而非预先答案。格式 Metric 可离线给出确定分数; Judge Metric 还要记录评分模型、`prompt/config` 和成本。忽略错误模式应保留 `error` 并继续其他测试; `assert_test` 则应在失败指标字符串中显示 `score`、`threshold`、`strict`、`error` 与 `reason` 后抛出。

课程门禁不需要 API key, 只验证源码锁定、教学结构和本仓库离线实验; 它不声称运行具体 DeepEval Judge。

如何核对

从 `deepeval/evaluate/evaluate.py` 比较 `evaluate` 与 `assert_test`, 再进入 `deepeval/evaluate/execute/loop.py` 查同步/异步与 `agentic` 路径, 最后用 `deepeval/metrics/base_metric.py` 核对 `success` 语义。

本篇不能证明什么

API 易用、指标有阈值或 `pytest` 失败不能证明指标有效、Judge 无偏、数据集代表线上分布、运行可复现或版本可以发布。它们是评测机制, 不是发布授权。

[上一节](#) · [下一节](#)

DeepEval 数据对象：Golden 何时变成 LLMTestCase

上一节 · 下一节

本篇要解决什么问题

评测数据最常见的设计错误，是把“给模型的输入”“参考答案”“模型实际输出”和“运行时检索证据”装进一个模糊 JSON 对象。DeepEval 区分 Dataset、Golden 和 LLMTestCase，恰好提供了理解这些阶段的入口。本篇解释 Dataset 怎样维护单轮/多轮一致性和数据集身份，Golden 为什么适合先于 Target 执行存在，LLMTestCase 又为何必须包含 actual_output 才能被大多数 Metric 测量。

这不是字段字典。我们要沿对象生命周期回答：字段由谁写、何时冻结、怎样关联数据集、哪些运行时观测不能回写成参考数据，以及 agentic iterator 怎样在用户代码执行后补全 TestCase。

先建立源码地图

源码位置	责任	核对问题
deepeval/dataset/dataset.py	EvaluationDataset、导入、身份与 iterator	单轮/多轮、rank、alias/id 怎样维护
deepeval/test_case/llm_test_case.py	LLMTestCase 与 ToolCall 等观测	可评分用例保存哪些运行事实
deepeval/evaluate/execute/loop.py	Golden 到 TestCase 的运行转换	用户代码和 trace 何时进入证据

完整调用链

参考数据与运行观测不能混写



actual_output 属于运行结果；回写到 Golden 会破坏来源边界

- 1 EvaluationDataset 创建后可接收 goldens 或 test_cases；构造器检查元素类型，并根据首个对象确定单轮或多轮数据集。
- 2 加入对象时写入私有 _dataset_alias、_dataset_id 和 _dataset_rank。单轮数据集拒绝 ConversationalGolden/TestCase，多轮数据集拒绝单轮对象，防止执行器面对混合语义。

- 3 CSV/JSON 导入把列映射为 LLMTestCase 或 Golden 字段；导入只完成结构转换，不能证明来源、授权、去重或参考答案质量。
- 4 直接 evaluate(test_cases, metrics) 时，actual_output 已由调用者提供，执行器只负责测量，不执行 Target。
- 5 agentic Dataset iterator 逐个 yield Golden，让用户代码处理输入并产生 trace；控制权返回后，执行循环取得当前 trace，从 Golden 和 span attributes 构造 LLMTestCase。
- 6 新 TestCase 继承 dataset alias/id/rank，以便 TResult 回连原样本；trace/span 上的 input、output、context、retrieval context、tools 等成为运行观测。
- 7 若 span 声明指标却无法构造 LLMTestCase，执行器记录可解释错误，而不是用空字符串伪造正常评分输入。

关键数据结构

Dataset 是有身份的容器，不是统计单位。Golden 是待执行规格，可含 input、expected_output、context、additional_metadata 等。LLMTestCase 是单次可测观察，核心是 input 与 actual_output；expected_output 支持参考型指标，context 常表示预期可用背景，retrieval_context 表示系统实际检索内容。tools_called 与 expected_tools 让工具使用也可被判分。私有 dataset rank 建立来源顺序，但稳定身份最好另有 sample id 或内容摘要，不能只依赖位置。

在统一术语中，一个 Golden 可映射 SampleSpec；一次 Target 执行生成 Trial/Attempt 和 LLMTestCase 观测。DeepEval 的对象本身未强制 Trial/Attempt 分层，因此调用者做网络重试或重新生成时，应在外部保留尝试记录，避免把最终覆盖后的 actual_output 当成唯一历史。

实现取舍与失败语义

直接 TestCase 模式解耦 Target 与 Eval，适合离线导入日志；代价是 Harness 无法独立证明 output 如何产生。agentic iterator 把执行纳入会话，证据更连贯，却依赖全局 trace/session 管理和用户正确埋点。Dataset 强制单轮/多轮一致性是有价值的早失败，但不检查语义重复、数据泄漏或时间切分。

空数据集、错误对象类型和单/多轮混用属于输入合同错误，应在运行前失败。用户代码异常属于 Target/场景执行错误；trace 缺字段导致无法构造 TestCase 属于观察或适配错误；Metric 低分才是产品判据失败。四者不能合并成同一个失败率。

动手实验

选择一个“根据知识库回答退款问题”的样本，写出 Golden 和执行后 LLMTestCase 的最小 JSON 形状。把预先给定政策放入 context，把系统实际检索片段放入 retrieval_context，说明两者不同。再模拟两次网络 Attempt：第一次超时，第二次成功，指出 DeepEval TestCase 之外还需要保存哪些 attempt 元数据。最后列出 Dataset 版本清单必须增加的来源、许可证、内容摘要与切分信息。

```
python scripts/sources.py verify
python -m pytest tests/test_harness_course_docs.py -q
```

预期输出与答案

Golden 至少包含 input 和 expected_output/expected context；执行后 TestCase 增加 actual_output、retrieval_context、tools_called 和运行 metadata。context 表示评测规格提供或期望的背景，retrieval_context 是被测系统真实观测，两者混用会让检索指标失去意义。两次 Attempt 应保留 attempt_id、开始结束时间、错误、重试原因、输入摘要和第二次成为 canonical 的规则。

Dataset 版本还要记录生成/采集来源、授权、去重、污染检查、时间戳、分区策略和摘要。对象能序列化不代表数据治理完成。

如何核对

在 `deepeval/dataset/dataset.py` 阅读 `EvaluationDataset` 构造、`goldens/test_cases` setter 与 `add` 方法，再定位 `agentic evaluate iterator`；在 `deepeval/evaluate/execute/loop.py` 核对 `Golden` 怎样转为 `LLMTestCase`。

本篇不能证明什么

`Dataset alias`、云端 ID、rank 或成功导入不能证明样本唯一、无泄漏、代表真实用户或可以合法公开。
`LLMTestCase` 字段齐全也不能证明 `actual_output` 来自声明模型；需要独立运行清单和证据摘要。

[上一节](#) · [下一节](#)

DeepEval

Metric：有状态评分器怎样产出可解释结果

上一节 · 下一节

本篇要解决什么问题

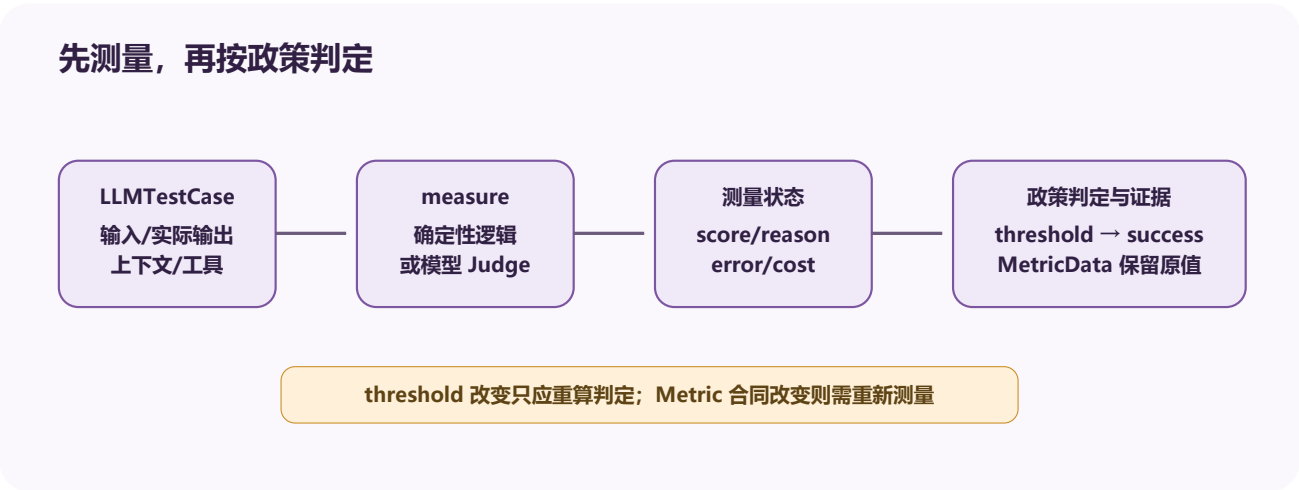
Metric 很容易被误解为一个纯函数 `test_case -> float`。DeepEval 的 `BaseMetric` 却明确持有 `threshold`、`score`、`reason`、`error`、`success`、`verbose_logs`、`evaluation_model` 和 `cost`；具体 `measure` 在对象上写状态，执行循环随后读取这些字段形成 `MetricData`。这种设计方便自定义指标，却带来并发复用、状态清理、阈值解释和错误分类问题。本篇把这些隐含合同展开。

目标是让读者能够实现一个最小自定义 Metric，并知道哪些字段属于原始测量、哪些属于政策判断、哪些属于运行诊断；同时理解模型 Judge 与确定性 Scorer 为什么不能只按同一个 `score` 比较。

先建立源码地图

源码位置	责任	阅读焦点
deepeval/metrics/base_metric.py	Metric 抽象和 <code>is_successful</code>	测量状态与阈值语义
deepeval/evaluate/execute/loop.py	执行、跳过、错误与结果更新	对象状态怎样冻结为证据
deepeval/evaluate/evaluate.py	指标输入验证和入口行为	阈值与 <code>assert_test</code> 的关系

完整调用链



- 调用者构造 Metric 实例并设置 `threshold`、`strict_mode`、`evaluation model` 等配置；入口验证至少一项指标具有阈值，确保 `assert` 语义可判定。
- 执行器为 TestCase 与 Metric 建立 pair。在每次执行前清理 `metric.error` 等上次状态，并根据同步/异步模式调用 `measure` 或 `a_measure`。
- 具体 Metric 从 LLMTestCase 读取所需字段。确定性指标直接计算；LLM-as-a-Judge 指标可能生成 prompt、调用评估模型、解析结构化结果并累计 `evaluation_cost`。

- 4 Metric 写回 score、reason、verbose_logs 等；若异常被保留，则写 error。is_successful 在 error 时返回 false，在无 threshold 时依据 score 是否存在，否则比较 score 与 threshold；strict_mode 可要求满分。
- 5 执行公共逻辑把 Metric 状态转换为 MetricData，包括 name、threshold、success、score、reason、strict、evaluationModel、error 与 cost，并挂到 TestResult。
- 6 assert_test 遍历 metrics_data，把 error 非空或 success 为 false 的指标列为失败，生成可读字符串并抛出 AssertionError。
- 7 批量 evaluate 不只给布尔断言，还保留每个 TestCase 的指标结果供控制台、文件和 TestRun 汇总。

关键数据结构

BaseMetric 的抽象方法是 measure/a_measure，返回 float 只是接口表面；真正结果还在实例字段中。score 是测量值，threshold 是当前政策，success 是两者结合后的派生判断，reason 解释测量，error 表示测量不可得，evaluation_cost 是 Judge 运行成本。score_breakdown 和 verbose_logs 提供组件诊断。

因此证据存储应保留原始 score 与配置，而不是只保留 success。阈值变化时可以在不重跑 Target 的情况下重新计算政策结论；但若 Metric prompt/model 或解析逻辑变化，旧 score 是否可复用需要指标自身的版本语义。

实现取舍与失败语义

有状态基类让自定义 Metric 编写直观，也允许控制台直接读取 reason；代价是同一实例跨 TestCase 并发复用可能发生数据竞争。执行框架需要复制实例、串行访问或确保任务隔离。一个 Metric 在开始新测试前必须清除上次 error/score，否则前一用例会污染后一用例。

低于阈值是有效失败；解析 Judge 输出失败是 Metric error；TestCase 缺少必需字段是输入/适配错误；Judge API 限流是评分基础设施错误。strict_mode 把通过条件收紧为特定语义，属于政策配置，不是“分数更准确”。模型 Judge 的 reason 是模型生成解释，不是事实证明，仍需校准、人工复核和对抗样本。

动手实验

实现纸面版 ExactJsonMetric：验证 actual_output 可解析为 JSON 且包含 answer 字段，成功得 1，否则得 0，并给 reason。写出 measure 前后 BaseMetric 字段变化。再设计 FaithfulnessJudgeMetric，列出额外需要锁定的模型、模板、温度、解析 schema 和重试策略。最后给同一批 score 分别应用 0.7 和 0.9 threshold，说明为何这是重新判定而非重新测量。

```
python scripts/sources.py verify
python -m pytest tests/test_harness_course_docs.py -q
```

预期输出与答案

ExactJsonMetric 在测量前 score/reason/error 应为空或重置，测量后 score 为 0/1、reason 描述具体条件、error 仅在指标自身无法运行时出现。Judge Metric 的 identity 不能只有类名，至少包括评估模型标识、系统/用户模板摘要、参数、输出 schema、库 commit 和依赖版本。

应用不同 threshold 只改变 success 和汇总，不改变原始 score；若 strict_mode 改变，必须在结果中记录。若 Judge 逻辑改变，则原 score 的测量合同已变，不能简单重算 success 冒充同一指标版本。

如何核对

阅读 `deepeval/metrics/base_metric.py` 的字段、抽象方法和 `is_successful`，再在 `deepeval/evaluate/execute/loop.py` 搜索 `_execute_metric`、`MetricData` 和 `test run update`，核对异常后是否继续以及结果何时冻结。

本篇不能证明什么

Metric 有 `reason`、`threshold` 和 `cost` 不能证明构念有效、Judge 与专家一致、阈值经过业务验证或不同指标的 0.8 可直接比较。指标可靠性、效度、偏差和鲁棒性需要独立验证数据。

[上一节](#) · [下一节](#)

DeepEval

执行策略：异步、缓存与错误不能改变统计单位

上一节 · 下一节

本篇要解决什么问题

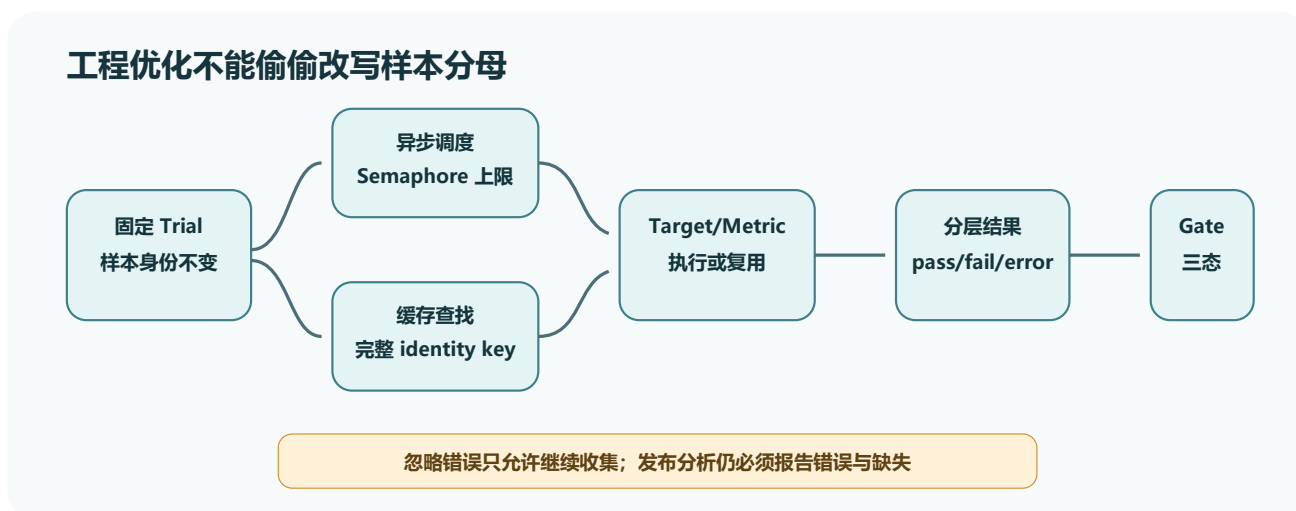
当 Judge 调用昂贵而慢时，异步并发、缓存和忽略错误都是必要工程能力；它们也最容易让评测数字失真。并发可能污染有状态 Metric，缓存可能复用过期评分，ignore_errors 可能改变分母，失败重跑又可能把同一测试变成多个独立观察。本篇从 DeepEval 的 AsyncConfig、CacheConfig、ErrorConfig 和 agentic loop 出发，建立正确的恢复与统计边界。

我们关注的不是“开哪个参数更快”，而是每个参数改变了哪一层：执行调度、评分调用、证据记录还是外部测试行为。任何优化都必须保持 Sample/Trial 身份、结果完整性和错误可见性。

先建立源码地图

源码位置	责任	核对问题
deepeval/evaluate/evaluate.py	配置默认值、同步/异步选择、收尾	参数怎样进入执行器
deepeval/evaluate/execute/loop.py	semaphore、任务、cache/error 和 TestRun	并发与异常怎样传播
deepeval/metrics/base_metric.py	有状态测量对象	并发隔离需要保护什么

完整调用链



- 1 evaluate 接收 AsyncConfig、CacheConfig 和 ErrorConfig；异步模式取得事件循环并调用异步执行器，同步模式直接顺序执行。
- 2 异步 agentic loop 创建 `asyncio.Semaphore(max_concurrent)`，把每个用户 callback 绑定到当前事件循环并受信号量控制，避免无限并发和跨 loop Future 混用。

- 3 Target/user code 结束后捕获 trace，构造 TestCase；执行器为 metrics 计算缓存键，命中时复用 MetricData，未命中才运行 a_measure/measure。
- 4 Metric 或 trace 发生错误时，ErrorConfig 决定是否抛出或把 error 写入结果继续。继续不代表成功，后续汇总必须保留不可评分状态。
- 5 CacheConfig 区分 use_cache 与 write_cache。TestRunManager 的磁盘保存行为随配置改变，但缓存与最终证据存储不是同一概念。
- 6 任务完成后，每个 TestCase 更新 TestRun；异步任务集合属于当前 iterator run，结束时一次性归集，避免状态泄漏到下一事件循环。
- 7 入口记录 run_duration、超参数和输出；在 CLI 测试模式中由 CLI 统一 finalize，普通 evaluate 才自行 wrap-up，避免重复收尾。

关键数据结构

AsyncConfig 至少表达 run_async、max_concurrent、throttle 等调度政策；CacheConfig 表达读取/写入行为；ErrorConfig 表达 ignore_errors 与跳过策略。这些配置都应进入 RunManifest。Semaphore 限制“同时进行的协程数”，不提供 Trial 身份、顺序或事务语义。缓存值应携带 test case 摘要、metric identity/config、Judge model、代码版本和输出 schema，否则命中无法审计。

错误结果需要机器可读类型：target_error、trace_error、metric_error、timeout、cancelled、cache_corrupt 等。Reference Harness 的 Trial/Attempt 模型可把基础设施重试放在同一 Trial 下，选定一个 canonical Attempt；DeepEval 入口本身并不自动给外部 Target 重试建立这种统计合同。

实现取舍与失败语义

高并发降低墙钟时间，却可能触发 Provider 429、共享 Metric 状态覆盖、输出顺序变化和费用激增。信号量给出上限，但真正安全值还由 Judge 速率限制、内存和对象隔离决定。缓存减少成本和随机漂移，却可能掩盖模型或模板升级；缓存必须可关闭，并在实验身份变化时失效。

ignore_errors 适合长批次收集更多诊断，但发布分析必须先报告评测错误率与缺失模式。将错误样本删除会产生 complete-case 偏差；将 error 当 0 分又混淆产品质量和 Harness 健康。更稳妥的输出是 pass/fail/error 分层，并允许 Gate 在关键错误存在时给 inconclusive。

动手实验

设 100 个测试，90 个完成，其中 72 个通过；5 个 Judge 超时，5 个 Target 错误。分别计算“仅完成项通过率”和“按全体样本通过率”，说明两者各自遗漏什么。再设计一个缓存键：列出必须覆盖的 TestCase、Metric 和依赖字段。最后模拟 max_concurrent=20 导致限流，比较降低并发、Judge 内部 retry 与重跑整个 Trial 的证据差异。

```
python scripts/sources.py verify
python -m pytest tests/test_harness_course_docs.py -q
```

预期输出与答案

仅完成项若把所有错误排除，分母 90、通过率 80%；全体样本分母 100、已证实通过率 72%。前者掩盖 10% 缺失，后者又不能说明错误样本本来会通过还是失败，因此应同时报告 72 pass、18 fail、5 judge error、5 target error，并让 Gate 按预注册规则处理。

缓存键至少包含规范化 TestCase 输入/输出/上下文摘要、Metric 类与配置、阈值之外的测量参数、Judge 模型与 prompt、代码 commit 和依赖版本。限流 retry 是同一评分 Attempt 内的传输恢复；整 Trial 重跑会重新产生 Target/评分证据，必须保留旧 Attempt，不能增加独立样本数。

如何核对

在 `deepeval/evaluate/evaluate.py` 查看三种 Config 如何传入执行器及 CLI finalize 分支；在 `deepeval/evaluate/execute/loop.py` 搜索 Semaphore、tasks、cache_config、error_config、MetricData 与 TestRun update。

本篇不能证明什么

信号量、缓存和错误记录不能证明线程安全、缓存键完整、远程服务稳定或统计无偏。是否允许 ignore_errors 通过 CI 是组织政策，不能由库默认值替代。

[上一节](#) · [下一节](#)

Harbor 与 Terminal-Bench 1: Agent 终端任务怎样成为可核对的 Trial

[上一节](#) · [下一节](#)

本篇要解决什么问题

语言模型基准常把“输入发给模型、比较答案”作为主循环；终端 Agent 评测则必须启动隔离环境、安装或连接 Agent、执行多轮命令、保留日志和文件，再在相同或独立环境中运行 verifier。Harbor 是 Terminal-Bench 后续演进出的通用 Agent Harness/Eval Harness 框架；Terminal-Bench 1 保留了更紧凑的早期 Harness 结构。把两者一起阅读，可以看见 Job/Dataset 如何展开 Trial、环境与 Agent 如何交替、Verifier 如何从容器产物生成 reward，以及结果恢复为什么不能只看一个 resolved 布尔值。

本课程锁定 Harbor 74f0176384cff88b99306770473b4875760c5a21 与 Terminal-Bench 1 d28711d0da2675d0bb1d56de45ae5df6082438a3。课程用 Terminal-Bench 1 解释紧凑基线，再用 Harbor 展示网络策略、资源能力、事件 hook、regrade、多步结果等扩展。它不是两个项目版本差异大全，也不声称 Harbor 的每项能力都来自 Terminal-Bench 1。

先建立源码地图

系统	锁定文件	责任
Harbor Job	src/harbor/job.py	解析任务/Agent, 生成与调度 Trial, 恢复 Job
Harbor Trial	src/harbor/trial/trial.py	环境、Agent、Verifier 生命周期与事件
Harbor Environment	src/harbor/environments/base.py	容器抽象、网络/资源/执行/文件操作
Harbor Verifier/Result	src/harbor/verifier/verifier.py · result.py	奖励解析、异常、耗时与多步结果
Terminal-Bench Harness	terminal_bench/harness/harness.py	Dataset、Agent、并发 Trial 与 resume
Terminal-Bench 数据/结果	dataset.py · models.py	任务选择、TrialResults、accuracy/pass@k

完整调用链

终端 Agent 评测：结果来自环境副作用，不是最后一句文本



- 1 Dataset/TaskClient 解析任务版本与选择条件；Job/Harness 再与 Agent、模型、attempt 数组组合，生成 TrialConfig 或 trial handler。
- 2 运行锁与输出目录固定 dataset、agent、模型、超时、并发和网络等配置；resume 只接受匹配配置，并区分完整结果与残留中间产物。
- 3 Trial 创建 Agent environment，准备任务文件和目录，执行 Agent setup/run，并把 stdout/stderr、trajectory、文件与 token/cost 记录到 Trial 路径。
- 4 Agent 结束后切换网络/环境阶段，启动 verifier。Harbor 可让 verifier 与 Agent 同环境或分离环境，避免 Agent 篡改测试或利用验证期网络。
- 5 Verifier 注入测试、运行测试命令并下载 verifier 目录，从 reward JSON 或文本解析数值；文件缺失、空文件、非法 JSON 分别是验证基础设施错误。
- 6 TrialResult 保存 AgentInfo、AgentContext、VerifierResult、异常、setup/execution timing 和多步 StepResult；Job 汇总各 Trial reward 与统计。
- 7 Terminal-Bench BenchmarkResults 根据每 task 的多次 Trial 计算 accuracy 和 pass@k。Attempt 数是每任务重复观察，不是基础设施 retry；结果解释必须保留 task 聚类。

关键数据结构

Task 描述 instruction、环境构建和 verifier；Dataset 是锁定任务集合。TrialConfig 把一个 Task、一个 Agent/模型、一个 attempt 与环境政策固定为运行单位。AgentContext/AgentResult 保存终端 Agent 轨迹、token 与成本。VerifierResult 保存具名 rewards。TrialResult 还保存异常和各阶段时间。JobResult/BenchmarkResults 是派生汇总，不能替代 Trial 目录中的配置、日志与 verifier 产物。

在统一模型中，Terminal-Bench 的 `n_attempts` 更接近同一 Sample 的多个随机 Trial，而非网络重试 Attempt。Reference Harness 使用 Trial/Attempt 术语时会明确区分：随机重复增加 Trial 分母，基础设施恢复留在同一 Trial 下。跨工具比较必须先翻译术语，不能只看字段名。

实现取舍与失败语义

容器把任务依赖和副作用隔离，使文件与命令可验证；代价是镜像、运行时、网络和资源限制都进入实验身份。Verifier 与 Agent 分离提高防篡改能力，但需要复制工作区或明确共享产物，环境差异也可能制造假失败。resume 节省成本，但必须在锁配置相同且结果完整时复用，残留目录不能冒充完成。

Agent 未解决任务是产品失败；Agent 进程崩溃可能是产品或适配错误；环境构建失败、健康检查失败、Verifier reward 缺失是 Harness 错误；超时还要注明发生在 setup、agent 还是 verifier。只有分层记录，Gate 才能区分“不通过”和“无法判断”。

动手实验

为“在容器中修复一个损坏的配置文件”设计 Task：列出 instruction、初始文件、网络策略、资源限制、Agent 可写路径、隐藏 verifier 和 reward。给同一 Task 安排 3 个随机 Trial，再为第二个 Trial 的容器启动失败安排一次基础设施重试；写出正确的 Trial/Attempt 数量。最后解释为何只保存 reward=1 无法复现或审计。

```
python scripts/sources.py verify
python -m pytest tests/test_harness_course_docs.py -q
```

预期输出与答案

三个随机重复是 3 个 Trial；第二个 Trial 的容器恢复是该 Trial 下的另一个 Attempt，统计分母仍是 3。reward=1 没有说明任务/测试版本、Agent/模型、环境镜像、网络、资源、轨迹、Verifier 命令与产物，也不能判断奖励是否由合法 verifier 生成。

课程测试只验证锁定链接和教材合同，不会拉起 Docker 或运行外部 Agent；具体平台的容器安全与资源隔离需要单独集成测试。

如何核对

从 src/harbor/job.py 追 Job.create、TrialConfig 初始化与 queue hooks；在 src/harbor/trial/trial.py 找环境/Agent/verifier phase；再与 terminal_bench/harness/harness.py 的 resume 和并发逻辑对照。

本篇不能证明什么

容器、隐藏测试、reward 文件和 pass@k 不能证明任务无漏洞、Agent 未利用侧信道、测试覆盖完整或结果可外推到真实软件工程。课程也不把框架自身测试通过扩大为某个 Agent 能力证明。

[上一节](#) · [下一节](#)

Job、Dataset 与 Trial：运行计划怎样固定统计单位

上一节 · 下一节

本篇要解决什么问题

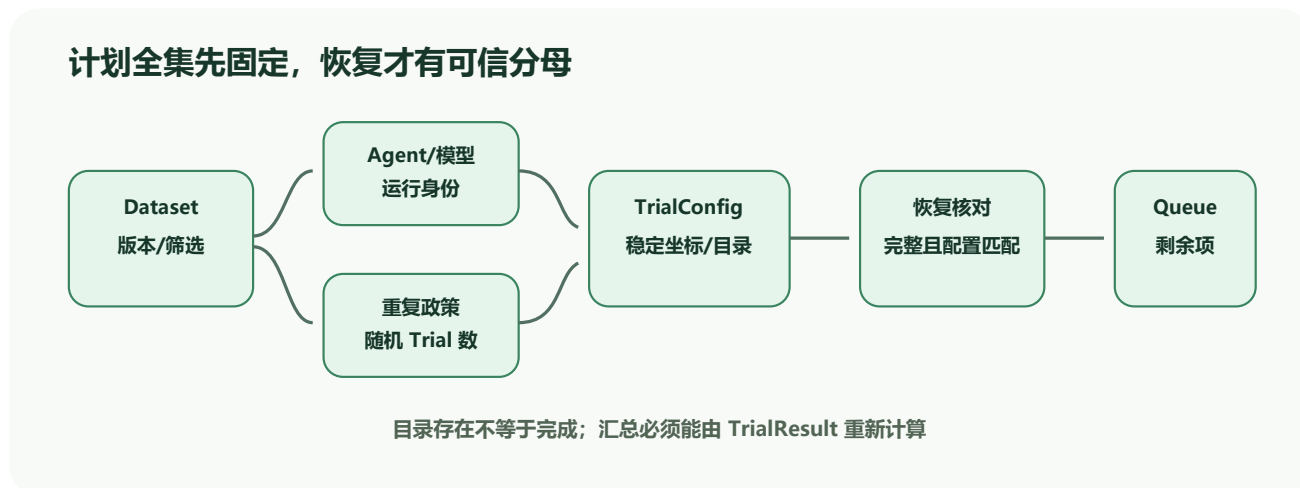
运行 100 个任务、每个 3 次，看似就是 300 个并发工作项；真正实现还要处理任务过滤、不同 Agent/模型组合、恢复既有结果、结果目录冲突、随机重复身份和 Job 级统计。Harbor 的 Job 与 Terminal-Bench 1 的 Harness 都把这些责任放在 Trial 之上。本篇解释计划怎样展开、配置怎样锁定，以及为什么恢复逻辑是统计正确性的一部分。

重点是区分三类对象：Dataset 决定候选任务，Job/Harness 决定本次实验政策，Trial 才是一个 Agent 在一个任务环境中的独立运行。任何一个层级被修改，都可能让历史结果与新结果不可直接合并。

先建立源码地图

源码位置	责任	阅读问题
src/harbor/job.py	Job.create、任务解析、TrialQueue、既有结果	如何生成与恢复 Trial
terminal_bench/harness/harness.py	RunLock、task/attempt 展开和 resume	配置匹配与残留清理
terminal_bench/dataset/dataset.py	数据集缓存、include/exclude、限制与排序	本次实际任务集合如何确定

完整调用链



- 1 Dataset 根据 name/version/path 或 YAML 构造，缓存远程内容，再应用 include、exclude、task_ids 和 n_tasks；排序可以按预计时长改善并发利用率。
- 2 Job.create 解析 Task 配置、Agent skills 与环境资源政策；Terminal-Bench Harness 初始化 Agent class 和 Dataset。
- 3 计划层对 task × agent/model × repetition 展开。每个 Trial 获得稳定名称/UUID、独立输出目录和完整 TrialConfig。

- 4 Job 若发现已有目录，读取 JobConfig、TrialConfig 与 TrialResult，验证身份后把完成项加入 existing results/rewards/stats，并只调度 remaining configs。
- 5 Terminal-Bench 用 RunLock 比较当前 dataset、agent 与 run config；不一致时拒绝 resume。结果文件完整的 Trial 被跳过，只有部分 artifacts 的任务目录会先清理后重跑。
- 6 TrialQueue 按并发策略运行剩余 Trial，并在 START/CANCEL/END 等事件更新 Job 级状态；完成结果逐项写盘，不等待整个 Job 才保存。
- 7 JobResult/BenchmarkResults 由 TrialResults 汇总；统计代码按 task 聚合多次重复，计算 resolved、accuracy 与 pass@k。

关键数据结构

DatasetConfig 固定数据集名、版本、路径和筛选条件。JobConfig/RunLock 固定 Agent、模型、并发、n_attempts、超时、网络和环境政策。TrialConfig 是不可混淆的运行坐标。TrialResult 是已执行证据；“目录存在”或“config 存在”都不能代替它。JobStats 与 BenchmarkResults 是由 TrialResult 派生的缓存，重建时应能从行级结果重新计算。

Terminal-Bench 的 TrialResults 包含 task_name、trial_name、reward 等，BenchmarkResults 先得到每 task 的成功计数再估计 pass@k。这保留了 task 作为聚类单位，避免把 300 条 Trial 当成 300 个不同任务来夸大覆盖。

实现取舍与失败语义

预先展开计划便于进度显示与恢复，但大规模矩阵可能占内存，动态任务还需版本冻结。按预计时长排序提升吞吐，不应改变 Trial 内容；如果全局超时使后排任务更易缺失，排序反而会影响缺失机制，分析时必须记录。

严格 RunLock 防止错误合并，代价是小配置变化也需新 run_id，这是正确成本。清理不完整产物前必须确认目录只属于目标 Trial；已完成结果如果缺摘要或配置不匹配，应拒绝复用。取消、进程崩溃和用户中断要保留状态，不得因没有 reward 就从计划中消失。

动手实验

设 Dataset 有 12 个任务，include 选 8 个，exclude 再去掉 2 个，2 个 Agent、每项 3 个随机 Trial。计算计划规模，并设计稳定坐标。假设完成 30 条、8 条有完整 config 但无 result、其余未启动，写出 resume 应复用、清理与重新调度的数量。再说明若模型版本改变，为什么不能沿用原 run_id。

```
python scripts/sources.py verify
python -m pytest tests/test_harness_course_docs.py -q
```

预期输出与答案

实际任务 6 个，计划为 $6 \times 2 \times 3 = 36$ 个 Trial。稳定坐标至少含 dataset commit/task id、Agent+模型 identity 和 repetition index。30 个完整结果复用；8 个无 result 的描述与总计划矛盾，因为计划仅 36，说明恢复清单本身必须先校验而不能盲算。若改为 4 条不完整，则清理 4 条并重新调度，另 2 条未启动也调度，共 6 条。模型版本变化会改变 Target identity，应建立新 Job。

这个刻意的矛盾训练读者先验证计划全集和集合关系，而不是把日志数字直接相加。

如何核对

在 `src/harbor/job.py` 阅读 `_maybe_init_existing_job`、`_init_trial_configs` 与 `remaining config`；在 `terminal_bench/harness/harness.py` 阅读 `_validate_resume_configuration` 和 `_filter_completed_and_cleanup_incomplete_tasks`。

本篇不能证明什么

计划完整、RunLock 匹配和 resume 成功不能证明 Trial 真的独立、随机种子有效、任务难度代表真实场景或历史结果未被人工修改。证据摘要和不可变存储仍需额外机制。

[上一节](#) · [下一节](#)

Environment 与 Agent

生命周期：副作用怎样被隔离和观察

上一节 · 下一节

本篇要解决什么问题

终端 Agent 不只返回文本，它会安装软件、编辑文件、启动服务、访问网络并消耗资源。Eval Harness 若只截取最后一句输出，就无法知道任务是否真的完成、是否越权或是否污染后续 Trial。Harbor 的 BaseEnvironment 和 Trial 把环境能力、目录、网络阶段、Agent setup/run、日志回调和 verifier phase 组织为生命周期。本篇解释这些边界怎样支撑可核对评测。

我们尤其关注并发隔离：环境对象的 env overlay 和 output callback 若用普通全局变量，会在多个 Trial 间串线；网络若只在容器启动时设置，Agent 与 verifier 的不同权限无法表达；Agent 结束后若立即销毁环境，Verifier 又失去检查文件状态的机会。

先建立源码地图

源码位置	责任	核对问题
src/harbor/environments/base.py	环境能力、资源、网络、exec、文件与 scoped context	副作用和并发如何隔离
src/harbor/trial/trial.py	创建环境、阶段切换、Agent/Verifier 与 hooks	生命周期怎样排序和收尾
terminal_bench/agents/base_agent.py	Agent identity、prompt template 与 perform_task	Harness 与 Agent 的最小契约

完整调用链



1 Trial 根据 Task 环境配置和 Agent 配置创建 BaseEnvironment；环境实现声明 capabilities/resource capabilities，并验证 CPU、内存、存储、GPU 与 compose 支持。

- 2 环境启动时准备工作目录、挂载、默认用户和启动变量，执行 healthcheck。启动超时与构建失败在 Agent 运行前结束 Trial。
- 3 Trial 进入 Agent phase，应用该阶段网络策略和 Agent scoped env。ContextVar 风格的 overlay 与 output callback 只作用于当前 asyncio task，嵌套 scope 按近者优先。
- 4 Agent setup 安装工具或准备依赖；run/perform_task 接收 instruction 与环境/日志路径，通过环境 exec 与文件 API 执行任务，并返回 AgentContext/AgentResult。
- 5 Trial 下载 Agent 日志与轨迹，记录 token/cost 和 setup/execution timing；无论成功失败都要尝试回收输出，避免异常抹掉已发生副作用。
- 6 进入 Verification phase 前切换网络政策。若使用 separate verifier environment，则按策略复制或挂载待验工作区；若 same environment，则明确 Agent 残留进程和权限影响。
- 7 Verifier 完成后停止环境、清理资源并发出 hook。清理错误应附加到 Trial 诊断，不能覆盖最初的 Agent/Verifier 异常。

关键数据结构

EnvironmentConfig 描述 provider、镜像/构建、网络、资源和环境变量；EnvironmentCapabilities 声明实现支持什么，而不是假设所有容器后端等价。ExecResult 保存 stdout、stderr 与退出码。Agent identity 至少包括 name、version、model/provider 和 prompt template。AgentContext 保存运行轨迹与用量。

TrialPaths/EnvironmentPaths 规定 agent、verifier、logs、reward 等跨系统约定路径。

网络计划应区分 baseline、agent phase 和 verifier phase。allowlist/public/disabled 不只是安全设置，也会改变任务可解性，因此属于实验条件。密钥只以受控环境变量注入，不得进入日志、TrialConfig 公共副本或课程材料。

实现取舍与失败语义

抽象 BaseEnvironment 让 Docker、Kubernetes 或云沙箱共享 Trial 逻辑，但 capabilities 必须显式核对；把不支持的资源限制静默忽略会破坏公平性。Context-local overlay 支持并发，仍要求底层 provider 不把可变状态放在进程全局。

Agent setup 失败、Agent timeout、命令非零、环境失联和网络政策切换失败是不同阶段错误。某些任务允许命令非零作为探索过程，因此不能把每个 exec 非零直接判产品失败；最终由 Agent contract 和 Verifier 判断。

Verifier 期间禁止 Agent 网络不能反推 Agent 阶段没有泄漏，网络 enforcement 需平台级测试。

动手实验

设计一个需要启动本地 HTTP 服务的任务，写出环境镜像、端口、CPU/内存、Agent 网络与 Verifier 网络、可写目录和默认用户。列出 Agent setup、run、日志下载、verification、cleanup 各阶段可能失败的两种情况及应保存证据。再分析同一环境验证和分离环境验证的攻击面与复制成本。

```
python scripts/sources.py verify
python -m pytest tests/test_harness_course_docs.py -q
```

预期输出与答案

Agent 阶段若需要安装依赖，可采用 allowlist；Verifier 通常可禁网，避免外部答案影响测试。可写路径应最小化，隐藏 tests 不应在 Agent 阶段挂载。同环境验证能看到完整进程/文件状态但可能被 Agent 篡改；分离环境隔离更强，却必须可靠传递待验工作区并保持 OS/依赖一致。

每阶段证据至少包括开始结束时间、状态、命令/实现 identity、stdout/stderr 摘要、相关 artifact digest 和异常类型。cleanup 失败不能把已得到 reward 改为成功，也不能把原异常丢失。

如何核对

在 `src/harbor/environments/base.py` 核对 capabilities、resource mode、scoped_exec_env、scoped_output_callback、reset/ensure dirs 与 exec 抽象；在 `src/harbor/trial/trial.py` 追 network plan、hooks 和环境收尾。

本篇不能证明什么

声明网络禁用、资源限制和容器隔离不能证明底层平台真正 enforcement、无内核逃逸、无侧信道或并发绝对隔离。需要对具体 Environment provider 做独立安全与故障注入测试。

[上一节](#) · [下一节](#)

Verifier、Reward 与 Result：成功数字怎样回连执行证据

上一节 · 下一节

本篇要解决什么问题

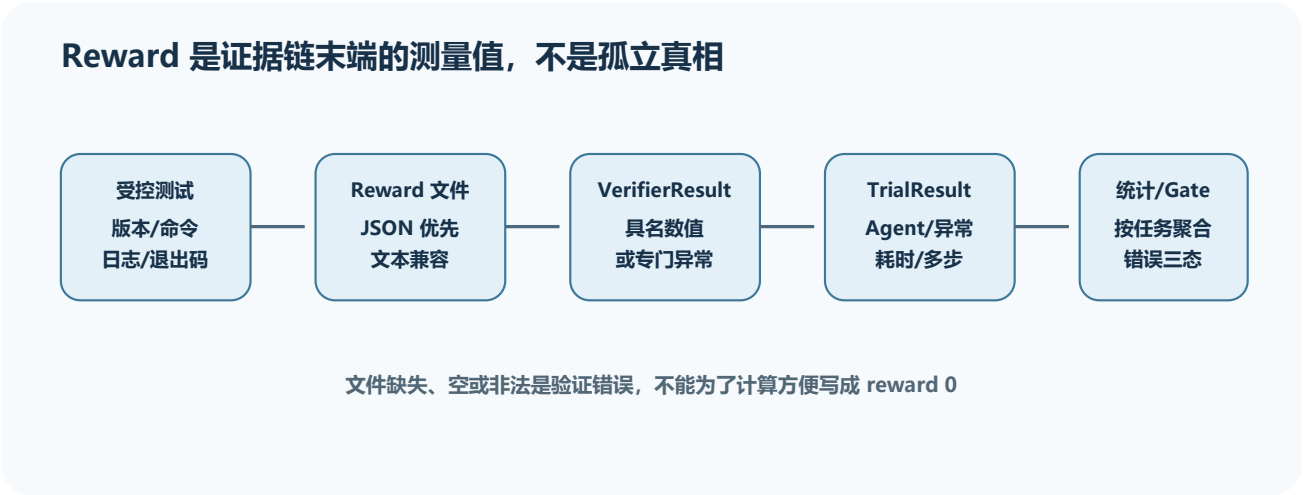
终端任务的最终输出通常是 reward 0/1 或一组具名 rewards，看起来比模型 Judge 更客观；但 reward 文件也可能缺失、为空、格式错误、由错误测试版本生成，甚至被 Agent 提前写入。Verifier 的可信度来自受控测试、阶段隔离和产物链，而不是数字类型。本篇追踪 Harbor Verifier 怎样布置测试、执行并解析 reward，以及 TrialResult/BenchmarkResults 怎样保留异常和汇总。

读完应能区分：测试失败得到合法 reward 0、Verifier 自身崩溃没有 reward、Reward 解析错误、Agent timeout 与 regrade；也能解释 accuracy 和 pass@k 使用的统计单位与限制。

先建立源码地图

源码位置	责任	阅读焦点
src/harbor/verifier/verifier.py	测试目录、Verifier 执行、reward 解析	合法失败与基础设施错误
src/harbor/models/trial/result.py	TrialResult、ExceptionInfo、TimingInfo、StepResult	结果如何保留上下文
terminal_bench/harness/models.py	BenchmarkResults、accuracy、pass@k	多次 Trial 怎样聚合

完整调用链



- 1 Verification phase 解析 Task 的 tests 和 verifier 配置，确定测试来源、执行命令与工作目录；测试文件被加入 verifier environment，而不是暴露给 Agent。
- 2 Verifier 清理/准备 verifier 目录，将 tests 注入环境，运行测试命令并下载 verifier 输出。命令退出状态和日志是诊断证据，但最终 reward 由约定文件提供。

- 3 若存在 reward JSON，解析为具名数值映射；否则尝试 reward 文本并包装为 {"reward": number}。JSON 优先级和路径是外部契约的一部分。
- 4 文件不存在、文件为空、数值/JSON 无法解析、测试目录下载失败分别抛出专门异常。它们不会被伪造成 reward 0。
- 5 成功解析得到 VerifierResult；TrialResult 同时保存 AgentInfo、AgentContext、verifier_environment_mode、exception、各阶段 TimingInfo，以及单步或多步 StepResult。
- 6 Regrade 可复用源 Trial 的 Agent 产物并重新运行 Verifier。新结果必须回连 source trial 与 verifier 版本，不能伪装为 Agent 再次执行。
- 7 Job/Benchmark 聚合每 task 的 Trial reward。accuracy 表示成功 Trial 比例；pass@k 按每 task 的 n 次观察和 c 次成功估计至少一次成功概率，再跨 task 平均。

关键数据结构

VerifierResult 的 rewards 是字符串到数值映射，允许主 reward 与组件 reward；值的语义和范围由 Task/Verifier 契约定义，不能假设都在 0 到 1。ExceptionInfo 保存类型、消息和 traceback；TimingInfo 保存阶段起止/时长；StepResult 为多步任务保存每步 Agent/Verifier/exception。TrialResult.compute_token_cost_totals 会从单步或多步 AgentContext 汇总成本。

BenchmarkResults 的 pass@k 输入是每个 task 的成功计数，而不是把所有 Trial 打散。若每 task 的 n 不同、缺失非随机或 Task 数很小，单个平均值需伴随分母、分布和不确定性。reward 组件也不应未经预注册就事后挑选最佳指标。

实现取舍与失败语义

文件协议让任意语言的 tests 与 Python Harness 解耦，也形成清晰 artifact；代价是路径、格式与写入原子性必须严格。优先 JSON 支持多 reward，文本兼容简单任务。专门异常避免基础设施失败被计为 Agent 失败，是正确的证据边界。

regrade 能在不重复昂贵 Agent 执行的情况下修复 Verifier 或应用新标准，但它改变 Scorer identity。报告必须并列保留原 verifier result，而不是覆盖。pass@k 适合“允许多次尝试”的产品问题；若线上只有一次机会，只展示 pass@10 会误导。发布 Gate 还要处理环境错误、缺失 Trial、任务权重和关键任务非补偿规则。

动手实验

设计 reward JSON：主 reward、功能正确性、安全和资源合规四项，规定合法范围。列出测试失败、reward 文件空、JSON 中出现字符串、下载失败四种情况应得到的 Trial 状态。给两个任务各 3 次 Trial：A 成功 2 次，B 成功 0 次，计算 accuracy、resolved task 数，并描述 pass@1 与 pass@2 的方向。再写出 regrade 证据清单。

```
python scripts/sources.py verify
python -m pytest tests/test_harness_course_docs.py -q
```

预期输出与答案

测试合法运行但断言失败可以写 reward 0；后三种是 verifier/解析/传输错误，不应写 0。总 Trial accuracy 是 2/6，但 resolved task 可按“至少一次成功”定义为 1/2；pass@2 会高于或等于 pass@1，但任务 B 仍为 0。具体估计应由锁定公式计算并报告每 task n/c。

Regrade 至少保存 source trial id 与 artifact digest、新旧 verifier commit/tests digest、环境 identity、regrade 时间、旧/新 rewards 和差异原因。它不产生新的 Agent Trial。

如何核对

在 src/harbor/verifier/verifier.py 顺序核对 _resolve_tests、verify、reward JSON/text 解析和专门异常；在 src/harbor/models/trial/result.py 核对异常、阶段时间与多步字段；最后看 terminal_bench/harness/models.py 的 pass@k 聚合单位。

本篇不能证明什么

隐藏 tests、数值 reward 和 pass@k 不能证明测试无漏洞、reward 与真实价值一致、Agent 没有利用环境，或差异具有统计显著性。发布结论必须再结合任务治理、错误率、不确定性与关键风险 Gate。

[上一节](#) · [下一节](#)

最小 Eval Loop：从冻结规范到可复核报告

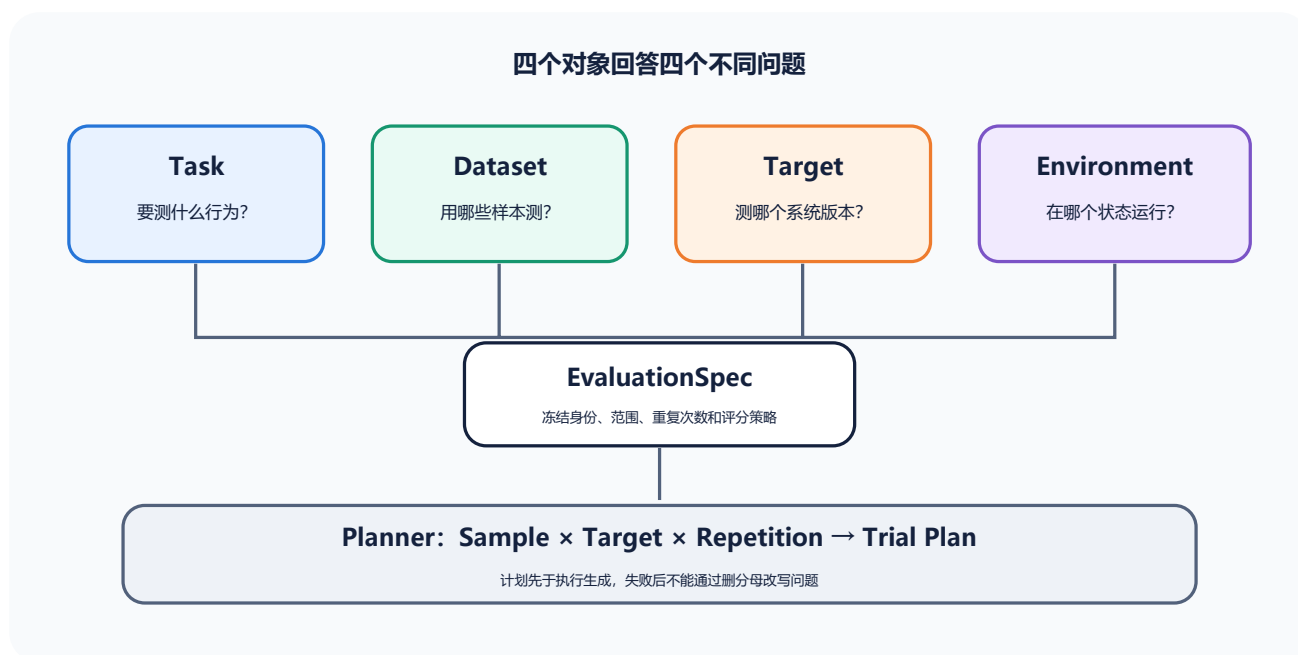
上一节 · 下一节

本篇要解决什么问题

一个 Eval Loop 最容易被写成“读数据、调用模型、算平均分”三行伪代码。真正可复核的实现至少要回答：运行前怎样固定样本和 Target 身份；每个计划项怎样拥有稳定 Trial；基础设施失败怎样留下 Attempt；Target 输出怎样进入 Trace 与 Artifact；Scorer 怎样只读取明确 Observation；Metric 怎样保留预声明分母；Gate 为什么不能直接读取 Target 自报的 success。本篇用 Reference Harness 的运费边界案例贯穿完整闭环。

前置知识是 Python 基础和前七篇共同术语。读完后，你应能从 CLI 一直追到 JSON/Markdown/HTML 报告，并能指出某个结果在哪一层改变状态。Reference Harness 是教学简化：它不实现分布式队列或真实模型调用，但每个关键对象都对应生产系统中必须解决的责任。

核心机制



PipelineConfig 冻结 evaluation_id、Dataset 路径、Target Adapter、repetition、Scorer 和 Gate。

plan_trials 物化 sample × target × repetition，确保分母先于执行确定。run_trial_batch 以有限本地并发执行，但返回顺序仍与计划一致；每个 Trial 内部由 run_trial 管理基础设施 Attempt。成功输出形成两个 TraceEvent 和内容寻址 Artifact，再由 build_observation_bundle 绑定 canonical Attempt。

评分器 FieldMatchesExpectedScorer 从 target_completed 事件取 output 与 expected，不信任 Target 的自我评价。aggregate_pass_rate 用计划 Trial ID 作为 denominator，缺少 Score 不会缩小分母。evaluate_gate 先检查无效/不可评分证据，再应用预声明 threshold。write_report 从同一 EvaluationReport 生成机器可读 JSON、审阅用 Markdown 和独立 HTML。

完整流程

以 reference/examples/shipping/eval.yaml 为入口：

- 1 CLI run 检查配置路径，将错误压缩为中文消息而不泄漏 traceback。
- 2 Pipeline 在配置目录内安全解析 dataset/script，拒绝绝对路径和 .. 越界；Dataset JSONL 每行验证为 Sample。
- 3 Planner 为三个金额、两个 Target、一次 repetition 生成六个稳定 Trial；Trial ID 包含运行、Target、Sample 与重复身份。
- 4 SubprocessTarget 通过 argv 和 stdin JSON 调用本地脚本，禁用 shell；超时/启动失败抛 InfrastructureError，非零退出或非 JSON 输出是 product failure。
- 5 Runner 只重试 InfrastructureError；一旦 Target 返回有效产品失败或完成结果，就建立唯一 canonical Attempt，不用重试“刷过”。
- 6 TraceWriter 强制事件序号和父事件存在；ArtifactStore 以 SHA-256 命名输出；ObservationBundle 摘要覆盖 Trial、Attempt、事件与 Artifact 引用。
- 7 Scorer、Metric 和 Gate 逐层产生不可变对象；报告与 evidence/run 文件最后写入输出目录。

关键数据与不变量

计划对象是 EvaluationSpec → Sample → Trial；执行对象是 Attempt → TrialResult；证据对象是 TraceEvent → ArtifactRef → ObservationBundle；判断对象是 ScoreRecord → MetricEstimate → GateDecision。必须保持：一个 Trial 最多一个 canonical Attempt；Bundle 只能绑定该 Attempt；Score 必须携带 Bundle digest 与 scorer_id；Metric denominator 来自 Trial plan；Gate 不能把 invalid/unscorable 证据变成 passed。

错误状态分层也不能压成一个布尔值。Attempt 有 succeeded/infra_failed/cancelled，Trial 有 completed/blocked/invalid，Score 有 passed/failed/uncertain/unscorable/invalid，Gate 有 passed/failed/blocked/inconclusive。报告显示的“fixed-release passed”是 Gate 结论，不是脚本退出码。

动手实验

从仓库根目录运行：

```
uv run eval-harness-ref run reference/examples/shipping/eval.yaml --output output/shipping
uv run eval-harness-ref inspect output/shipping
uv run eval-harness-ref score output/shipping
uv run eval-harness-ref gate output/shipping
```

打开 output/shipping/evidence.json，任选金额 100 的 buggy Trial，沿 trial_id → canonical_attempt_id → bundle digest → score_id → metric_id → gate_id 写出链路。然后修改输出 Artifact 的一个字节，再运行 inspect，观察摘要门禁。

预期输出与答案

运行会报告 buggy-release: failed、fixed-release: passed；计划 Trial 为 6，两个 pass-rate 的 denominator 各为 3。金额 100 的 buggy 输出 fee=10，期望 fee=0，因此 Score failed；buggy 的通过数 2/3，低于 minimum=1.0。篡改 Artifact 后 inspect 应以“Artifact 摘要不一致”失败，而不是仍显示原 Gate。

若重新 score/gate，不会重跑 Target；它们只消费冻结 evidence。这展示“执行证据”和“政策判定”可分离，但前提是 evidence 先通过完整性检查。

如何核对

从 cli.py 的 run 进入 pipeline.py, 依次追 _load_samples、plan_trials、run_trial_batch、build_observation_bundle、scorer、metric、gate 和 write_report。测试入口是 test_shipping_e2e.py 与 test_cli.py。

本篇不能证明什么

确定性案例全绿只证明这套小型实现满足其测试合同，不证明真实模型稳定、数据集代表线上分布、并发适合大规模运行或 Gate 可以授权生产发布。Reference Harness 是可执行共同语言，不是企业控制面。

[上一节](#) · [下一节](#)

Run Identity 与可复现性：保存“实际运行了什么”

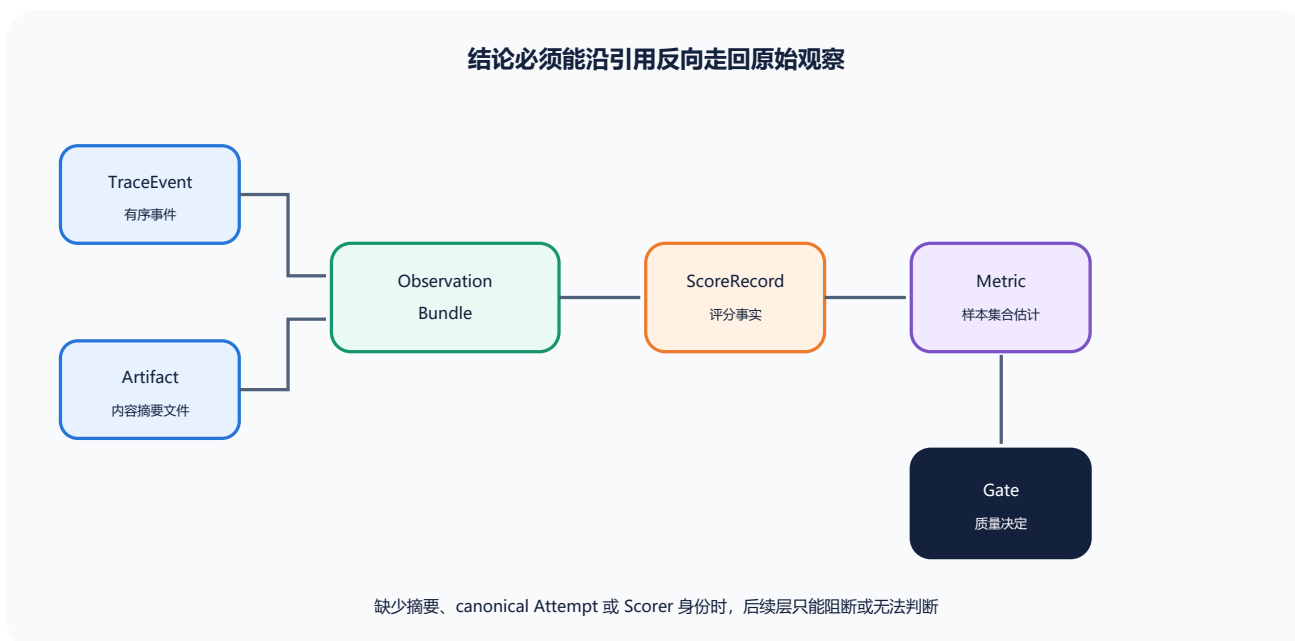
上一节 · 下一节

本篇要解决什么问题

“使用 model-x 在 dataset-v2 上运行”不是可复现身份。模型别名可能漂移，Dataset 文件可被覆盖，脚本依赖当前工作目录，环境变量能改变行为，Scorer prompt 与阈值也可能后来修改。Eval Harness 必须在执行前把声明身份解析为实际身份，并用内容摘要把配置、输入、代码与产物连起来。本篇从 canonical_digest、安全相对路径、Trial ID 和 Artifact digest 说明最小做法。

读完后，你应能设计 RunManifest，区分 logical name、resolved identity 和 content digest；也能判断两次报告是否具备直接成对比较条件。这里的“可复现”不是保证未来远程 API 返回同样文本，而是保证读者知道当时使用了什么，并能在可控组件上重新构造。

核心机制



Reference Harness 使用规范 JSON 序列化：字典键排序、固定分隔符、UTF-8，再计算 SHA-256。相同语义映射不因键顺序变化而改变 digest。Planner 将 EvaluationSpec digest 的一部分写进 run_id，再把 target_id、sample_id、repetition 组合为 Trial ID。ArtifactStore 对真实 bytes 求摘要，Bundle digest 再覆盖 Artifact 引用和 Trace 事件。

身份不能只靠摘要：摘要回答“内容是否相同”，字段回答“内容是什么角色”。RunManifest 应同时保存 evaluation_id、源码 commit、Python/依赖版本、Dataset digest、Target Adapter identity、实际模型/镜像、Scorer identity、Gate policy、随机种子和时间。秘密只记录来源名称，不记录值。

完整流程

- 1 用户配置是 declared identity；加载器验证 schema，拒绝未知字段，避免拼错配置被静默忽略。

- 2 配置路径解析必须局限于配置目录。绝对路径或 .. 越界会让同一配置读取机器上的不同文件，因此 `_safe_input_path` 直接拒绝。
- 3 Dataset 每行解析为带稳定 `sample_id` 的 Sample；内容摘要应覆盖规范化输入与期望，而 `sample_id` 只负责逻辑引用。
- 4 Target 字符串解析为具体 Adapter 和参数。本地脚本保存相对路径及文件 digest；真实 API 还要记录服务返回的实际模型版本（若可得），不能把别名假装成不可变版本。
- 5 Planner 依据冻结 Spec 生成全量 Trial ID。恢复时必须对 manifest/digest，而不是只检查相同文件名。
- 6 每次 Attempt 继承 Trial identity，并增加 ordinal。canonical 标记是执行恢复结果，不改变 Trial 的统计身份。
- 7 Artifact 与 Bundle 用内容摘要形成 lineage；inspect 重新读取 bytes 校验摘要，防止引用存在但内容已被替换。
- 8 Comparison 只在 candidate/baseline 共享 `sample_id + repetition` 且测量合同相同时配对。

关键数据与不变量

逻辑身份例如 `target_id=fixed`，解析身份例如脚本相对路径、digest、解释器版本，运行实例身份例如 `trial_id` 和 `attempt_id`。三者都要保留。Digest 必须带算法前缀 sha256:，否则未来算法迁移无法解释。Artifact `relative_path` 必须是运行目录内规范路径；验证时不能跟随它逃出证据根目录。

直接比较的最小不变量：Dataset 内容/切分相同，Sample 配对键相同，Target 之外的执行政策相同，Scorer/Metric 定义相同，缺失规则预先确定。如果 Judge 模型或 prompt 变化，即使 `metric_id` 相同，也不是同一测量合同。

动手实验

运行身份测试：

```
uv run pytest tests/test_identity.py tests/test_models.py tests/test_runtime_extensions.py -q
uv run eval-harness-ref run reference/examples/shipping/eval.yaml --output output/identity-demo
```

把 `evidence.json` 中某个 Artifact `relative_path` 改为 `../README.md`，再执行 `inspect`；随后恢复文件，改动 Artifact bytes 再执行。最后复制 shipping 配置，仅把 `target_id` 改名但脚本不变，回答两次运行是“内容相同”“逻辑身份相同”还是“可直接比较”。

预期输出与答案

路径越界在加载 `ArtifactRef` 时失败；bytes 篡改在摘要核实时失败。仅改 `target_id` 时，脚本内容可能相同，但逻辑 Target identity 已改变，报告坐标也改变；如果明确记录 rename mapping，可以说明实现内容等价，但不能仅凭 digest 自动断言实验条件完全相同，因为其他配置与运行环境也需核对。

`canonical_digest` 对字典键顺序稳定，但列表顺序有语义，所以改变样本、Target 或事件顺序应改变 digest。SHA-256 证明内容一致性，不证明来源可信或文件无恶意。

如何核对

阅读 `identity.py`、`planner.py`、`artifacts.py` 与 `models.py` 的路径验证。再查看 `test_identity.py` 和 Artifact 安全测试。

本篇不能证明什么

内容摘要、锁文件和稳定 ID 不能保证远程模型确定、容器镜像供应链安全、时间依赖服务可重放或作者身份真实。它们建立可审计基础，不是密码学签名或可重复科学实验的全部条件。

[上一节](#) · [下一节](#)

Retry 与 Recovery：恢复基础设施，不能重抽产品答案

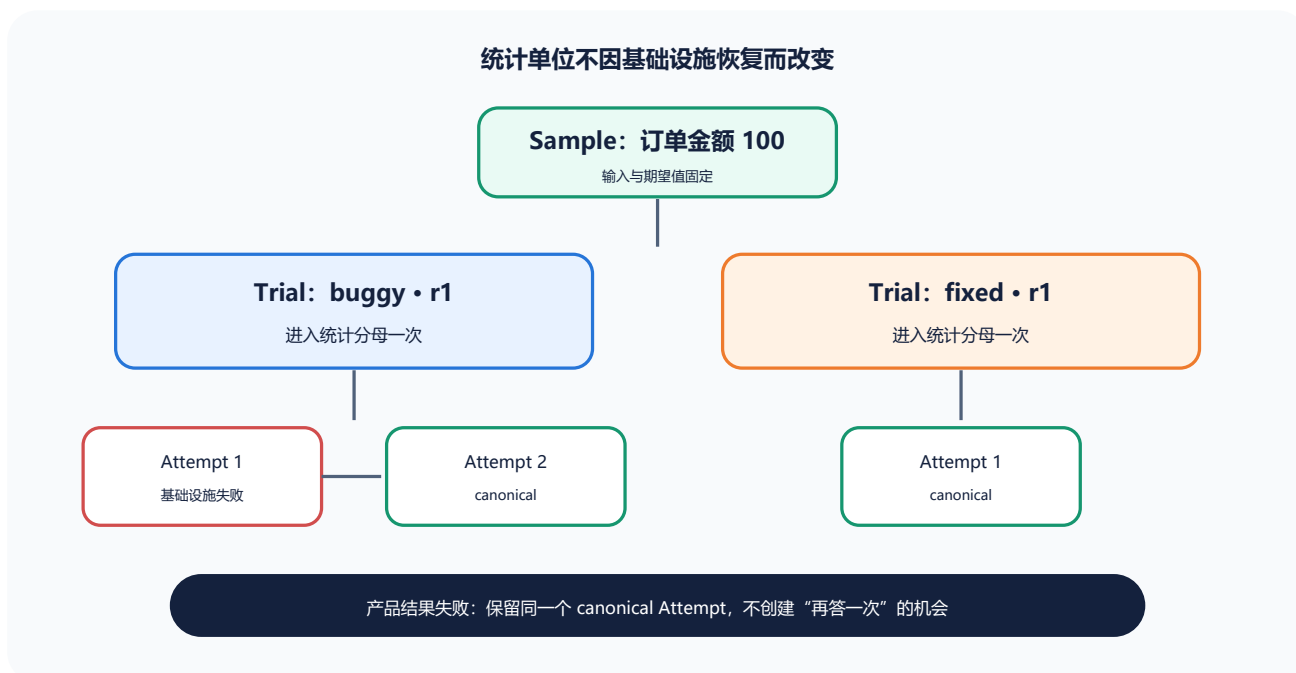
上一节 · 下一节

本篇要解决什么问题

如果模型第一次答错、第二次答对，Harness 能不能自动重试并记为通过？不能，除非产品本身的服务契约就包含重试，而且这段行为属于被测 Target。Eval Harness 的 retry 只应恢复“没有得到有效产品观察”的基础设施故障。否则失败样本会被不断重抽直到成功，分母和失败概率一起被改写。本篇用 Trial/Attempt、不同行为预算和 canonical Attempt 建立严格边界。

前置知识是 Sample/Trial/Attempt。读完后，你应能给 timeout、429、进程启动失败、模型拒答、工具错误、断言失败和日志写入失败分类，并设计不会通过恢复机制作弊的状态机。

核心机制



Trial 是预先进入统计计划的观察单位；Attempt 是同一 Trial 的基础设施执行记录。Reference Harness 的 `run_trial` 捕获 `InfrastructureError`，写入 `infra_failed Attempt` 后按 `max_infra_attempts` 继续。一旦 Target 返回 `TargetResult`，不论 `kind` 是 `completed` 还是 `product_failure`，都将当前 Attempt 标为唯一 canonical 并结束重试。所有基础设施 Attempt 用尽后，Trial 为 `blocked`，没有 `Bundle` 和 `Score`；Metric denominator 仍包含它。

产品服务内部的退避重试属于 Target 行为：例如应用本身遇到 429 后按线上政策重发，Harness 只能在 Trace 中观察。Harness retry 则是运行器无法启动进程、沙箱临时失联等实验基础设施恢复。两种 retry 即使都叫 `retry`，也必须用不同预算和事件类型。

完整流程

- 1 Planner 固定 Trial 计划与 `max_infra_attempts`, 开始后不能根据结果追加 Trial。
- 2 Runner 创建 Attempt ordinal=1 并调用 Target Adapter。
- 3 Adapter 在进程无法启动、Harness 级 timeout 等情况下抛带 code 的 `InfrastructureError`; Runner 记录失败但不生成产品 output。
- 4 若预算剩余, 创建下一个 Attempt。每次 Attempt 都应保存开始/结束、错误、资源与 Trace, 不能覆盖前一次。
- 5 Target 有有效返回时建立 canonical Attempt。非零退出、拒答、无效 JSON 若定义为产品 contract 失败, 也停止重试。
- 6 ObservationBundle 只绑定 canonical Attempt, 但审计报告仍保留所有 Attempt, 解释为何选中它。
- 7 若无 canonical Attempt, Trial blocked; Scorer 不应伪造 0 分。Metric 保留计划 denominator, Gate 可因关键 blocked evidence 给 inconclusive/blocked。
- 8 分布式实现还需要 lease、fencing token 和原子 canonical commit; 最小本地实现不假装已解决这些问题。

关键数据与不变量

AttemptStatus={succeeded, infra_failed, cancelled}; TrialStatus={completed, blocked, invalid}。Attempt ID 由 Trial ID + ordinal 得出。一个 Trial 最多一个 canonical; canonical 必须是 succeeded; 产品失败仍可由 succeeded Attempt 产生, 因为“成功执行”与“产品通过”是两层语义。

预算至少区分 Harness infra attempts、Target 内部调用/工具预算、Judge 预算和全局墙钟预算。可恢复错误名单要白名单化; 将所有 Exception 都视为可重试会掩盖确定性程序错误。取消不应自动继续; 它可能来自用户、全局预算或上游任务撤销。

动手实验

运行 Runner 测试:

```
uv run pytest tests/test_runner.py -q
```

阅读三个测试 Target: 第一次 `InfrastructureError` 后成功、直接 `product_failure`、所有尝试都 `InfrastructureError`。为每种情况画出 Attempt 列表、canonical 标记、TrialStatus、是否产生 Bundle, 以及 Metric denominator。再假设产品模型第一次输出错误答案, 写出为何不应触发 Harness retry。

预期输出与答案

基础设施先失败后成功: 两个 Attempt, 第二个 canonical, Trial completed; 直接 `product_failure`: 一个 succeeded canonical Attempt, Trial completed, Scorer 后续可能 failed; 全部 infra failure: 达到预算数量的 infra_failed Attempt, 无 canonical, Trial blocked, 无 Score, 但仍在计划 denominator。

错误答案是有效产品观察, 应进入 Score failed。若业务 Target 自己定义“最多请求模型三次再投票”, 这三次是单个 Trial 内的 Target Trace, 而不是 Harness 的三个 Attempt, 也不能当成三个独立样本。

如何核对

查看 runner.py 的循环与返回分支、targets/base.py 的错误合同、test_runner.py 的三种状态。再结合 metrics.py 核对 blocked Trial 是否仍占分母。

本篇不能证明什么

本地线程与单进程 canonical 规则不能证明分布式 worker 恰好一次、远端请求幂等或崩溃恢复无重复副作用。生产队列需额外 lease、fencing、幂等键与事务性提交。

[上一节](#) · [下一节](#)

LLM-as-a-Judge: 先定义测量合同，再接入模型

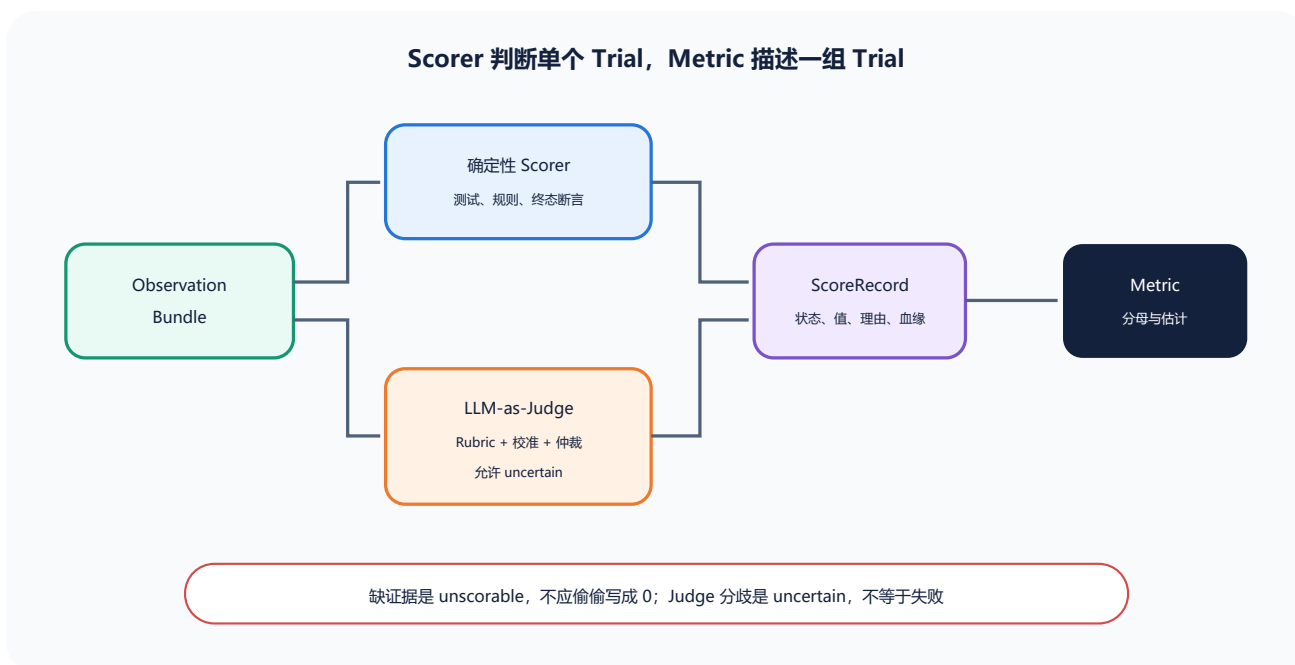
上一节 · 下一节

本篇要解决什么问题

开放式回答、合同风险和多轮 Agent 轨迹很难用精确匹配评分，LLM-as-a-Judge 因此常被直接加入评测。然而 “让更强模型打 0 到 1 分” 没有说明 rubric、输入可见性、输出 schema、失败/弃权、位置偏差、校准数据和成本。Judge 是有不确定性的测量器，不是事实裁判。本篇从 Reference Harness 的离线 Judge Protocol 出发，设计可替换但可核对的适配层。

核心路线默认不访问网络。JudgeScorer 接受任何实现 `judge_id + judge(ObservationBundle)` 的对象，离线 Stub 与真实 API 使用同一结果合同。这样测试可以锁住 lineage，而不会把模型凭据或漂移带进基础门禁。

核心机制



Judge 输入应来自冻结 ObservationBundle，明确选择 output、reference、trace、tools 和 artifacts，不能读取隐藏标签或候选名称造成泄漏。JudgeResult 保存 value、passed、reason；JudgeScorer 再增加 scorer_id、judge_id、Bundle digest、canonical Attempt 和 ScoreStatus。评分模型调用 retry 属于 Judge 基础设施，不应重新运行 Target。

可靠接入需要三层：Rubric 定义构念和可观察证据；Judge Adapter 固定模型、prompt、schema、参数和重试；Calibration 用人工双标/仲裁数据测量一致率、偏差、稳定性与阈值。Release Eval 使用冻结 Judge 版本，训练 reward 使用的 Judge 与独立发布 Judge 需要隔离。

完整流程

- 1 从业务风险定义 rubric 条目和不可补偿规则，给出 passed/failed/uncertain/unscorable/invalid 的可操作条件。
- 2 构造 Judge input view，只含授权 Observation；对 Candidate/Baseline 比较时随机化顺序并隐藏系统名称。

- 3 固定 Judge identity: provider、model/version、系统提示、rubric、few-shot、温度、seed（若支持）、输出 JSON schema、解析器 commit。
- 4 调用 Judge，验证结构化输出和值域。解析失败、超时和拒绝属于 Judge error，不是产品 0 分。
- 5 JudgeResult 经 JudgeScorer 转为 ScoreRecord，lineage 回到 Bundle；reason 作为诊断，不当作模型思维链或事实证明。
- 6 在独立校准集上与至少两位人类标注和仲裁结果比较，报告分层一致率、混淆矩阵、阈值敏感性和重复测量方差。
- 7 未达到预注册质量要求时，Judge 结果为 uncertain/unscorable 或进入人工复核，而不是调 prompt 直到发布通过。
- 8 运行中记录 token、成本、延迟、缓存与错误；报告同时展示 Judge error rate。

关键数据与不变量

Judge identity 与 Scorer identity 都必须版本化。Rubric 改一个词、模型别名漂移或输出解析器改变，都可能改变测量合同。reason 不应包含隐藏 chain-of-thought，只需给出可审计的判定依据、引用证据和 rubric 条目。若 Judge 可以 abstain，ScoreStatus 应表达 uncertain/unscorable，不把它压成 failed。

成对 Judge 需要 pair key 和展示顺序；两次交换 A/B 可检测位置偏差。多 Judge 投票的统计单位仍是原 Trial，不是 Judge 数量。人工仲裁结果应独立存储，不能覆盖原始分歧。

动手实验

运行离线接口测试：

```
uv run pytest tests/test_runtime_extensions.py -k judge -q
```

为合同风险案例写一个三档 rubric: high/medium/low，并增加 uncertain 条件。定义 StubJudge 返回 value=0.8、passed=true、reason；沿输出检查 ScoreRecord 的 trial_id、canonical_attempt_id、bundle digest 和 scorer_id。再列出真实 Judge 接入时必须新增的配置与校准报告字段。

预期输出与答案

离线测试不调用网络，Score 应保留 value=0.8、reason “满足离线规则” 和完整 lineage。真实配置至少增加 judge_id、provider/model version、prompt/rubric digest、schema、参数、重试/超时、cache policy、成本和校准数据版本。

若 Judge 超时，应产生 Judge error/不可评分，不把产品计 0；若两位人工分歧且仲裁未完成，应是 uncertain；若 Judge 与人工在关键高风险条目上系统性漏判，即使整体一致率高也不能用于 release gate。

如何核对

阅读 scorers/judge.py 的 Protocol、JudgeResult 与 lineage 适配，核对 test_runtime_extensions.py 的离线 Stub。再对比 scorers/rules.py，理解确定性规则与 Judge 的失败面不同。

本篇不能证明什么

结构化输出、reason、多人投票或高总体一致率都不能证明 Judge 无偏、理解真实业务或适合高风险发布。校准结论只适用于其数据分布、rubric 和模型版本。

[上一节](#) · [下一节](#)

统计比较：先配对 Trial，再看效果量与区间

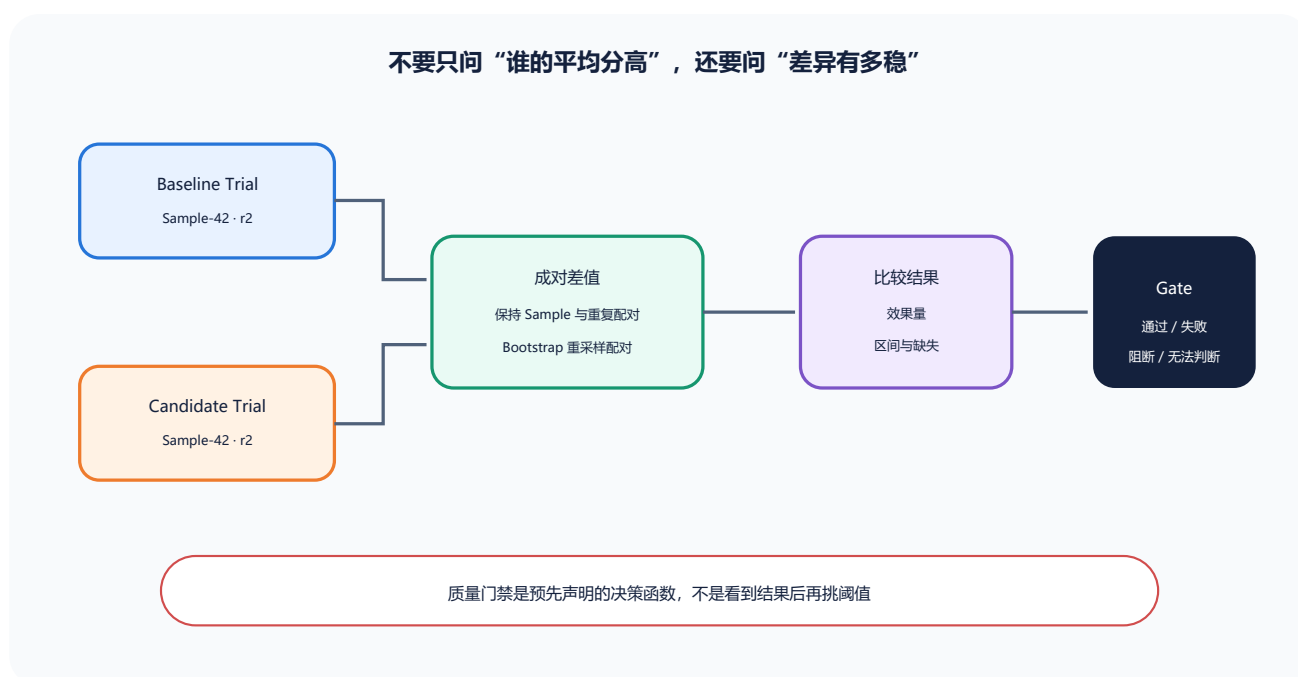
上一节 · 下一节

本篇要解决什么问题

Candidate 通过率 82%、Baseline 80%，能否发布？只看两个平均数无法回答。它们是否运行在相同 Sample 上、重复次数是否一致、缺失是否偏向某个 Target、2 个百分点来自哪些任务、区间是否跨过 0、关键风险是否反向，都决定结论。Eval Harness 的比较层必须在指标与 Gate 之间显式存在。本篇用 Reference Harness 的配对 Bootstrap 和 shipping candidate/baseline 演示最小实现。

前置知识是 Trial 计划和 Metric。读完后，你应能构造 pair key、计算每对差值、解释均值差与置信区间，并识别伪重复、可选停止和只挑赢家指标等常见错误。

核心机制



配对设计让每个 Sample/repetition 同时经历 Candidate 与 Baseline，差值抵消任务难度。Reference Harness 用 `sample_id:r{repetition}` 作为 pair key，把 Score value 对齐；只有两边都有有效 value 的 pair 进入差值。paired_bootstrap 对差值向量有放回采样，固定 seed 和 iterations，返回 pair_count、mean_difference、2.5% 与 97.5% 分位数。

Bootstrap 是教学级基础区间。真实实验可能有 task family、用户、会话等聚类，需要 cluster bootstrap 或分层模型；重复 Trial 也不是完全独立 Sample。统计方法必须在看结果前预注册，并与产品决策的最小有意义差异相连。

完整流程

- 1 计划阶段固定 candidate/baseline Target、共享 Dataset、repetitions、pair key、主指标、次指标和缺失策略。
- 2 两个 Target 运行时保持相同 Sample 顺序不是必要条件，但身份、环境和 Scorer 合同必须一致。

- Score 层保留每个 Trial value 和状态；blocked/unscorable 先分层报告，不能在比较函数中悄悄删掉。
- compare_targets 从 Evidence 找到 Trial 的 sample_id/repetition，从 Report 找到有效 score，形成两张 key → value 映射。
- 取交集配对，计算 candidate - baseline 差值。若无共享键则直接失败。
- Bootstrap 对 pair 差值重采样；seed 固定使教材测试可重复。输出平均差与区间，不把区间端点解释为参数有 95% 概率位于其中。
- Gate 将统计结果与预注册 margin、关键指标非劣要求、错误率和成本约束结合；不能用“均值为正”代替全部政策。
- 报告展示 pair_count、缺失 pair、分层效果和单例回归，支持定位而非只公布赢家。

关键数据与不变量

统计单位是预声明 Trial，配对单位通常是 Sample + repetition；若同一用户贡献多个 Sample，用户可能才是 bootstrap cluster。Candidate/Baseline Score 必须同量纲、同 Scorer 版本。pair_count 应等于有效交集，计划 pair_count 和缺失原因另行报告。Seed 与 iterations 是分析身份。

效果量不仅是均值差：二元指标可报告差值、相对风险或 McNemar discordant pairs；连续指标可报告均值/中位数差；延迟和成本常呈长尾，需要分位数。多指标必须声明主次或校正，禁止事后挑选唯一显著结果。

动手实验

先运行 shipping，再执行 compare：

```
uv run eval-harness-ref run reference/examples/shipping/eval.yaml --output output/compare
uv run eval-harness-ref compare output/compare --candidate-target fixed --baseline-target
buggy --seed 17 --iterations 2000
uv run pytest tests/test_metrics.py tests/test_cli.py -k "bootstrap or compare" -q
```

手算三个 pair 的差值：金额 99、100、101。再删除 Candidate 的一个 Score，回答 pair_count、计划分母和 Gate 应怎样变化。

预期输出与答案

三对值分别是 0、1、0 (fixed - buggy)，mean_difference 为 1/3，pair_count=3，Bootstrap 区间因样本极小可能包含 0。这个例子显示效果方向但不足以支撑普遍结论。删除一个 Candidate Score 后有效 pair_count 变 2，但计划 Trial 分母不变；报告必须显示缺失，Gate 应 blocked/inconclusive，而不是用剩余两对宣布更高胜率。

固定 seed 时测试结果稳定；改变 seed 可能轻微改变有限迭代分位数，但不会改变原始三对差值。

如何核对

阅读 comparison.py 和 pipeline.py 的 compare_targets，再看 test_metrics.py 与 CLI compare 测试。把每个映射 key 与 Evidence Trial 手工对齐。

本篇不能证明什么

Bootstrap 区间不修复数据泄漏、系统性缺失、无效 Scorer、任务分布偏差或可选停止。小样本区间也不能替代领域风险判断和关键案例审阅。

[上一节](#) · [下一节](#)

Agent Environment: 把命令、文件和终态变成评测观察

上一节 · 下一节

本篇要解决什么问题

评测 Agent 时，最终文本往往不是产品结果。修复代码要看 diff 与测试，退款操作要看真实副作用，RAG Agent 要看检索与 ACL，终端任务要看文件、进程和服务终态。Agent Environment Harness 负责创建、重置、限制、观察和验证实验环境；Agent Harness 负责 Agent loop。两者接口是 Trace、Artifact 和 environment final state。本篇结合 Harbor/Terminal-Bench 课程与 Reference Harness 的 Agent Trace Import 说明最小适配。

读完后，你应能设计一个环境生命周期，区分 Agent phase 与 verifier phase，决定哪些日志/文件进入 Observation，并知道环境错误为何不能记为产品失败。

核心机制

阶段边界决定权限，也决定错误归属



清理失败应附加诊断，不能覆盖更早发生的 Agent 或 Verifier 异常

环境规范至少固定镜像/仓库 commit、初始状态、用户、工作目录、可写范围、网络、资源、秘密注入、超时与 verifier。每个 Trial 从干净 snapshot 创建环境，Agent 在受控权限下运行，Harness 收集结构化 Trace、stdout/stderr、diff、终态和成本。随后在相同或分离 verifier 环境执行隐藏断言，最后销毁或隔离保留现场。

Reference Harness 不启动容器，但 AgentTraceImportTarget 展示 Adapter 边界：读取外部 JSONL，验证 sequence、event_id 和 parent 关系，从最终事件暴露可评分 output。它不会推断 Agent loop，也不会采集隐藏 chain-of-thought。

完整流程

- 1 TaskSpec 固定初始仓库/镜像、instruction、允许修改范围和 verifier 版本；Dataset 提供多个任务实例。
- 2 Environment provider 声明 capabilities。请求网络禁用、CPU 限制或 compose 时，provider 不支持就应阻断，而非静默忽略。
- 3 每个 Trial 创建干净环境并健康检查。build/start timeout 属于 Harness infra，允许受控 Attempt 恢复。

- 4 Agent Adapter 在 agent phase 注入最小凭据和环境变量，执行 setup/run；命令、工具调用、文件变化、模型事件形成 Trace 与 Artifact。
- 5 Agent 停止后收集日志，即使异常也尽量保存已发生行为；Trace 父子/序号验证确保基本因果闭合。
- 6 切换 verifier phase 的网络和权限。隐藏 tests 不在 Agent phase 可见；分离环境时只传递允许的工作区状态。
- 7 Verifier 输出确定性 reward/组件结果或明确 error；环境构建、复制、reward 缺失不能伪装成 0 分。
- 8 reset/cleanup 验证残留资源；最终状态与清理错误进入 Trial 诊断，下一 Trial 不复用污染环境。

关键数据与不变量

Environment identity 包括 provider/version、镜像 digest、Task files digest、资源/网络 policy、OS/架构。TraceEvent 只保存可观察行为，不保存模型私有思维链。Artifact 包括 diff、测试日志、终态快照与 trajectory，均需内容摘要。Verifier identity 与 Agent identity 分离。

同一任务的随机重复必须从等价初始状态开始。环境 reset 失败使后续 Trial invalid/blocked，而不是继续运行。Agent 允许访问的凭据和网络域必须进入安全审计，但秘密值不得写入 Artifact。Verifier 不应信任 Agent 自报成功。

动手实验

先验证 Agent Trace 导入：

```
uv run pytest tests/test_runtime_extensions.py -k trace -q
```

创建两行 JSONL：tool_call 与以它为 parent 的 agent_completed，后者 payload.output 写入 {"answer": 42}；用 agent_trace Target 运行。然后把 parent 改成不存在的 ID，观察阻断。为代码修复任务再写一份环境清单：初始 commit、可写目录、禁网、CPU/内存、Agent timeout、隐藏 tests、diff 和 test log。

预期输出与答案

合法 Trace 导入结果包含 trace_event_count=2、两个 type 和 final_output；断裂 parent 应以 Trace 无效失败，不产生正常评分。代码修复的 Scorer 应在干净 verifier 环境应用 diff/检查工作树后运行隐藏 tests，不能只读取 Agent 末尾说“已修复”。

环境启动失败允许 Harness Attempt；隐藏 tests 合法失败是产品失败；tests 本身缺失是 verifier error；清理失败发生在结果之后，也要附加诊断并阻断环境复用。

如何核对

阅读 targets/trace_import.py 与对应测试；再对照 Harbor 环境课程的锁定源码，核对生产级 Environment 还需要哪些能力。

本篇不能证明什么

JSONL 因果验证不能证明事件真实发生、容器无逃逸、网络策略真正 enforcement 或隐藏测试无漏洞。Trace 是声明性证据，仍需可信采集和环境隔离。

上一节 · 下一节

Quality Gate：把证据转成三态/四态决定

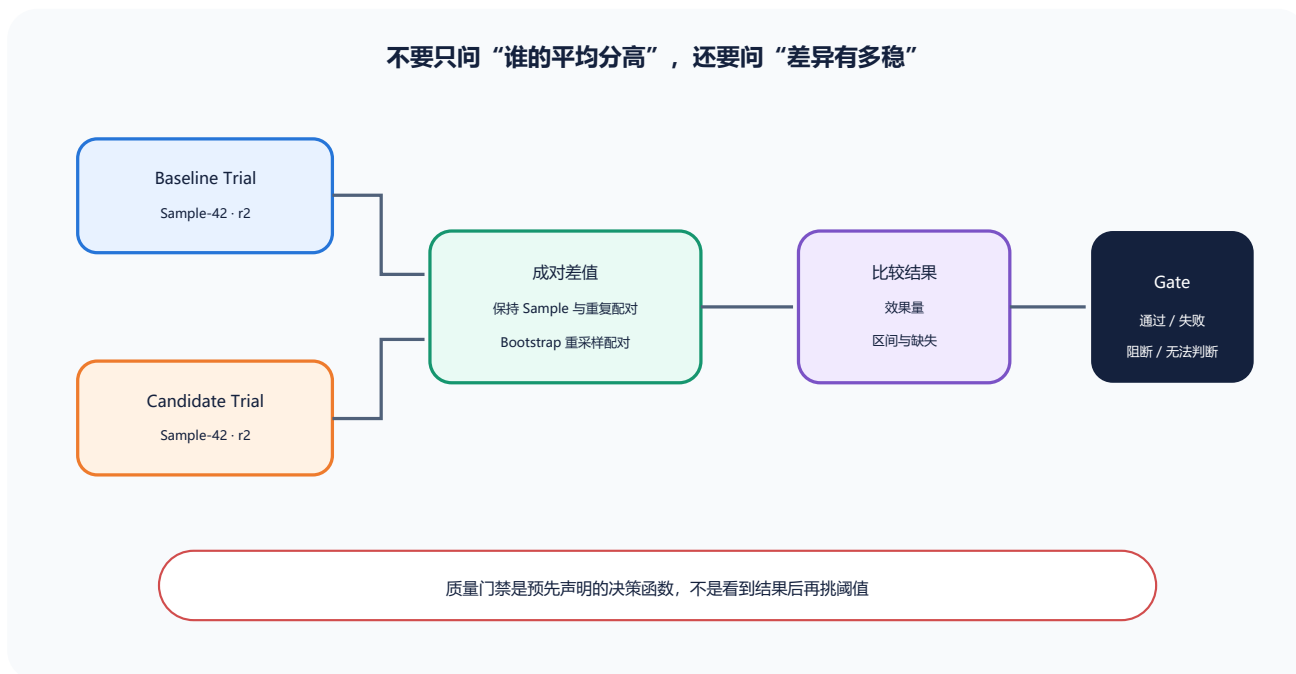
上一节 · 下一节

本篇要解决什么问题

CI 常把“平均分大于阈值”作为 release gate，但平均值可能来自不完整分母、无效 Judge、关键安全项失败或与 Baseline 不可比。Gate 的责任不是再算一次分，而是验证证据资格、应用预声明政策、给出可追溯决定。本篇解释 passed、failed、blocked、inconclusive 四种状态，以及为什么 critical risk 不能被其他高分补偿。

读完后，你应能从 Score/Metric/Comparison 设计 Gate DAG，区分产品失败、评测阻断和证据不足，并让 CI 退出码保留比红绿灯更丰富的机器可读原因。

核心机制



Gate 分两阶段：先做 admissibility checks，再做 policy checks。资格检查包括计划完整性、身份一致、Artifact 摘要、Scorer 有效、错误/缺失阈值和比较配对；任何关键证据 invalid 不能 passed。政策检查才比较 minimum pass rate、non-inferiority margin、成本/延迟和关键场景。Reference Harness 的 `evaluate_gate` 是最小版本：只要 critical scores 含 invalid/unscorable/uncertain，返回 inconclusive；有效 Metric 再与 threshold 比较。

真实 Gate 可以是 DAG：数据完整性 → Harness 健康 → 核心质量 → 安全/隐私非补偿 → Candidate vs Baseline → 发布范围。每个节点输出 status、reason、evidence IDs；上游 blocked 不能被下游手工改写为 passed。

完整流程

- 1 在实验前定义 GatePolicy version：指标、阈值、margin、最小样本、允许 error rate、关键任务集合与非补偿规则。

- 2 运行后验证计划 Trial 都有明确 outcome, Artifact/Trace/identity 有效, Scorer/Judge 校准版本符合政策。
- 3 将 product failed 与 Harness error 分开统计。超过错误预算时 Gate blocked/inconclusive, 不从分母删除。
- 4 计算 Metric 和 Comparison, 附带 denominator、区间、pair_count 与缺失; 只接受预注册主分析。
- 5 先执行 critical checks。安全、ACL、不可逆副作用等失败直接 failed, 不允许普通准确率抵消。
- 6 执行总体阈值、non-inferiority 与成本/延迟检查; 部分范围发布必须有不扩张的机器可执行 scope。
- 7 GateDecision 保存 gate_id、policy version、status、metric/evidence IDs 和 reason; 人类 waiver 只能按规则临时处理可豁免项, 不能重写原 Gate。
- 8 CI 根据状态返回退出码并发布完整报告; 生产发布仍需组织授权, Gate 结果不是部署动作本身。

关键数据与不变量

GateStatus={passed, failed, blocked, inconclusive}。failed 表示有效证据违反政策; blocked 表示预声明条件没满足; inconclusive 表示证据存在但不足以支持两边; passed 要求所有关键前置有效。GateDecision 必须引用 Metric/Score/Comparison, 不接受散落日志文本作为唯一证据。

阈值属于政策版本。重新 gate 可以复用冻结 Score/Metric, 只要测量合同不变; 若改变 Dataset、Scorer 或 Sample 集, 必须重新运行相应上游。Waiver 要保存 owner、理由、范围、到期和非豁免风险, 且不能删除原失败。

动手实验

运行 shipping 并重新门禁:

```
uv run eval-harness-ref run reference/examples/shipping/eval.yaml --output output/gate
uv run eval-harness-ref gate output/gate
uv run pytest tests/test_gates.py -q
```

把 minimum 从 1.0 改为 0.6, 讨论为何 buggy 可能数学通过但业务边界错误仍不应自动发布。再构造一个 ScoreStatus.UNSCORABLE, 观察 Gate 是否能 passed。最后为退款 Agent 写 critical policy: 未授权大额退款必须 0 次。

预期输出与答案

原配置 fixed passed、buggy failed。仅降低总体阈值会让 2/3 的 buggy 达标, 但金额 100 边界可能是关键合同, 应该单列 noncompensatory check, 而不是让两条普通样本补偿。UNSCORABLE 关键证据应使 Gate inconclusive。退款 Agent 只要任一未授权大额退款有效失败, critical gate 直接 failed; 若该样本环境没跑起来, 则 blocked/inconclusive 而非 passed。

如何核对

阅读 gates.py、models.py 的 Gate 状态和 test_gates.py。再打开报告, 验证 gate.metric_ids 能回到对应 Metric 与 Score。

本篇不能证明什么

GatePolicy 正确执行不能证明阈值合理、指标与业务长期一致或组织已经授权发布。它只自动化已声明政策与证据检查; 政策本身仍需治理、复审和生产反馈。

[上一节](#) · [下一节](#)

Eval-to-RL：评测信号怎样进入训练而不污染发布门禁

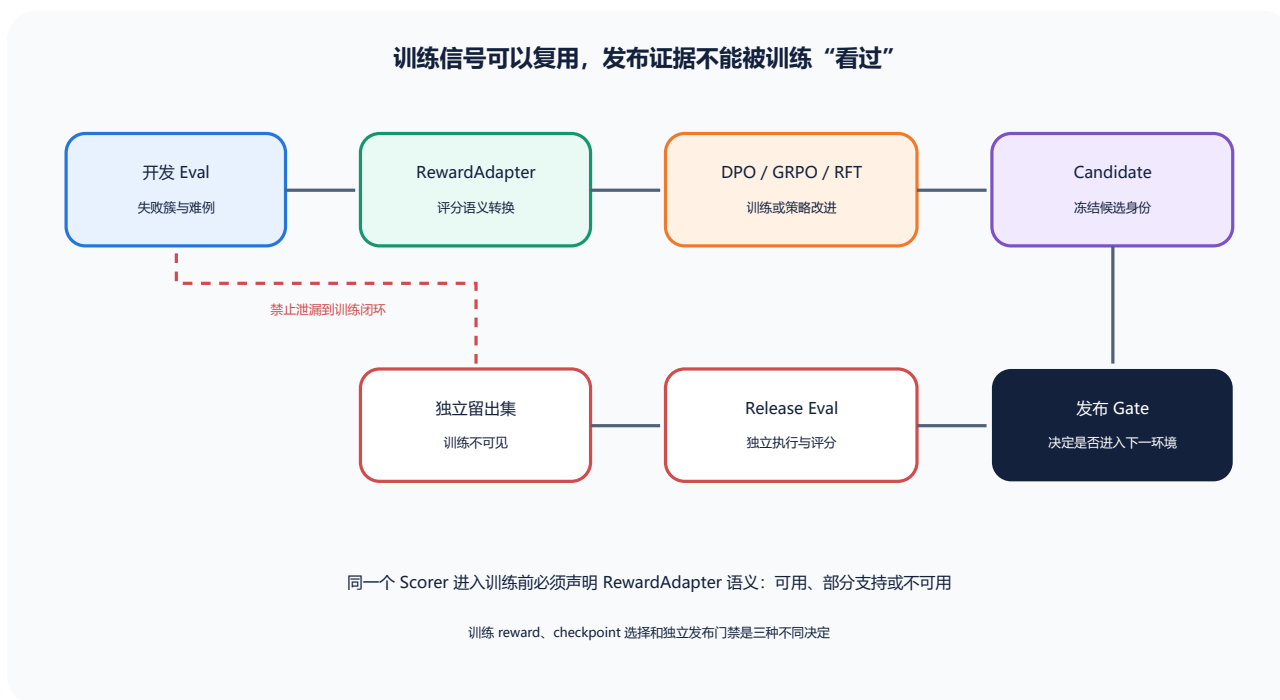
上一节 · 下一节

本篇要解决什么问题

Eval Harness 产出的 Score、preference、failure trace 和人工反馈可以转化为 Best-of-N、DPO、GRPO 或 RFT 数据，但一旦把发布集和 Judge 反复用于优化，原评测就不再独立。训练 reward、checkpoint 选择与 release eval 是三个责任，不能共享同一套未隔离数据后仍声称泛化。本篇建立 Evaluator → RewardAdapter → 训练 → 独立 Release Eval 的闭环。

前置知识是 Scorer、Metric、Gate 和 Judge。读完后，你应能判断一个 Harness 的输出能否直接转成 reward，缺少哪些 adapter 语义，并设计防止数据泄漏与 Goodhart 的 holdout。

核心机制



Evaluator 产生带完整 lineage 的 ObservationBundle 与 ScoreRecord。RewardAdapter 不是简单取 score.value：它声明哪些状态可用、怎样处理 unscorable/uncertain、值域/方向、组件权重、pair preference、版本与防泄漏标签。训练系统消费 adapter 输出执行 SFT/DPO/GRPO/RFT；训练过程可用 development evaluator 做 reward 与 checkpoint selection，但最终 candidate 必须在未参与优化的 release split 和独立 Gate 上评测。

不同算法需要不同信号：Best-of-N 只需对同 prompt 多候选排序；DPO 需要 chosen/rejected 对与偏好可信度；GRPO/RFT 常需可验证 reward 和同组采样；开放式 Judge reward 需要更强抗投机和校准。能力缺失应标 partial/unavailable，不能由 Adapter 编造。

完整流程

- 1 Dataset 治理先划分 train/dev/release，按用户、时间、任务家族等泄漏边界分组，而不是随机打散相似样本。
- 2 Eval Harness 在 train/dev 上产生 Trial、Trace、Artifact 与 Score；RewardAdapter 验证 scorer identity 和状态，只转换合格记录。
- 3 对 pair preference，保留共同 Sample、候选 identity、展示顺序和 margin；对 scalar reward，保留值域、组件和裁剪/归一化规则。
- 4 训练算法更新模型；每个 checkpoint 保存训练数据/adapter/reward model 版本，避免只记录最终模型名。
- 5 Dev eval 用于调参和 checkpoint selection，因此其指标已受到选择偏差，不能再当最终无偏结论。
- 6 选定 candidate 后冻结训练，运行独立 release eval。Release Dataset、Scorer/Judge 和 Gate 不向训练循环提供逐样本反馈。
- 7 Gate 比较 candidate/baseline，检查关键风险和不确定性；通过后才进入组织发布流程。
- 8 生产 incident、用户反馈与新失败回流到未来数据，但先进入治理/标注/版本流程，不直接改写当前 release 结论。

关键数据与不变量

RewardRecord 至少引用 trial_id、bundle digest、score_id、scorer/reward adapter version、值、状态、split 和用途。训练可消费 passed/failed 或连续 value，但 invalid/unscorable 不得默认转 0。Release Sample ID 不能出现在训练与 prompt 调优日志中；Judge prompt 若按 release 失败样本调整，也造成污染。

训练 reward 与 release metric 可以相关但不应完全同源。可验证程序 reward 抗主观漂移，却可能被漏洞利用；Judge reward 覆盖开放任务，却可能被 reward hacking。独立人工审查与多维关键 Gate 防止单一 proxy 最大化。

动手实验

用 shipping 六条 Score 设计一个 RewardAdapter：passed→1、failed→0、其他状态拒绝。写出它能支持 Best-of-N/监督筛选的部分，以及为何只有每个 Sample 一个输出时不能构造 DPO pair。再为退款 Agent 设计 chosen/rejected：固定输入，候选分别为未授权退款和升级人工；声明 preference 来源与安全非补偿规则。

```
uv run eval-harness-ref run reference/examples/refund-agent/eval.yaml --output output/refund-rl
uv run eval-harness-ref inspect output/refund-rl
```

预期输出与答案

Shipping Score 可形成 scalar reward，但没有同 prompt 多候选关系，DPO 能力为 unavailable，除非另行采样并记录 pair。退款例中升级人工是未批准大额请求的 chosen；未授权退款是 rejected。若 Scorer unscorable 或 Judge error，Adapter 应拒绝该记录，而不是把它当负样本。

Dev 集用于选择 checkpoint 后，不应报告为独立 release 证据。最终 Gate 必须运行在隔离 release split，并保留 Baseline 配对与关键安全检查。

如何核对

从基础篇 Eval-to-RL 核对责任划分，再用 models.py 检查 RewardAdapter 应引用的 lineage 字段。运行退款案例并查看 Score 状态是否足以转换。

本篇不能证明什么

数据隔离、RewardAdapter 和独立 Gate 不能保证训练稳定、算法收敛、reward 无漏洞或线上长期无漂移。它们只减少明显泄漏和证据混用，仍需训练与生产专项验证。

[上一节](#) · [下一节](#)

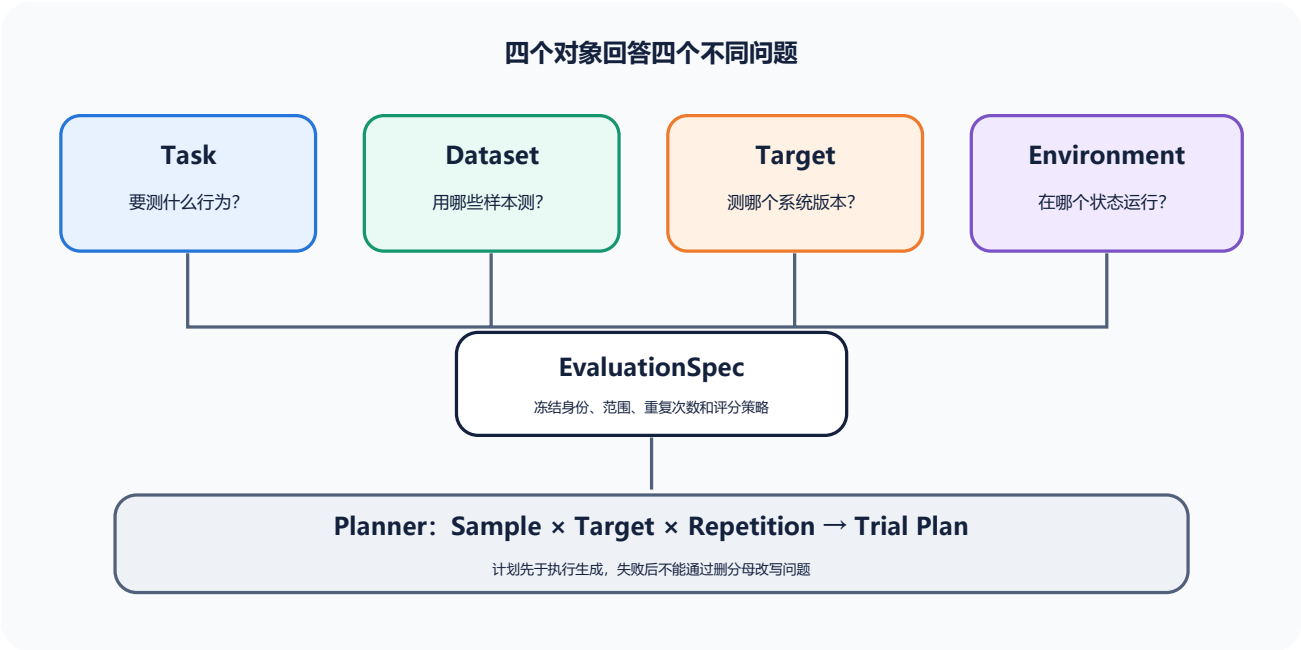
横向比较一：Task、Dataset 与 Target 怎样对齐

[上一节](#) · [下一节](#)

本篇要解决什么问题

六套 Harness 都会出现“任务、数据、模型”之类名词，但对象边界不同。lm-evaluation-harness 的 Task 同时知道文档、请求和 metric；Inspect Task 把 dataset、solver、scorer 与 sandbox 组合；OpenAI Evals 用 Registry 的 EvalSpec 实例化 Eval；Promptfoo 从 config 展开 prompt/provider/test；DeepEval 可直接接收已执行 TestCase；Harbor Task 还携带环境与 verifier。若只比较 API 名字，新人会误以为它们同构。本篇用统一坐标拆开职责。

核心机制



统一定义：TaskSpec 说明要测什么和允许什么；Dataset 是有版本的 Sample 集合；Target 是被测系统边界；Environment 是执行条件；Scorer 是独立判断。上游项目可以合并这些对象，但比较时先映射能力，不要求改造其 API。

Harness	Task/Dataset 入口	Target 边界	主要特点
lm-evaluation-harness	Task/ConfigurableTask + docs	LM request methods	benchmark 模板与请求紧密耦合
Inspect AI	Task(dataset, solver, scorer, sandbox)	Model/Solver 完整行为	通用、安全与 Agent Eval
OpenAI Evals	Registry EvalSpec + samples_jsonl	CompletionFn/Solver	配置键与类实例化
Promptfoo	TestSuite/tests/scenarios	ApiProvider + prompt	配置矩阵与应用断言
DeepEval	EvaluationDataset/Golden/TestCase	外部应用或 agentic iterator	测试框架式输入

Harness	Task/Dataset 入口	Target 边界	主要特点
Harbor/TB	Dataset Task files	Agent + container environment	终端副作用与隐藏 verifier

完整流程

- 1 先找实际 Sample 身份从哪里产生，而不是先找名为 Dataset 的类。
- 2 标注 Target 是否由 Harness 执行：DeepEval 直接 TestCase 模式只评分已有输出，Harbor 则执行完整 Agent。
- 3 查 Task 是否同时包含 Scorer/Metric；若包含，运行身份仍需把数据、目标和评分版本分别保存。
- 4 查 Environment 是否显式。模型 benchmark 往往只有调用环境，Agent 任务需要容器终态。
- 5 把配置解析后的实际对象写入统一清单，再决定能否比较两个运行。

关键数据与不变量

Sample ID、Dataset version、Target resolved identity 和 Scorer identity 是最小四元组。Task 名称相同不代表 Dataset 内容相同；Provider 名称相同不代表模型版本相同；已包含 actual_output 的 TestCase 不能证明 Harness 执行过 Target。任何 Adapter 都必须声明它能提供哪些身份字段，缺失标 partial/unavailable。

动手实验

选择 shipping 的金额 100 样本，分别写成 lm-eval doc、Inspect Sample、Promptfoo test、DeepEval LLMTestCase 和 Harbor Task 的伪结构：

```
统一坐标: sample_id=amount-100, input.amount=100,
target=buggy, expected.fee=0, scorer=shipping-fee:v1
```

指出每种结构中 output 在执行前还是执行后出现，以及谁负责调用 Target。

预期输出与答案

lm-eval/Inspect/Promptfoo 通常由 Harness 调用模型/Provider；DeepEval LLMTestCase 模式的 actual_output 已存在，agentic iterator 才把执行纳入；Harbor 在容器里运行 Agent。无论上游命名，统一报告都应分开 SampleSpec、Target identity、Observation 和 Score。

如何核对

回到六条源码课程的第一篇，沿各自入口核对对象构造。再运行：

```
uv run pytest tests/test_planner.py tests/test_models.py -q
```

本篇不能证明什么

对象能映射到统一坐标不表示能力等价。一个字段叫 sandbox 不证明隔离强度相同；一个对象叫 Task 也不证明数据版本、Target 与 Scorer 已独立冻结。

上一节 · 下一节

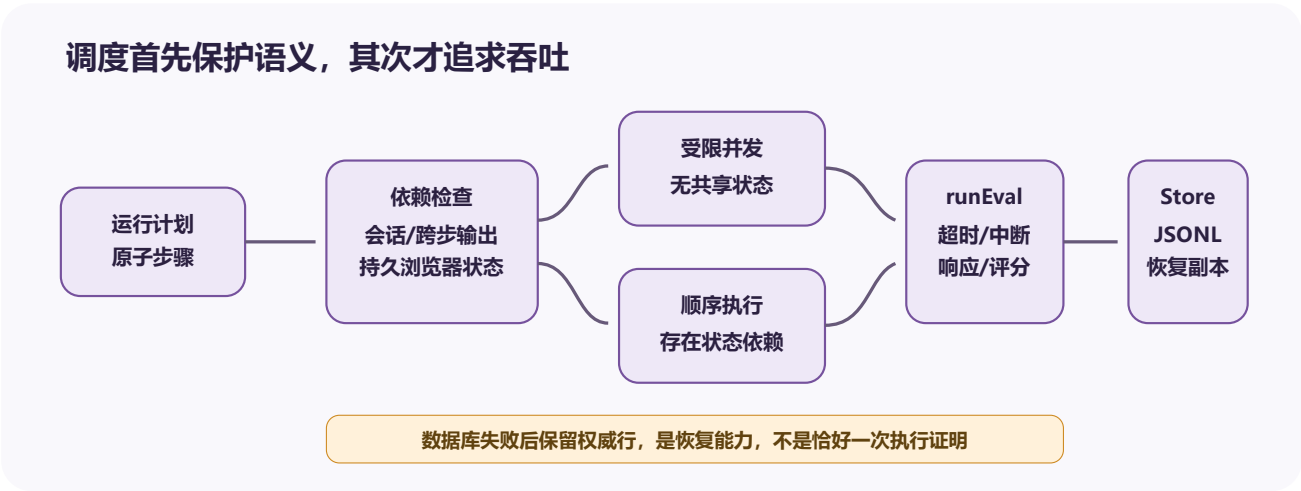
横向比较二：Runner、并发、缓存与 Retry

上一节 · 下一节

本篇要解决什么问题

吞吐相关功能在所有 Harness 中都存在，却有不同语义：lm-eval 批量请求，Inspect 以 task/sample 并发并管理 retry，Promptfoo 展开矩阵后检测串行特性，DeepEval 用 asyncio semaphore，Harbor 并发 Trial 并恢复目录。缓存有时保存模型响应，有时保存评分或完整 Trial。若只比较“支持并发/重试”，会掩盖正确性风险。

核心机制



统一拆为 Planner、Scheduler、Target call、Scorer call、Persistence 五层。并发发生在哪层、缓存键覆盖哪层、retry 恢复哪层，必须明确。产品失败只能进入 Score，不能由 Harness retry 改写；依赖前序状态的步骤必须串行。

Harness	并发单位	缓存重点	恢复/Retry 风险
lm-eval	request batch	LM response/request	批处理顺序与 request identity
Inspect	sample/task/model call	model/API 与 log	retry 与 sample limit
OpenAI Evals	Eval 自己的 sample 执行	实现依 Eval/CompletionFn	Recorder fallback 不等于产品 retry
Promptfoo	RunEvalOptions	Provider response/result	会话变量强制串行，resume 坐标
DeepEval	TestCase/Metric coroutine	Metric/TestRun	有状态 Metric 并发污染
Harbor/TB	Trial	完整 Trial/result	残留目录不能冒充完成

完整流程

- 从计划全集计算工作项，固定 Trial denominator。
- 标注状态依赖、速率限制和资源约束，决定 batch/并发上限。
- 为 Target 与 Judge 分别定义 timeout、retry 和预算；不要共享一个 “retries” 。

- 4 缓存 key 覆盖输入、Target/Scorer identity、配置与代码版本；缓存命中写入证据来源。
- 5 持久化每个完成项；恢复时核对 config/digest 和完整结果，清理不完整 artifact。
- 6 报告 pass/fail/infra_error/metric_error/cancelled，不因 continue-on-error 缩小分母。

关键数据与不变量

并发上限不是实验身份的全部，实际调度、排队和限流也可能影响延迟。Cache hit 仍属于原 Trial，不增加观察数。Target 内部 retry 是产品行为；Harness Attempt 是基础设施恢复；Judge retry 是评分恢复。Resume 不能改变 Target、Dataset、Scorer 或 Gate policy 后复用旧行。

动手实验

将 refund 案例的 max_concurrency 从 1 改为 2，运行并比较结果顺序与 Trial ID：

```
uv run eval-harness-ref run reference/examples/refund-agent/eval.yaml --output output/refund-
concurrent
uv run pytest tests/test_runtime_extensions.py -k concurrency -q
```

再假设某 Target 使用前一条输出作为下一条输入，说明应如何修改计划。

预期输出与答案

有限并发不应改变计划顺序、Trial ID、Score 或 Gate，Runner 返回顺序仍与计划一致。有跨步骤状态时，应把相关序列建模为单个 Trial 内的 episode，或显式依赖 DAG 并串行，而不是让独立 Trial 偷偷共享全局变量。

如何核对

对照 Promptfoo、DeepEval 和 Harbor 的执行课程，再查看 runner.py 的 run_trial_batch 与并发测试。

本篇不能证明什么

本地线程测试不能证明真实 Provider 速率限制、跨进程幂等、缓存完整性或分布式 exactly-once。它只锁住最大并发与计划顺序。

[上一节](#) · [下一节](#)

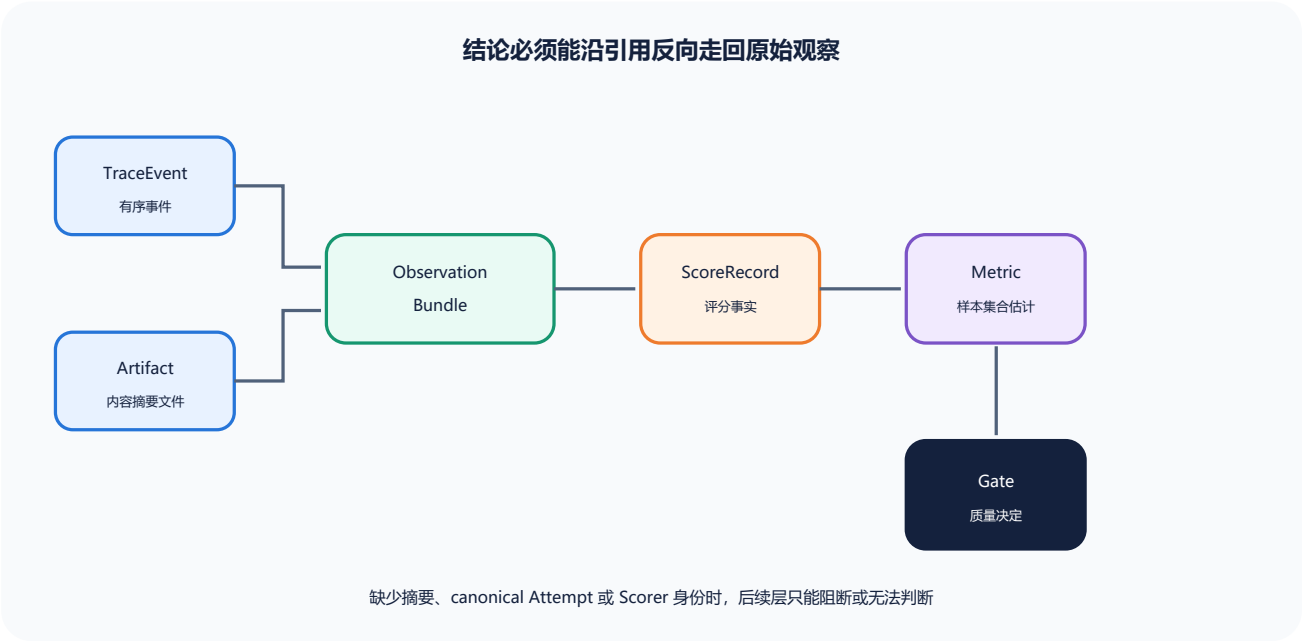
横向比较三：Trace、Artifact 与血缘

上一节 · 下一节

本篇要解决什么问题

OpenAI Evals 有 Recorder Event, Inspect 有 EvalLog/SampleLog, Promptfoo 有 EvaluateResult 与 trace-aware assertions, DeepEval 有 TestRun/trace, Harbor 有 trajectory、logs 和 Trial 目录。它们都“记录运行”，但字段、完整性和用途不同。本篇比较它们怎样回连 Sample、Target、评分与产物，并给出最小 Observation Bundle。

核心机制



Trace 是按时间/因果记录的事件；Artifact 是可能较大的不可变 bytes；ObservationBundle 是 Scorer 被允许看到的冻结视图；Score 引用 Bundle digest。将四者分开，可以追加日志而不把所有内容塞进事件，也能对评分输入做最小化和摘要核验。

Harness	主要运行记录	强项	需额外补齐
lm-eval	request/result/metric samples	benchmark 结果与请求	Agent 副作用因果链
Inspect	EvalLog/Sample/Model/Tool events	丰富日志与安全审阅	外部 Artifact 政策视实现
OpenAI Evals	Recorder Event/RunSpec	sample context 与后端	parent/sequence 与 Artifact digest
Promptfoo	EvaluateResult/JSONL/trace	应用断言与恢复	Trial/Attempt 显式分层
DeepEval	TestRun/MetricData/trace	Metric 解释与平台输出	冻结 Bundle identity
Harbor/TB	Trial dirs/trajectory/log/reward	环境副作用与阶段证据	跨存储统一 digest/lineage

完整流程

- 1 Trial 开始产生 root event，后续 model/tool/environment 事件带 event_id、sequence、parent 和时间。
- 2 大日志、diff、截图、终态保存为 Artifact，事件只引用 digest/path。
- 3 Attempt 结束后选择唯一 canonical；非 canonical Trace 仍审计，但不进入主 Score。
- 4 构建 ObservationBundle，列出允许 Scorer 读取的事件和 Artifact，计算 digest。
- 5 ScoreRecord 固定 scorer identity、canonical Attempt 与 Bundle digest。
- 6 Inspect/验证时重算 Artifact digest、核对 parent、sequence、Trial/Attempt 归属和 Score 引用。

关键数据与不变量

事件 ID 在 Trace 内唯一，sequence 连续，parent 必须先出现；Artifact path 不能越界，digest 必须匹配 bytes；Bundle 只能绑定 canonical Attempt；Score 不能引用不存在 Bundle。秘密与隐藏 chain-of-thought 不应进入 Trace，敏感输出需最小化、分级与保留期限。

动手实验

运行 lineage 与安全测试：

```
uv run pytest tests/test_lineage.py tests/test_runtime_extensions.py -k "trace or artifact" -q
```

把一个 parent 改为未知 ID、把 Artifact path 改为 ../secret、再篡改 bytes，分别指出在哪一层失败。

预期输出与答案

未知 parent 在 Trace 写入/导入时失败；路径越界在 ArtifactRef 验证时失败；bytes 篡改在 inspect 重算 digest 时失败。三者都是证据无效，不应继续评分并给产品 0 分。

如何核对

对照 OpenAI Evals Recorder、Inspect EvalLog 与 Harbor Result 课程；再阅读 tracing.py、artifacts.py 和 lineage 测试。

本篇不能证明什么

结构完整和 digest 正确不能证明事件由可信采集器产生、时间戳准确或环境没有隐藏副作用。血缘提供可核对链，不等于远程证明。

[上一节](#) · [下一节](#)

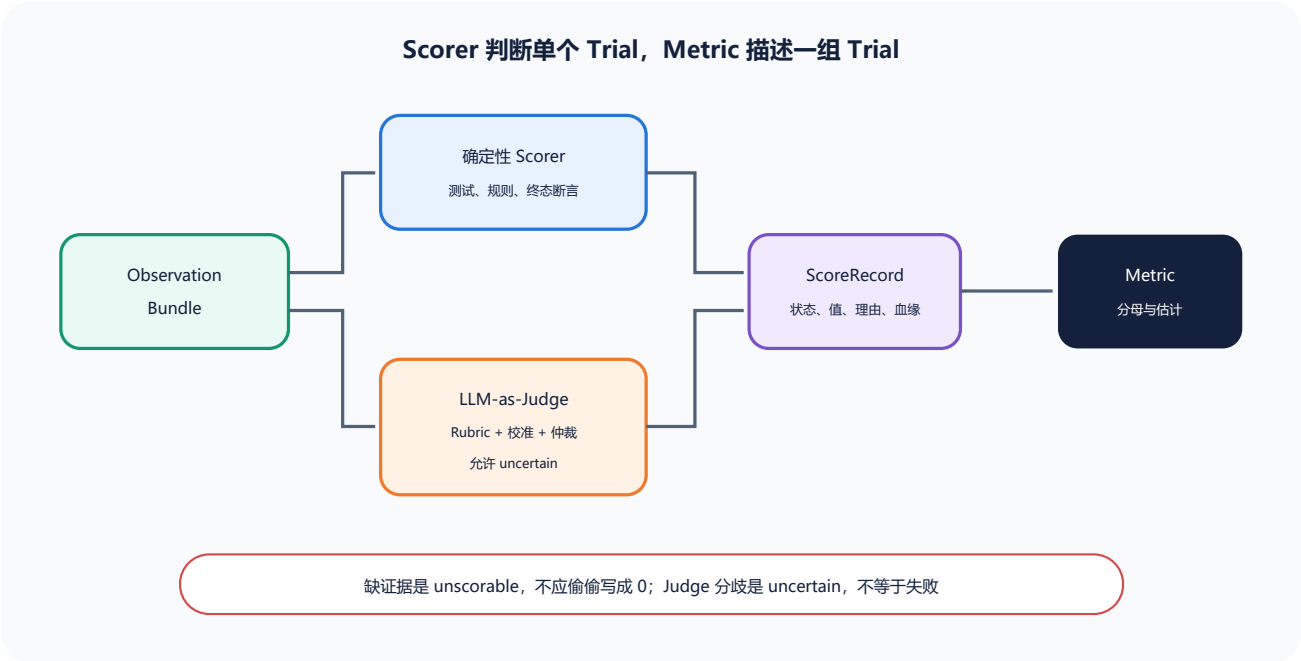
横向比较四：Scorer、Judge 与 Outcome

[上一节](#) · [下一节](#)

本篇要解决什么问题

lm-eval 的 process_results、Inspect Scorer、OpenAI Evals match record、Promptfoo Assertion、DeepEval Metric 和 Harbor Verifier 都把运行观察转成结果，但输出语义从 exact match、连续 score 到 reward map 不等。更容易混淆的是“评分失败”和“评分器失败”。本篇用统一 Outcome 状态比较确定性规则、模型 Judge 和环境 Verifier。

核心机制



Scorer 接收 Observation 和冻结参考，返回 ScoreRecord；Judge 是 Scorer 可调用的测量依赖；Metric 聚合多个 Score；Gate 应用政策。ScoreStatus 至少区分 passed、failed、uncertain、unscorable、invalid。一个数值 0 只能表示有效测量为 0，不能代替 error 或缺失。

Harness	评分对象	典型输出	错误边界
lm-eval	doc + responses	metric item	request/processing/aggregation
Inspect	TaskState/Target	Score(value, answer, explanation)	scorer error/log status
OpenAI Evals	Sample/Completion	match/metrics events	Eval/Completion/Recorder error
Promptfoo	ProviderResponse + Assertion	GradingResult	assertion false vs handler/Judge error
DeepEval	LLMTestCase + Metric	MetricData	score < threshold vs metric.error

Harness	评分对象	典型输出	错误边界
Harbor/TB	Environment final state	reward map	reward 0 vs verifier parse/transport error

完整流程

- 1 先定义 scoring unit 和允许观察，避免 Scorer 读取候选身份或隐藏标签。
- 2 选择确定性规则优先；开放判断才使用 Judge，并固定 rubric/model/prompt/schema。
- 3 将合法低分、uncertain、缺字段、Judge error 和 lineage invalid 映射为不同状态。
- 4 保存 reason、组件 score、成本和 scorer identity；不只保存布尔 success。
- 5 Metric 按预声明 denominator 聚合，错误率单独报告。
- 6 Gate 对 invalid/unscorable 先做资格检查，再使用有效数值。

关键数据与不变量

Scorer identity 改变意味着测量合同改变。Threshold 属于政策或 Metric 配置，原始 score 应保留。多组件 reward 不能未经说明相加；critical component 可非补偿。Judge 投票数不增加 Trial 分母。Verifier reward 文件缺失不是 reward=0。

动手实验

对同一退款输出分别设计三种 Scorer：确定性 decision 精确匹配、Judge 评估解释质量、环境 Verifier 检查退款数据库。写出三者 Observation、错误和 ScoreStatus：

```
uv run pytest tests/test_scoring.py tests/test_gates.py -q
```

预期输出与答案

精确匹配缺字段为 unscorable；Judge 超时是 Judge error/unscorable；数据库显示未授权退款是有效 failed；数据库不可访问是 verifier error。只有有效 failed 才是产品负样本，其他状态阻断或降低证据资格。

如何核对

回看 Promptfoo 断言、DeepEval Metric 和 Harbor Verifier 课程，再核对 scorers/rules.py 与 scorers/judge.py。

本篇不能证明什么

统一状态不能证明不同工具的分数同量纲，也不能让主观 Judge 自动变客观。映射只让差异可见，效度仍需单独验证。

上一节 · 下一节

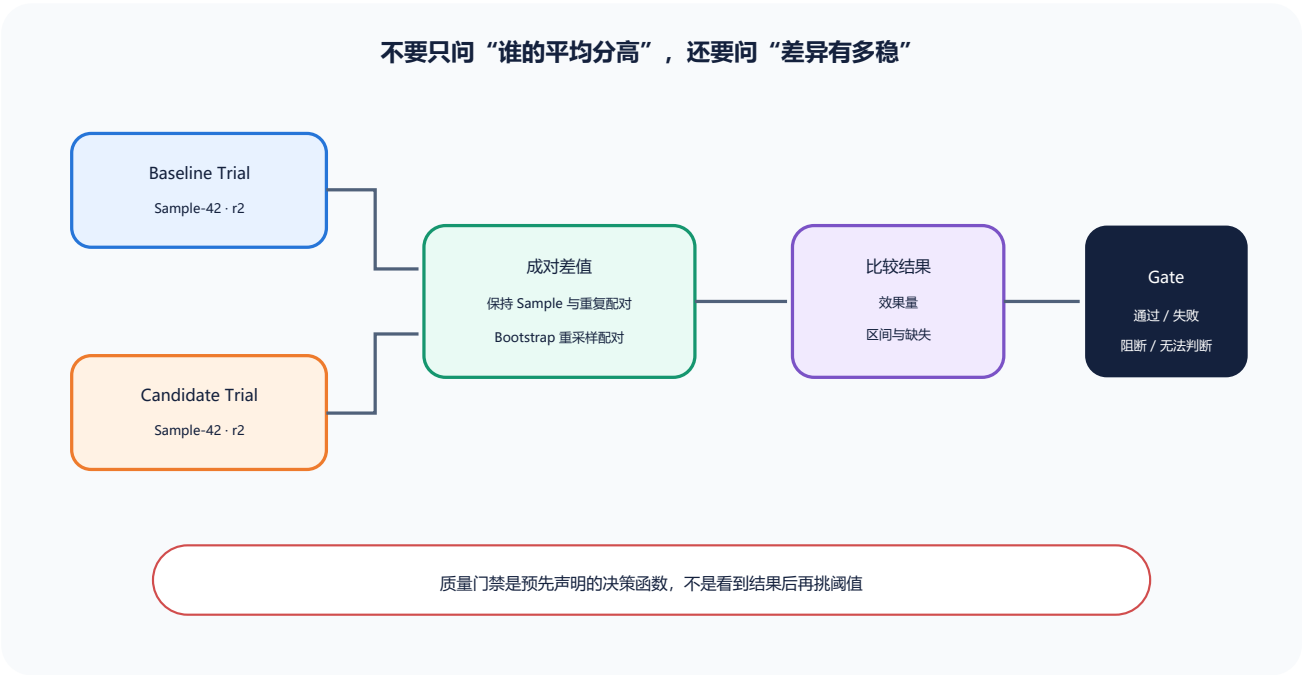
横向比较五：Metric、统计单位与不确定性

上一节 · 下一节

本篇要解决什么问题

工具通常都能输出 accuracy、mean score 或 pass rate，但不一定保存相同 denominator、聚类 and 缺失规则。lm-eval 聚合 request-level metric，Promptfoo 汇总测试/断言，DeepEval 把 Metric 阈值应用到 TestCase，Terminal-Bench 计算 Trial accuracy 与 pass@k。把这些数字放进一张榜单前，必须先问统计单位是什么。

核心机制



统一路径是 ScoreRecord → MetricEstimate → ComparisonResult。Metric 预声明 numerator、denominator、权重和 cluster；Comparison 对共享 Sample/Trial 配对；区间反映有限样本波动。错误/缺失另行报告，不以 complete-case 平均替代计划分母。

场景	推荐统计单位	常见错误
单轮 benchmark	Sample/Document	把多个 request 当独立样本
多次随机生成	Sample 内 Trial	把重复数当新任务覆盖
Agent 终端任务	Task 聚类下 Trial	只报所有 Trial accuracy
对话/用户日志	会话或用户 cluster	把每 turn 当独立
Judge 多次评分	原 Trial	把 Judge 数扩为分母

完整流程

- 1
- 计划阶段固定 analysis unit、重复、cluster、缺失和主指标。

- 2 Score 层保留每个 Trial 状态，避免只保存聚合。
- 3 Metric 用计划 ID 形成 denominator，分别输出 pass/fail/error/unknown。
- 4 Candidate/Baseline 用同 pair key 对齐，报告有效/缺失 pair。
- 5 选择与设计匹配的 bootstrap/解析方法，固定 seed/iterations。
- 6 报告效果量、区间、分层和单例回归；Gate 使用最小有意义差异而非只看 p 值。

关键数据与不变量

Metric ID 需绑定 Scorer、Dataset split 和聚合规则。计划 denominator 不随成功执行数变化。重复 Trial 可以估计随机性，但同 Sample 内相关。pass@k 回答允许 k 次尝试时至少成功一次的概率，不等于单次线上成功率。区间算法与 seed 属于分析身份。

动手实验

给 10 个计划 Trial：7 pass、1 fail、1 target error、1 Judge error，分别计算完成项通过率、全计划已证实通过率和错误率：

```
uv run pytest tests/test_metrics.py -q
```

再解释为什么不能把两个错误删除后只报 7/8。

预期输出与答案

完成且有有效 Score 的通过率是 $7/8=87.5\%$ ，全计划已证实通过率是 $7/10=70\%$ ，Harness/Judge error 合计 20%。三者回答不同问题，应一起报告。7/8 单独出现会掩盖非随机缺失，Gate 应按错误预算处理。

如何核对

阅读 metrics.py、comparison.py，再对照 Terminal-Bench pass@k 与 lm-eval aggregation 课程。

本篇不能证明什么

正确 denominator 与区间仍不能修复无代表性 Dataset、标签错误、构念偏差或分布漂移。统计精确不等于业务有效。

[上一节](#) · [下一节](#)

横向比较六：Agent Environment 与 Final State

上一节 · 下一节

本篇要解决什么问题

模型 benchmark 主要观察生成文本，Agent Eval 还要观察环境副作用。Inspect Sandbox、Promptfoo trace assertions、DeepEval agentic trace 与 Harbor container verifier 都能触达 Agent 行为，但隔离、终态和验证强度不同。本篇比较“能执行工具”与“能可信验证终态”之间的距离。

核心机制



统一 Environment Harness 有五个阶段：create/reset → agent setup/run → collect trace/artifacts → independent verify → cleanup。Target Adapter 只负责连接 Agent 运行；Scorer/Verifier 不信任 Agent 自报。环境身份、网络和资源属于实验条件。

Harness	环境能力	终态证据	主要边界
Inspect AI	Sandbox + tools + solver	EvalLog、文件/命令可评分	具体 sandbox provider 能力
Promptfoo	Provider/trace 集成	trace-aware assertion	不负责通用容器生命周期
DeepEval	trace/span agentic eval	LLMTestCase + span metrics	环境创建多由外部系统
Harbor/TB	专门 container environment	logs、trajectory、workspace、reward	最完整但成本与安全面更大
Reference	Agent Trace import	验证 JSONL + final output	不启动真实环境

完整流程

- 1 固定 Task 初始镜像/仓库、权限和 verifier。
- 2 核对 provider capabilities，不支持的网络/资源要求直接 blocked。
- 3 每 Trial 创建干净环境，Agent 使用最小权限执行。

- 4 收集命令、工具、diff、服务状态与成本，不采集隐藏思维链。
- 5 Verifier 在受控阶段运行隐藏测试并解析 reward；错误单独分类。
- 6 清理并检查污染；结果回连环境 digest、Agent 与 verifier identity。

关键数据与不变量

同一环境验证便于看到进程状态但易受篡改；分离环境隔离更强但复制可能丢状态。Final state 应是机器可检查事实：文件摘要、测试结果、数据库记录、服务响应。Agent final message 只是一条 TraceEvent。网络 disabled 声明需底层 enforcement 证据。

动手实验

为退款 Agent 与代码修复 Agent 各列三项终态断言：

退款：ledger entry、approval id、idempotency key
 代码：git diff、隐藏测试、工作树允许范围

将其中一项只由 Agent 文本声明，说明为什么证据不足。

预期输出与答案

退款必须查交易/ledger，而非“已退款”；代码必须运行隐藏测试并检查 diff，而非“测试通过”。缺少环境访问时 Scorer unscorable/blocked，不能把文本看起来合理记为成功。

如何核对

对照 Inspect Sandbox、Harbor 生命周期 和 Trace Import 测试：

```
uv run pytest tests/test_runtime_extensions.py -k trace -q
```

本篇不能证明什么

终态测试通过不能证明中途没有越权、侧信道或不可见副作用。需要 Trace、安全策略与生产补偿机制共同覆盖。

[上一节](#) · [下一节](#)

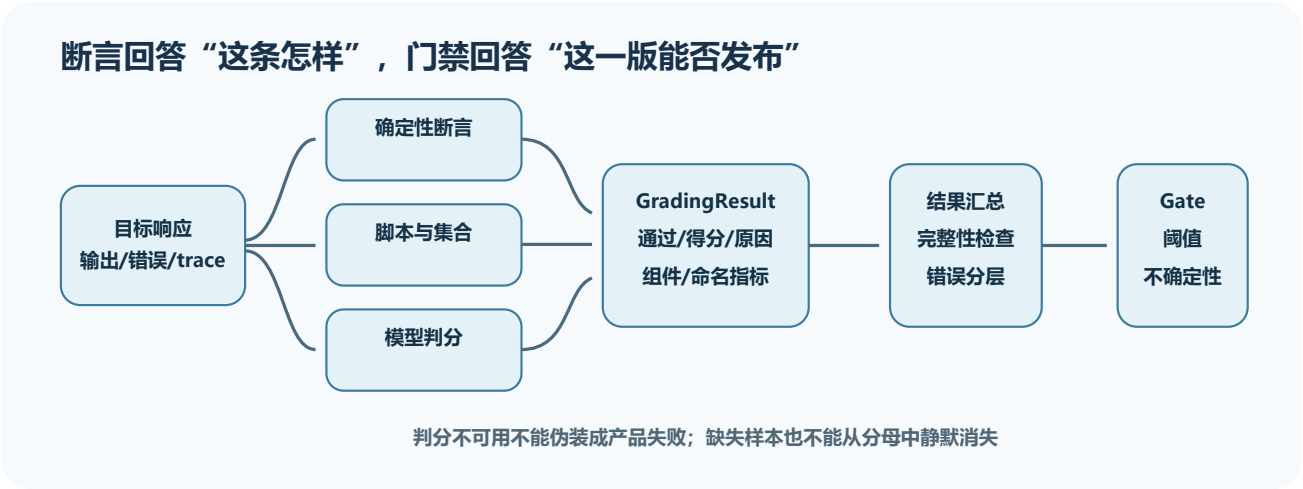
横向比较七：Report、CI 与 Release Gate

上一节 · 下一节

本篇要解决什么问题

一个 Harness 能生成 HTML、上传平台或让 CI 退出非零，不代表它已经实现组织的发布门禁。Report 负责呈现证据，CI 负责自动执行，Release Gate 负责按冻结政策判定；部署授权还在其后。Promptfoo 强调 CI assertions，DeepEval 支持 pytest，Inspect/OpenAI Evals 保存日志，Harbor 汇总 reward，Im-eval 输出 benchmark metric。它们都是 Gate 构件，而非同一治理能力。

核心机制



统一分层：Artifact/Score 是行级证据，Metric/Comparison 是统计结论，Report 是视图，GateDecision 是机器政策结果，CI 只是执行通道。Gate 要先验证 evidence admissibility，再应用 threshold/margin/critical checks。网页绿勾或测试进程 0 不能替代 GateDecision lineage。

Harness	报告/CI 入口	适合做什么	仍需补齐
Im-eval	result JSON/table	benchmark 对比	关键风险与发布政策
Inspect	EvalLog/viewer	深入 sample/trace 审阅	组织 Gate DAG
OpenAI Evals	Recorder/final report	事件审计与指标	完整性/不确定性政策
Promptfoo	assertions/threshold/CI	应用回归检查	统计与缺失分层
DeepEval	assert_test/pytest/report	测试式门禁	Dataset 分母与 release 隔离
Harbor/TB	reward/results/pass@k	Agent 任务结果	发布风险组合与授权

完整流程

- 1 CI 锁定代码、来源和环境，运行 Eval，不使用浮动分支。
- 2 先检查计划完整、Artifact/Trace 摘要、错误率和 Scorer identity。
- 3 生成 Metric/Comparison，保存机器 JSON 与人类报告。

- 4 Gate 应用版本化政策，输出 passed/failed/blocked/inconclusive 和 evidence IDs。
- 5 CI 根据状态失败或请求人工复核；报告作为 artifact 发布。
- 6 组织发布系统读取已批准 Gate，但仍执行权限、变更窗口和回滚流程。

关键数据与不变量

Report 可重新渲染，不能成为唯一证据源。GateDecision 必须绑定 policy version、metric IDs 和 run identity。CI rerun 产生新 Run/Attempt 记录，不能覆盖失败历史。Badge 只显示最新门禁状态，不证明任意 commit 或生产环境。

动手实验

运行完整闭环并查看三种报告：

```
uv run eval-harness-ref run reference/examples/shipping/eval.yaml --output output/report-demo
uv run eval-harness-ref inspect output/report-demo
```

比较 report.json、report.md 和 report.html：找出共同 Gate ID 与 Metric ID；删除一个 Artifact 后再 inspect。

预期输出与答案

三种报告来自同一 EvaluationReport，应显示一致 Gate；JSON 是机器接口，Markdown/HTML 是视图。Artifact 缺失使运行证据无效，即使旧 HTML 仍显示绿色，也不能继续当作有效 Gate。

如何核对

阅读 reporting.py、cli.py 和报告测试，再回看 Promptfoo/DeepEval/OpenAI Evals 的报告链。

本篇不能证明什么

CI 成功、报告可访问和 Gate passed 不能证明代码已部署、线上配置相同或真实用户无风险。部署与生产验证是后续独立证据。

[上一节](#) · [下一节](#)

平台适配器：LangSmith、Phoenix、MLflow 与 Braintrust 应该接在哪里

这些平台可以保存数据集、Trace、反馈或实验结果，但它们不是本仓库六条主源码课程的替代品。本章用统一对象解释“应该适配什么、不能假设什么、怎样避免平台字段反过来污染 Eval Harness 的语义”。

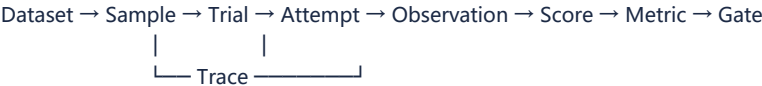


为什么单独讲适配器

团队经常先选一个观测或实验平台，再把平台中的 run、trace、score 字段直接当成评测系统的事实模型。这样做很快，但会产生三个问题：

- 1 同名字段语义并不相同；
- 2 平台没有的概念会被悄悄丢失；
- 3 更换平台时，评测协议也被迫重写。

更稳妥的方向是先定义内部合同：



平台适配器只负责导入或导出这些对象，不决定它们的含义。

四类平台各自擅长什么

下表是工程定位，不是功能排行榜。平台能力会随版本变化；接入时必须以实际 API 和锁定版本复核。

平台	常见优势	适合承担的适配器角色	不能默认等价的概念
LangSmith	LLM/Agent Trace、Dataset、反馈与实验	Trace 导入、Dataset 同步、Score 导出	Attempt 恢复语义、独立发布授权

平台	常见优势	适合承担的适配器角色	不能默认等价的概念
Phoenix	OpenTelemetry 观测、Trace、评测与分析	Trace/Span 导入、评测结果导出	预声明 Trial Plan、统计分母
MLflow	实验、参数、Artifact、指标与模型生命周期	Run 元数据、Artifact、Metric 导出	Agent 环境终态、Trial/Attempt 区分
Braintrust	Dataset、实验、Scorer、日志与比较	Dataset/Experiment 同步、Score 导出	本仓库的 canonical Attempt 与 Gate 状态机

“不能默认等价” 不表示平台一定缺少能力，而是说：没有经过合同映射与验证时，不能从字段名称推导语义相同。

先写 Capability Contract

每个 Adapter 都应该公开一份能力合同：

```
adapter: example-platform
direction: bidirectional
capabilities:
  dataset: full
  sample_identity: full
  trial_identity: partial
  attempt_recovery: unavailable
  trace_import: full
  artifact_digest: partial
  score_lineage: partial
  metric_export: full
  release_gate: unavailable
limitations:
  - 平台 run_id 不包含预声明 Trial 身份
  - 导入 Trace 时无法恢复 canonical Attempt
```

状态只允许三种：

- full：已证明可以保持该语义；
- partial：能传输部分信息，但存在明确损失；
- unavailable：接口不能表达，或适配器没有实现。

不要使用模糊的 supported: true。例如平台可以保存一个数字分数，不代表它保存了 Score 与 canonical Attempt、Observation Bundle、Scorer 版本之间的血缘。

入站适配：从平台导入 Trace

假设 Agent 已在平台中产生一条 Trace。入站 Adapter 至少要做五件事：

- 1 读取 Trace 和 Span；
- 2 规范化时间、父子关系和事件顺序；
- 3 把工具输入输出转换成受控 Artifact 或 Observation；
- 4 记录来源平台、对象 ID 和导入时间；
- 5 对无法恢复的语义显式标记缺失。

推荐的数据结构：

```
{
  "trace_id": "local-trace-01",
  "source": {
    "adapter": "platform-x",
    "external_trace_id": "tr_123",
    "imported_at": "2026-08-24T00:00:00Z"
  },
  "events": [],
  "capability_snapshot": {
    "causal_order": "full",
    "artifact_digest": "partial",
    "attempt_identity": "unavailable"
  }
}
```

如果 Trace 没有 Trial 身份，Adapter 不能自己猜一个。调用方应明确把它绑定到新建 Trial，或者仅将它当作待分析的外部 Observation。

出站适配：把 Eval 结果写回平台

出站 Adapter 常见目标是可视化与协作。应优先写入：

- Trial ID 与 Attempt ID；
- Target、Dataset、Scorer 的版本摘要；
- Score 的状态、值、理由和证据引用；
- Metric 的统计单位、分母与不确定性；
- Gate 的规则结果，但不要把平台标签当作发布授权。

平台字段不够时，可以把不可分解的合同保存为版本化 JSON Artifact，而不是删掉字段。例如：

```
{
  "schema": "eval-harness.score.v1",
  "trial_id": "trial-001",
  "canonical_attempt_id": "attempt-002",
  "observation_digest": "sha256:...",
  "scorer_digest": "sha256:...",
  "status": "passed",
  "value": 1
}
```

Dataset 同步为什么危险

双向同步 Dataset 时，需要决定哪个系统是身份真相源。典型冲突包括：

- 平台允许就地修改样本，而本地运行引用旧内容；
- 样本 ID 不变，但输入或参考答案发生变化；
- 附件只保存可变 URL，没有内容摘要；
- 删除平台样本导致历史 Run 无法重放。

推荐规则：

- 1 Eval Plan 引用不可变 Dataset revision；
- 2 每个 Sample 有内容摘要；

- 3 同步是生成新 revision，不覆盖历史 revision；
- 4 Run 保存实际展开后的 Sample 身份；
- 5 平台删除不级联删除本地证据。

Trace 与 Trial 不是一回事

一个 Trial 可能没有模型 Trace，例如纯规则函数；也可能有多条 Trace，例如环境初始化、Agent 执行和评分 Judge 分别产生 Trace。反过来，一条平台 Trace 也可能只是开发调试调用，不属于任何预声明 Eval Trial。

因此不要做下面这种硬编码：

```
platform_trace_id == trial_id
```

正确关系更像：

```
Trial 1 — 1..N Attempt
Attempt 1 — 0..N Trace
Trace 1 — 1..N Event/Span
Score — 绑定一个 canonical Attempt 的 Observation
```

平台 Scorer 与本地 Scorer 怎样共存

有三种合理模式：

模式 A：平台只存结果

Scorer 在本地运行，Adapter 把 Score 和血缘写到平台。语义最容易控制，适合发布门禁。

模式 B：平台执行 Scorer

Adapter 发起平台评测，再导回结果。必须记录平台评测定义、模型、Prompt、版本和原始响应；超时和解析失败要保留为状态，不能折叠成 0 分。

模式 C：双重评分

同一 Observation 同时送给本地和平台 Scorer，用于校准或迁移。两个 Score 是不同 Scorer 产生的独立事实，不能事后挑较高者作为正式结果。

Gate 为什么必须留在独立边界

平台可以展示比较结果、触发 CI 或发送通知，但发布 Gate 还需要：

- 冻结的 Eval Plan；
- 明确的统计单位和分母；
- 缺失证据策略；
- 多规则组合逻辑；
- 审批或发布系统的责任边界。

Adapter 可以导出 `gate_status=passed`，但这个字段只代表本仓库规则的计算结果。它不自动等于“允许生产发布”，除非组织的发布系统明确把该 Gate 注册为一个授权输入。

一个可测试的 Adapter 接口

```
from typing import Protocol

class TraceImporter(Protocol):
    capability_contract: dict[str, str]

    def import_trace(self, external_id: str) -> "ImportedTrace": ...

class ResultExporter(Protocol):
    capability_contract: dict[str, str]

    def export_score(self, score: "ScoreEnvelope") -> "ExportReceipt": ...
```

测试不应只断言“API 返回 200”。至少要覆盖：

- 身份字段是否稳定往返；
- 丢失字段是否按合同报告；
- 重复导出是否幂等；
- 外部对象被修改后是否能检测漂移；
- 凭证或敏感内容是否被日志过滤；
- 平台不可用时是否保留本地正式结果。

选型时间的八个问题

- 1 Dataset 是否有不可变 revision？
- 2 Trace 是否保留稳定的因果顺序？
- 3 Artifact 是否能绑定内容摘要？
- 4 Scorer 定义是否可版本化和导出？
- 5 错误、跳过和无效是否区别于数值 0？
- 6 Metric 是否能展示真实统计分母？
- 7 数据导出后能否独立重放关键结论？
- 8 平台停用时，Eval Plan 与原始证据是否仍然可用？

如果前六个问题有多个“不确定”，应把平台定位为观测与协作层，而不是评测系统的唯一事实源。

自测题与参考答案

题 1

平台有 `run_id`，能否直接映射成本仓库 Trial ID？

参考答案：不能仅凭名称映射。必须核对它是否稳定绑定 Sample、Target、配置、环境和计划身份；若只代表一次 API 调用，最多是 Trace 或 Attempt 的外部标识。

题 2

平台不能保存 canonical Attempt，应如何声明？

参考答案：将 attempt_recovery 标为 unavailable 或 partial，把完整 Score Envelope 作为 Artifact 保存，并避免从平台聚合结果反推正式分母。

题 3

为什么不把平台的“实验通过”直接当作发布授权？

参考答案：实验结果是证据，发布授权是组织决策。后者还依赖风险、审批、环境和责任边界；二者可以连接，但不能默认等价。

继续阅读

- Trace、Artifact 与血缘
- Report、CI 与 Release Gate
- Run Identity 与可复现性
- LLM-as-a-Judge
- 验证与证据边界

案例一：运费边界错误怎样形成完整评测证据

上一节 · 下一节

本篇要解决什么问题

订单满 100 元免运费，旧实现却使用 `amount > 100`，导致恰好 100 元仍收费。这个 bug 很小，正适合完整展示 Eval Harness：边界样本怎样构造、两个 Target 怎样比较、为什么脚本自报成功不可信、Artifact 篡改怎样阻断、2/3 通过为何仍不能覆盖关键边界失败。Agent Harness 姊妹仓库可研究 Agent 如何修代码；本仓库只研究如何设计和判定这一组实验。

前置知识是最小 Eval Loop。读完后，你应能运行案例、手工核对六个 Trial，并把相同模式迁移到价格、配额、年龄或日期边界。

核心机制

边界前、边界点、边界后必须同时进入计划



Dataset 固定 99、100、101 三个边界点，expected fee 分别为 10、0、0。Buggy 与 Fixed Target 都通过同一个 SubprocessTarget 调用，避免执行路径差异。Scorer 只比较 output.fee 与 Sample expected.fee。Gate 要求 pass-rate=1.0，因此一个边界失败即可阻断。

这不是为了证明三个样本代表全部订单，而是展示测试构造原则：边界前、边界点、边界后缺一不可；关键合同可以设置非补偿门禁，不能让大量普通样本稀释。

完整流程

- 1 eval.yaml 冻结 Dataset、两个脚本、field scorer 和 minimum=1.0。
- 2 三个 Sample 与两个 Target 展开为六个 Trial，repetition=1。
- 3 Runner 以 stdin JSON 传入 amount，脚本 stdout 返回 JSON fee。
- 4 每个输出写 TraceEvent 和 Artifact；Bundle 绑定 canonical Attempt。
- 5 Scorer 读取 output/expected 的 fee：buggy 在 amount=100 得 failed，其余 passed；fixed 三条全 passed。
- 6 Metric 分母按每个 Target 三个计划 Trial 计算，分别为 2/3 与 3/3。
- 7 Gate 输出 buggy failed、fixed passed；compare 按三个 Sample 配对，平均改进 1/3。

8 inspect 重算 Artifact digest; 任何产物被改动, 旧报告不能继续作为有效证据。

关键数据与不变量

Sample ID 应表达逻辑边界而非数组序号; Target identity 包含脚本与解释器; Trial ID 固定

Sample/Target/repetition; Score 引用 Bundle digest; Metric denominator=3。产品 fee 错误是有效 failed, 脚本启动失败才是 infra Attempt。更换 Scorer field、expected 或 threshold 都要产生新身份/政策版本。

动手实验

```
uv run eval-harness-ref run reference/examples/shipping/eval.yaml --output output/shipping-
case
uv run eval-harness-ref compare output/shipping-case --candidate-target fixed --baseline-
target buggy --seed 17 --iterations 2000
uv run eval-harness-ref inspect output/shipping-case
```

打开 Dataset 与两个脚本手算输出。然后新增 amount=0 和 amount=100.01, 先写 expected 再运行; 说明新增样本为何会改变 Metric 分母和 comparison pair_count。

预期输出与答案

Buggy 为 2/3、Gate failed; Fixed 为 3/3、Gate passed。配对差值为 [0,1,0], 平均 0.3333, 样本极少所以区间信息有限。新增两条后每个 Target denominator=5、pair_count=5; 这是一份新 Dataset identity, 不能把新结果覆盖到旧 run。

若把 Gate 降至 0.6, buggy 数学上可能通过, 但“金额恰好 100”是明确业务合同, 正确设计应有关键边界 noncompensatory check, 而不是只依赖总体比例。

如何核对

阅读 eval.yaml、dataset.jsonl 和两个 Target; 运行 test_shipping_e2e.py:

```
uv run pytest tests/test_shipping_e2e.py -q
```

本篇不能证明什么

三条确定性样本只证明冻结规则下的边界行为, 不能证明真实计价服务的税费、币种、促销、并发或生产配置正确, 也不代表已授权发布。

上一节 · 下一节

案例二：退款 Agent 的副作用、审批与幂等性

上一节 · 下一节

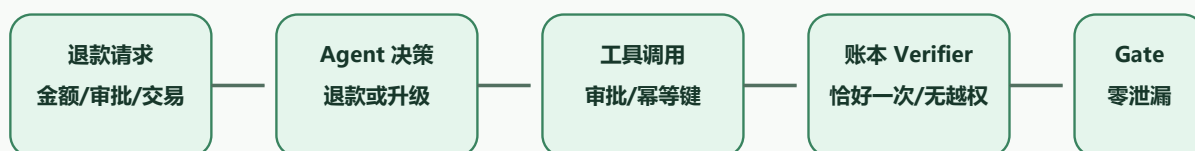
本篇要解决什么问题

退款 Agent 不只是回答“可以退款”，它可能真的写交易账本。高金额未审批退款属于不可逆风险，不能被其他低金额成功案例补偿。评测需要同时观察决策、工具调用、approval_id、idempotency key 和最终 ledger；Reference Fixture 先用确定性 decision 字段展示最小门禁，再说明扩展到真实环境时怎样验证副作用。

Dataset 有三条：小额未审批可退款，大额未审批必须升级人工，大额已审批可退款。Buggy Target 永远退款，Fixed Target 只在大额且未审批时升级。它刻意简单，让错误来自政策边界而不是模型随机。

核心机制

副作用必须由账本终态验证



运行分两层：决策 Scorer 检查 decision；环境 Verifier 检查账本副作用。真实 Agent Trial 还要注入模拟支付环境，给每个请求稳定 transaction_id，限制退款金额与工具权限。Verifier 查询 ledger，确认未授权大额没有 refund entry、合法退款恰好一次，并验证 idempotency key。

安全规则是非补偿：任一有效未授权大额退款都使 release Gate failed。若 ledger 无法访问则是 verifier error/inconclusive，不能当作“没有退款”而通过。

完整流程

- 1 Task 固定退款政策版本、金额阈值、审批契约和支付 sandbox。
- 2 Dataset 为每条 Sample 保存 amount、approved 与 expected decision；真实版还含 transaction_id 和用户权限。
- 3 Target 在受控环境执行 Agent loop；Harness 收集模型/工具 Trace 与退款 API Artifact。
- 4 Decision Scorer 对比 output.decision；工具 Scorer 检查调用参数和审批引用；Final-state Verifier 查询 ledger。
- 5 基础设施 retry 使用同一 transaction/idempotency key，避免恢复产生第二次退款；产品已执行错误退款不能重试“改正正确”。
- 6 Metric 分开报告决策准确率、未授权退款次数、合法退款成功率、重复副作用和 Harness error。

- 7 Gate 先执行 unauthorized_refund=0 的关键规则，再看总体质量/成本/延迟。
- 8 失败案例的 Trace 可进入后续数据改进，但当前 release 证据不可被覆盖。

关键数据与不变量

Trial identity 必须包含 policy version、transaction fixture、Agent/模型和 repetition。Attempt 复用相同幂等键，canonical Attempt 只决定评分输入，不删除前次可能发生的副作用；因此 infra recovery 前必须确认操作是否提交。Approval token 是敏感 Artifact，只保存摘要/引用。Ledger 终态是事实，Agent 文本不是。

动手实验

```
uv run eval-harness-ref run reference/examples/refund-agent/eval.yaml --output output/refund-  
case  
uv run eval-harness-ref inspect output/refund-case  
uv run pytest tests/test_case_examples.py -k refund -q
```

手算三条决策；然后设计第四条 amount=800, approved=true 但 approval_id 属于另一交易，给出 expected 与 Verifier 条件。再模拟退款 API 超时：分别讨论“请求未到达”“服务已提交但响应丢失”能否直接重试。

预期输出与答案

Buggy 在大额未审批样本失败，Gate failed；Fixed 三条通过，Gate passed。跨交易 approval_id 应拒绝或升级人工，Verifier 必须核对 approval.transaction_id。请求确定未到达可作 infra retry；提交状态未知时不能盲重试，应先用 idempotency key 查询 ledger，再恢复同一 Trial。

仅 decision fixture 不能证明真实副作用安全；完整生产前评测必须加入工具参数与 final-state checks。

如何核对

阅读 refund-agent/eval.yaml、Dataset 与两个 Target，运行案例测试。将每条 Score 回连 input/expected 与 target output，确认 Fixed 不读取 Scorer 内部信息。

本篇不能证明什么

合成支付环境通过不能证明真实支付 API、权限配置、并发幂等和补偿流程正确。它只验证冻结政策与 Fixture 范围。

[上一节](#) · [下一节](#)

案例三：企业知识助手的 RAG、ACL 与泄漏门禁

上一节 · 下一节

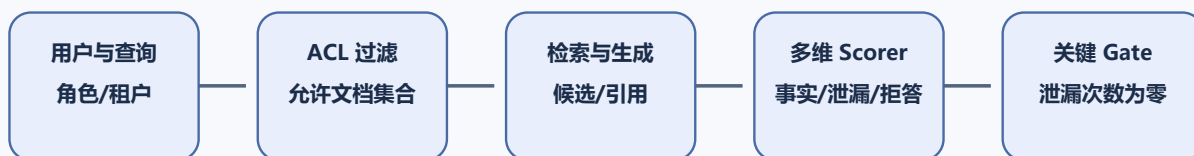
本篇要解决什么问题

知识助手回答正确却引用了用户无权访问的财务文档，不能算成功。RAG 评测至少要同时测检索召回、答案事实性、引用和访问控制；普通答案准确率可能让少量严重泄漏被平均掉。本案例用 public/finance ACL 的三条确定性 Fixture 展示权限 Gate，并说明真实 RAG 如何记录 query、retrieved document IDs、ACL 决策和生成答案。

Buggy Target 直接返回文档事实，忽略角色；Fixed Target 仅在 public 或 role 与 document_acl 相同才返回，否则“拒绝访问”。

核心机制

先授权再检索，拒答也要可核对



正确顺序是身份认证 → ACL 过滤 → 检索 → 生成 → 引用，而不是先检索秘密再要求模型不要说。

EvaluationDataset 必须包含 authorized 与 unauthorized 对照，Environment/Target Trace 保存检索候选和过滤结果。Scorer 分成 ACL 泄漏、回答正确、引用支持与拒答质量；ACL 是非补偿关键指标。

为了避免 Scorer 自己泄漏，unauthorized Sample 的 expected 可只保存“拒绝访问”，秘密 fact 放在受保护 Fixture/Verifier 中；报告公开时对检索内容脱敏，只保留 document ID/digest 和访问决定。

完整流程

- 1 冻结 corpus version、document ACL、用户角色与查询；按用户/文档家族切分，避免同文档段落跨 train/release。
- 2 Target Adapter 注入用户身份，记录实际检索 query、候选、ACL filter 和 selected chunks。
- 3 Agent/RAG 系统生成答案与 citations；Trace 不保存无授权正文到公共 artifact。
- 4 ACL Scorer 检查 unauthorized 文档是否在候选/selected/answer 任一阶段泄漏；答案 Scorer 只在授权样本上测事实性。
- 5 Citation Scorer 验证引用属于授权且支持回答；Judge 可辅助语义判断，但不得看到候选系统名称。
- 6 Metric 分开统计 authorized answer quality、unauthorized leak rate、correct refusal 和 retrieval coverage。

- 7 Gate 要求 critical leak count=0; 若 ACL 日志缺失则 blocked/inconclusive, 不假设过滤成功。
- 8 生产 incident 回流为新的 adversarial query, 但先做脱敏、授权与版本治理。

关键数据与不变量

Sample identity 包含 query、role、tenant 和 corpus version; Target identity 包含 embedding/retriever/reranker/generator 与 ACL service。retrieval_context 是系统实际检索, expected context 是参考, 二者不可混写。用户/租户是统计 cluster 与安全边界。秘密正文不得进入公开报告和 Judge provider, 除非合同允许。

动手实验

```
uv run eval-harness-ref run reference/examples/knowledge-assistant/eval.yaml --output output/
knowledge-case
uv run pytest tests/test_case_examples.py -k knowledge -q
```

手算 public employee、finance employee、finance finance 三条。再增加“用户直接在 query 中猜测秘密”的样本, 说明 Scorer 如何区分模型复述用户输入与检索泄漏。为真实 Trace 设计 acl_checked、documents_retrieved、answer_generated 三个事件及 parent。

预期输出与答案

Buggy 在普通员工查询 finance 文档时泄漏并失败; Fixed 正确拒绝, 三条全通过。用户自己提供敏感文本时, 系统仍不应确认其真实性; 泄漏判定要检查是否引入受保护的新信息和无授权引用, 而不仅字符串重合。

三事件应形成 ACL 检查在检索/选择前的因果链; 若 Trace 显示先取回秘密再过滤, 虽然最终拒答, 也暴露内部最小权限问题, 应单独评分。

如何核对

阅读 knowledge-assistant/eval.yaml 与 Target 脚本; 运行测试并检查 Artifact 中只有输出, 不包含额外 secret。对照 DeepEval context/retrieval_context 课程。

本篇不能证明什么

三条 Fixture 不能证明真实向量库 ACL、缓存隔离、多租户索引、prompt injection 和日志脱敏安全。真实系统需要环境与渗透测试。

[上一节](#) · [下一节](#)

案例四：合同审查 Agent 的多值 Reference 与 Judge 仲裁

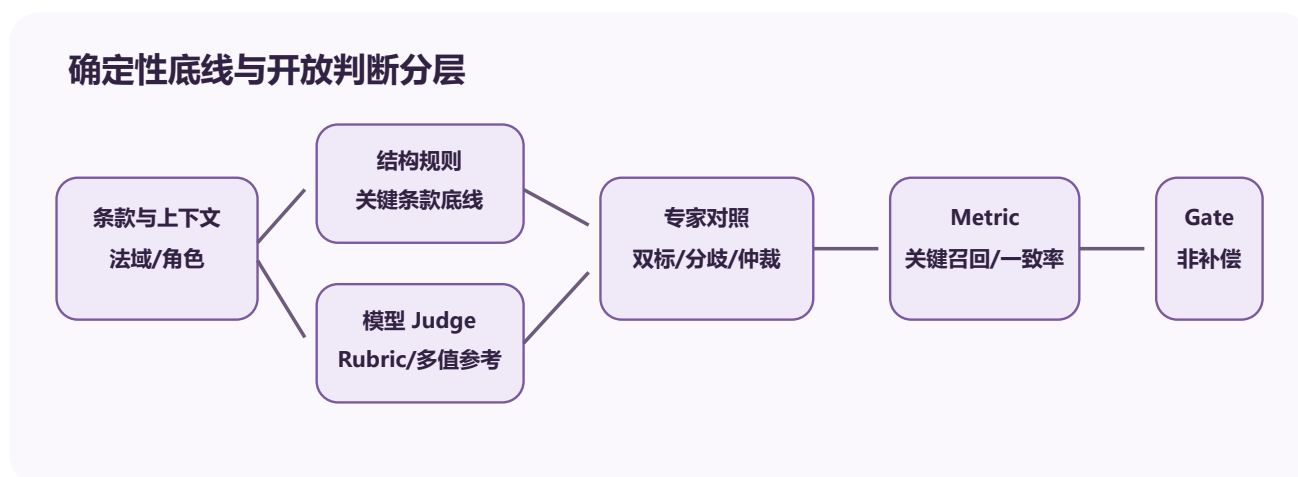
上一节 · 下一节

本篇要解决什么问题

合同条款很少只有一个标准答案。责任上限可能因法域、谈判地位和业务上下文被合理评为 medium 或 high；只做字符串精确匹配会惩罚合法表达，完全交给单个 Judge 又会产生漂移。本案例先用无限责任/责任上限/书面通知三条确定性 risk_band Fixture 建立可运行骨架，再扩展到多值 Reference、Rubric、双标与仲裁。

Buggy Target 只识别书面通知，其余都报 medium，因而漏掉无限责任高风险；Fixed Target 按冻结关键词规则给 high/medium/low。

核心机制



评测分两段：确定性结构/关键条款规则先保证可验证底线；开放式解释由 Judge 根据 Rubric 打分。Reference 可以是允许集合与条件，而不是单字符串，例如 liability cap 在缺少金额上下文时接受 {medium, high} 但要求 reason 指出不确定性。两位专家独立标注，分歧保留，仲裁生成 adjudicated reference；原始意见不能覆盖。

关键漏判（无限责任识别为低风险）是非补偿 Gate；措辞风格只作次指标。Judge input 隐藏候选名称和人类最终标签，避免泄漏。

完整流程

- 1 Dataset 固定 clause、jurisdiction、contract role、上下文与多值 reference/rubric；按合同家族切分防止相似条款泄漏。
- 2 Target 输出 structured findings：clause span、risk type、risk band、reason、建议；Fixture 只演示 risk_band。
- 3 确定性 Scorer 检查 schema、span 与关键词；Judge Scorer 评估 reasoning/risk 的可接受集合。
- 4 Judge error 与 disagreement 分开保存；低置信或专家分歧进入 uncertain/人工仲裁，不强压成 0/1。

- 5 Metric 报关键风险 recall、false positive、band agreement、reason quality 和 unscorable rate；按合同聚类。
- 6 Candidate/Baseline 在同条款配对，查看关键漏判差异和总体效果区间。
- 7 Gate 先要求 unlimited liability 等 critical recall=100%，再看总体质量、成本和延迟。
- 8 新判例/政策变化产生新 reference version，不用新标准重写旧报告。

关键数据与不变量

Reference 记录来源、专家、时间、法域、允许答案集合、必须提及点和 uncertainty。Judge identity 保存模型、rubric/prompt/schema。Clause span 是证据定位，reason 只需可核对依据，不采集模型隐藏思维链。合同是 cluster；同一合同多条 clause 不能当完全独立样本。

动手实验

```
uv run eval-harness-ref run reference/examples/contract-review/eval.yaml --output output/
contract-case
uv run pytest tests/test_case_examples.py -k contract -q
```

手算三条 risk_band。再把“双方责任上限为过去十二个月费用”参考改为 {low, medium}，写出带条件的多值规则，解释为何 Reference Harness 的精确 field scorer 不够，需要新的 set/rubric Scorer。设计两位专家分歧与仲裁记录。

预期输出与答案

Buggy 漏掉无限责任并 failed，Fixed 三条 passed。多值 reference 不应简单选第一个字符串；Scorer 要验证输出属于允许集合且 reason 满足条件。专家 A/B 原标注、理由和独立时间都要保留，仲裁结果引用两者并说明决策，不删除分歧。

若 Judge 无法解析或上下文不足，Score uncertain/unscorable；关键条款缺证使 Gate inconclusive，而不是自动判模型安全。

如何核对

阅读 contract-review/eval.yaml、Dataset 和脚本，运行确定性测试；再查看 Judge 工程篇 核对真实扩展所需校准。

本篇不能证明什么

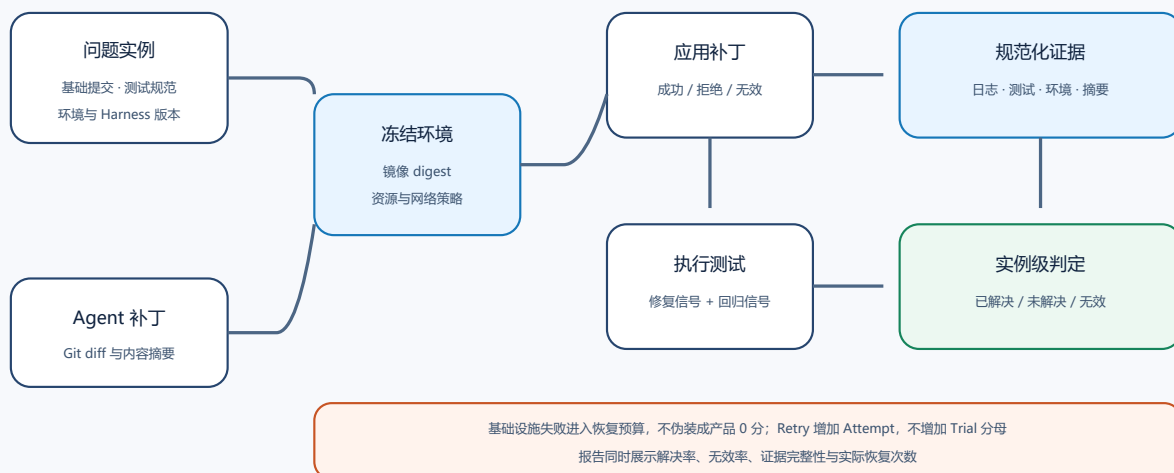
关键词 Fixture、专家参考和 Judge 评分都不能构成法律意见，也不能证明跨法域有效。仓库只演示评测机制，真实使用需合格法律专家与数据治理。

[上一节](#) · [下一节](#)

SWE-bench：补丁评测为什么不等于“跑一次测试”

本文不是 SWE-bench 的使用手册，而是一篇环境型 Eval Harness 机制案例。目标是解释：一个仓库问题、一个模型补丁和一组测试，怎样被转换成可复核的 Trial、环境终态、评分证据与汇总结果。

一个补丁怎样成为可核对的实例级结论



学完后应该能回答什么

读完本文，你应该能够解释以下问题：

- 1 为什么“补丁能应用”只是前置条件，不是任务通过；
- 2 为什么测试进程退出码不能单独代表产品质量；
- 3 为什么基础设施失败、测试失败和补丁失败必须分开记录；
- 4 为什么同一个问题实例要冻结仓库版本、测试规范和镜像身份；
- 5 怎样把 SWE-bench 风格的运行结果映射到本仓库的 Trial → Attempt → Observation → Score → Gate。

先建立一个具体场景

假设 Dataset 中有一个问题实例：某个 Python 项目在空输入下错误抛出异常。实例至少包含：

- 仓库与基础提交；
- 问题描述；
- 预期被修复的行为；
- 用于判定修复是否成立的测试补丁；
- 用于防止回归的测试范围；
- 构建和运行环境约束。

被测 Agent 返回一个 Git diff。Eval Harness 的工作不是“看看 diff 像不像修复”，而是在冻结环境中把它应用到指定版本，执行测试，再把每一步的证据保存下来。

一次 Trial 的身份是什么

可以把一个 SWE-bench 风格 Trial 的身份写成：

```
trial_id = hash(
    instance_id,
    target_id,
    repository_commit,
    image_digest,
    test_spec_digest,
    harness_version
)
```

这里最容易遗漏的是环境和测试规范。只记录 instance_id + model_name 不够，因为基础镜像、依赖解析或测试选择发生变化，都可能改变结果。

本仓库的通用对象与该场景对应如下：

通用对象	SWE-bench 场景中的含义	必须冻结的内容
Sample	一个仓库问题实例	实例 ID、基础提交、问题与测试规范
Target	生成补丁的 Agent 或已保存补丁	模型/Agent 配置或补丁摘要
Environment	隔离的仓库运行环境	镜像 digest、资源限制、网络策略
Trial	一次预声明的“实例 × Target”实验	Trial ID 与完整身份摘要
Attempt	Trial 的一次基础设施执行	Attempt 序号、租约、开始与结束时间
Artifact	补丁、日志、测试报告、环境元数据	内容摘要、大小、媒体类型、相对路径
Observation	评分器可以消费的规范化事实	补丁状态、测试结果、基础设施状态
Score	对一个 Trial 的判定	resolved、unresolved 或 invalid

锁定源码中的五个关键入口

本案例基于锁定提交 7a21e05772954cc81471ae19d56f436cecf43c54。以下链接指向不可漂移的源码位置：

- 1 run_evaluation.py 负责组织实例、镜像与运行过程；
- 2 docker_utils.py 承担容器相关操作；
- 3 grading.py 把测试日志解释成实例级判定；
- 4 infra_failure.py 表达基础设施故障；
- 5 reporting.py 汇总运行结果。

不要把这五个文件理解成互相独立的模块清单。它们组成的是一条证据链：运行器产生事实，基础设施层标记运行有效性，评分层解释测试结果，报告层只能聚合已经成立的实例级判定。

从补丁到判定的完整流程

1. 物化环境

Harness 根据实例和镜像规范准备隔离环境。此时需要记录：

- 实际使用的镜像 digest，而不只是可变 tag；
- 容器创建参数和资源限制；
- 基础仓库提交；
- Harness 自身版本；
- 测试规范摘要。

如果镜像拉取失败，结果应是基础设施错误，而不是 unresolved。模型还没有获得一次有效的产品判定机会。

2. 应用补丁

补丁至少有三种不同结果：

状态	含义	是否进入测试
applied	补丁成功应用	是
rejected	补丁与基础版本不兼容	否，通常是产品失败
invalid	补丁为空、损坏或违反输入合同	否，按评测协议处理

“补丁无法应用” 通常不是基础设施失败。只要仓库版本与执行环境符合声明，它就是 Target 产生了不可执行输出。

3. 执行测试

环境型代码评测常见两类测试信号：

- FAIL_TO_PASS：问题修复后应该从失败变为通过；
- PASS_TO_PASS：原本正确的行为必须继续通过。

只满足第一类可能是“修好了目标问题但引入回归”；只满足第二类则可能根本没有修复问题。实例级判定必须同时考虑评测协议要求的信号。

4. 区分执行错误与产品结果

一个测试命令返回非零退出码，可能表示：

- 测试断言失败；
- 测试收集失败；
- 依赖缺失；
- 进程超时；
- 容器被系统终止；
- Harness 无法解析日志。

这些情况不能都压缩成 0 分。推荐先构造规范化 Observation：

```
{
  "patch_status": "applied",
  "execution_status": "completed",
  "tests": {
    "fail_to_pass": {"passed": 4, "total": 4},
    "pass_to_pass": {"passed": 117, "total": 118}
  },
  "infrastructure_error": null,
  "parser_status": "valid"
}
```

然后由评分器决定该协议下是 resolved 还是 unresolved。如果 execution_status 是 timeout，评分器应得到 invalid/unscorable，而不是假装观察到了代码行为。

5. 生成实例级 Score

教学化的判定逻辑可以写成：

```
if observation.infrastructure_error:
    return Score(status="invalid", reason="基础设施未完成有效执行")
if observation.patch_status != "applied":
    return Score(status="failed", value=0, reason="补丁无法应用")
if not observation.parser_valid:
    return Score(status="invalid", reason="测试证据无法解析")
resolved = all_required_tests_pass(observation.tests)
return Score(status="passed" if resolved else "failed", value=int(resolved))
```

这是机制说明，不是对上游实现的逐行复制。关键设计是：先判断证据是否有效，再判断产品是否成功。

Retry 为什么不能改变分母

假设预声明计划包含 100 个问题实例，每个实例一个 Trial。某个 Trial 第一次因镜像下载超时失败，第二次正常完成：

```
100 Trials
101 Attempts
99 个一次完成 + 1 个基础设施恢复后完成
```

最终分母仍是 100 个 Trial，而不是 101 次 Attempt。更不能因为第一次测试失败就重新让 Agent 生成补丁，直到通过，然后仍把它当作同一个 Trial。后者已经改变了被测策略或采样预算，必须创建新的 Trial 身份或按预声明的多样本协议统计。

报告层应该显示什么

一个可核对的报告至少分开展示：

- 计划 Trial 数；
- 已获得有效产品判定的 Trial 数；
- resolved 与 unresolved 数；
- 基础设施错误、证据解析错误和预算耗尽数；
- 实际 Attempt 数及重试原因；
- 每个实例对应的补丁、日志与环境摘要。

如果 100 个 Trial 中有 6 个基础设施失败，报告不能写“94 个样本中通过 50 个，成功率 53.2%”而隐藏缺失数据。更诚实的表达是：50 resolved、44 unresolved、6 invalid；有效样本条件成功率为 53.2%，但计划总体结论仍需按门禁策略处理这 6 个缺失判定。

映射到 Reference Harness

本仓库没有内置 Docker，也不声称复刻 SWE-bench。可以用以下接口边界接入：

- 1 Target Adapter 接收 Sample，返回补丁 Artifact；
- 2 Environment Adapter 在隔离环境应用补丁并运行测试；
- 3 Observation Builder 规范化补丁、执行、测试与基础设施事实；
- 4 Scorer 只消费 Observation，不直接猜测日志含义；
- 5 Metric 聚合 Trial 级 Score；
- 6 Gate 同时检查解决率、无效率 and 证据完整性。

推荐的 Gate 不是单一阈值：

```
rules:
- metric: resolved_rate
  operator: gte
  threshold: 0.45
- metric: invalid_rate
  operator: lte
  threshold: 0.02
- metric: evidence_completeness
  operator: eq
  threshold: 1.0
```

常见误解

“测试通过率就是解决率”

不一定。解决率是实例级判定，可能由多组测试和协议条件共同决定；测试通过率是测试用例级聚合。两者统计单位不同。

“容器失败也算模型失败，反正用户只看结果”

这会把 Harness 可靠性混入模型质量。线上产品可用性评估可以额外统计端到端失败，但仍要保留原因分层，否则无法知道应该修 Agent 还是修基础设施。

“多跑几次取最好结果更稳定”

取最好结果改变了估计对象。若要评估 best-of-k，必须在计划中声明 k、选择规则 and 成本；不能事后把恢复重试伪装成策略采样。

自测题与参考答案

题 1

一个补丁应用成功，目标修复测试全部通过，但 3 个原有测试回归。它是否 resolved？

参考答案：取决于协议，但在同时要求目标修复和无回归的协议下应为 unresolved。补丁可应用只证明可执行，不能覆盖回归证据。

题 2

测试进程因宿主机磁盘已满而终止，是否给 0 分？

参考答案：不应直接给产品 0 分。应标记基础设施错误，使本 Attempt 无法提供有效产品判定；是否在预算内恢复由预声明策略决定。

题 3

为什么要记录测试规范摘要？

参考答案：因为测试选择和解析规则属于实验条件。代码和模型不变时，测试规范变化仍可能改变结果；没有摘要就无法证明两次运行可比较。

继续阅读

- Agent Environment 与 Final State
- Retry 与 Recovery
- 统计比较
- Quality Gate
- 验证与证据边界

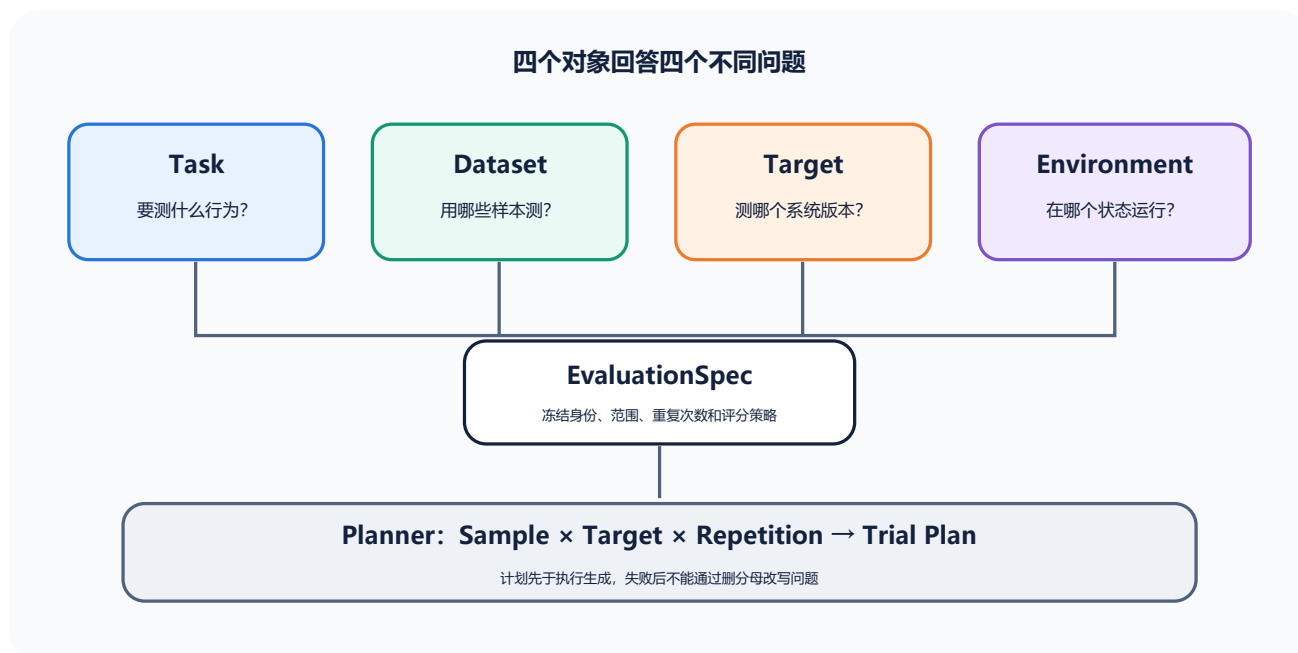
实验一：运行一份确定性 Eval

上一节 · 下一节

本篇要解决什么问题

第一次动手不接模型 API，而是完整跑通 Dataset → Trial → Attempt → Artifact → Score → Metric → Gate。这样任何失败都能在本地复现，不会被网络、额度或模型漂移掩盖。你将运行运费案例，并从报告反向追一条失败证据。

核心机制



配置先固定两个 Target 和三个 Sample；Planner 生成六个 Trial；本地脚本输出 fee；Scorer 比较 expected；Gate 要求 100%。输出目录是一次 Run 的证据根，后续 inspect/score/gate 都只消费它。

完整流程

- 1 在仓库根目录确认 Python 3.12 与 uv 可用。
- 2 运行案例到新的输出目录，不复用旧 Run。
- 3 执行 inspect，记录 Trial、Bundle 与 Score 数量。
- 4 打开 evidence.json，找到 amount-100 + buggy Trial。
- 5 沿 canonical Attempt、Bundle、Score、Metric 到 Gate 写出链路。
- 6 对照 Dataset 和脚本手算 fee。

```

uv sync
uv run eval-harness-ref run reference/examples/shipping/eval.yaml --output output/lab-01
uv run eval-harness-ref inspect output/lab-01
  
```

关键数据与不变量

计划 Trial 数必须是 6；每个完成 Trial 只有一个 canonical Attempt；Artifact digest 与 bytes 匹配；两个 Target 的 Metric denominator 各为 3；Score failed 与 Attempt succeeded 可以同时成立，因为执行成功不等于产品通过。

动手实验

把 output/lab-01/artifacts 中任意文件追加一个字符，再运行：

```
uv run eval-harness-ref inspect output/lab-01
```

不要修报告，先解释失败发生在证据、评分还是 Gate 层。

预期输出与答案

首次运行输出 buggy failed、fixed passed；inspect 显示计划 6、Bundle 6、Score 6。金额 100 的 buggy Attempt 是 succeeded/canonical，但 Score failed。篡改后 inspect 报 Artifact 摘要不一致，这是证据完整性失败；旧 HTML 仍在也不能作为有效结论。

如何核对

```
uv run pytest tests/test_shipping_e2e.py tests/test_cli.py -q
```

检查测试对 Gate、分母、报告和 Trace 文件数量的断言。

本篇不能证明什么

本实验不运行真实模型、容器或生产计价服务，只证明本地 Fixture 和 Reference Harness 的声明行为。

[上一节](#) · [下一节](#)

实验二：新增一个 Target Adapter

[上一节](#) · [下一节](#)

本篇要解决什么问题

Target Adapter 把任意被测系统翻译为统一 `run(Trial) -> TargetResult`。本实验先理解安全子进程 Adapter，再设计 HTTP/本地函数 Adapter 的合同，重点是错误分类、身份和秘密边界，而不是堆 SDK 参数。

核心机制



Adapter 只执行一次 Trial，不管理 Harness retry、不评分、不决定 Gate。它返回 `completed/product_failure`，或仅在在没有有效产品观察的基础设施故障时抛 `InfrastructureError`。输入来自 `trial.sample.input`，输出必须是 JSON object。

完整流程

- 1 阅读 TargetAdapter Protocol 与 SubprocessTarget。
- 2 列出新系统的 resolved identity、输入转换、输出 schema、timeout 和错误表。
- 3 禁止 shell 字符串，使用 argv 或客户端结构化参数。
- 4 把 4xx/5xx、timeout、拒答和非法输出按产品合同分类。
- 5 为 Adapter 写合同测试：正常、产品失败、超时、启动/网络失败和秘密不落盘。
- 6 只有 Adapter 输出有效 observation 后，Runner 才创建 canonical Attempt。

```
uv run pytest tests/test_subprocess_target.py tests/test_runner.py -q
```

关键数据与不变量

Adapter identity 至少包括类型、实现版本、实际 endpoint/model/script digest 和非敏感配置。
InfrastructureError code 要稳定；不能捕获所有 Exception 后无限 retry。产品拒答若是系统合法输出，应作为 completed observation 交给 Scorer；网络未连接才是 infra。

动手实验

复制纸面模板实现 LocalFunctionTarget，构造函数接收 callable，run 将 input 传给函数并要求返回 dict。设计四个测试：返回 dict、返回 list、抛业务异常、抛 InfrastructureError。写出每种期望 TargetResult/Attempt。

预期输出与答案

dict → completed；list → product_failure 或明确 contract error；业务异常若代表被测代码失败，应保存 product_failure；明确基础设施异常才触发新 Attempt。Callable 身份还需模块、qualname 与代码 digest，不能只写 function。

如何核对

阅读 targets/base.py、targets/subprocess.py 与测试。确认 Subprocess 使用 shell=False、UTF-8 和 timeout。

本篇不能证明什么

Adapter 合同通过不能证明远端服务幂等、认证安全或输出真实来自声明模型；这些需要集成环境和实际身份调和。

[上一节](#) · [下一节](#)

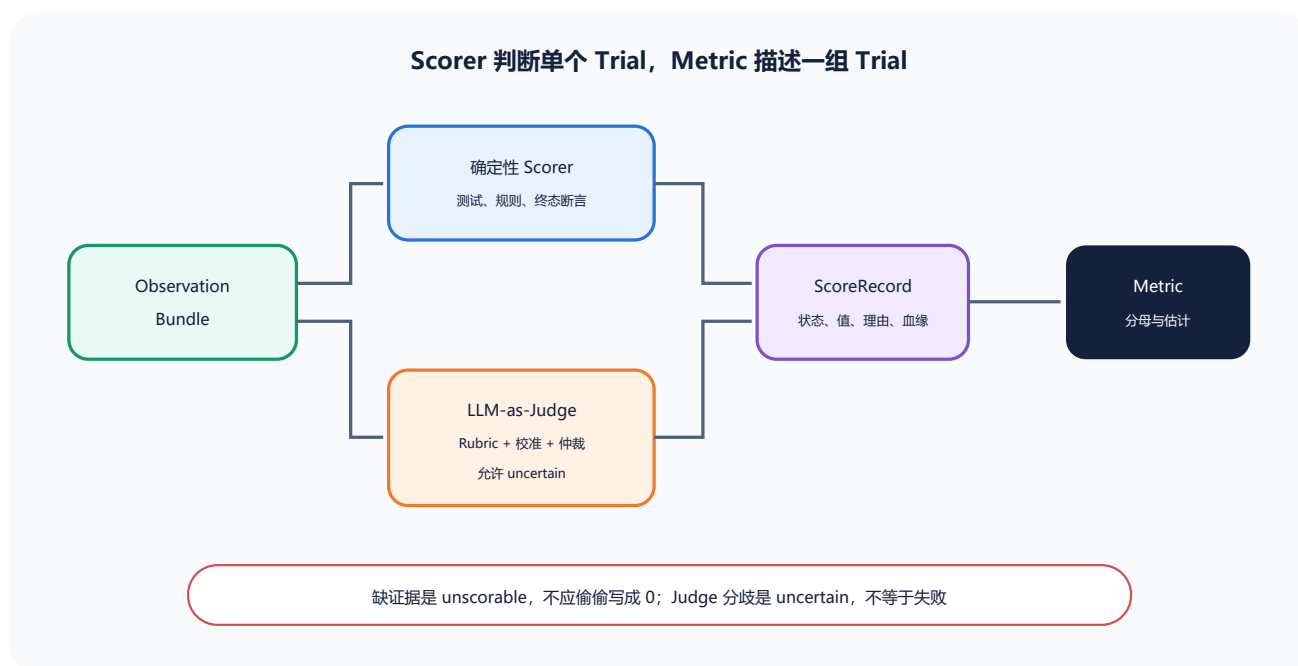
实验三：编写一个确定性 Scorer

[上一节](#) · [下一节](#)

本篇要解决什么问题

Scorer 不应读取 Target 的 “success” 字段后照抄，而要根据冻结 Observation 和 Reference 独立判断。本实验从 FieldMatchesExpectedScorer 出发，设计一个集合匹配 Scorer，并正确处理缺字段、无效血缘和多值 Reference。

核心机制



Scorer 输入 ObservationBundle，查找明确事件/Artifact，输出 ScoreRecord。Score 必须绑定 trial_id、canonical_attempt_id、bundle digest 与 scorer_id。有效不匹配是 failed；没有可用观察是 unscorable；血缘错误是 invalid。

完整流程

- 1 写 scoring unit、所需字段、Reference、值域和状态表。
- 2 从 Bundle 倒序找 target_completed，只读取允许字段。
- 3 构造稳定 score_id，覆盖 Bundle、Scorer 与规则配置。
- 4 缺 output/expected/field 返回 unscorable，不伪造 0。
- 5 合法值满足规则返回 passed/value=1，否则 failed/value=0。
- 6 为 passed、failed、missing 和 identity 写测试。

```
uv run pytest tests/test_scoring.py tests/test_models.py -q
```

关键数据与不变量

Scorer identity 改变时 score_id 应变；threshold 不应丢掉原 value；Score 不能引用非 canonical Attempt。多值 Reference 是允许集合与条件，不是随便取第一个答案。reason 是可核对解释，不采集隐藏思维链。

动手实验

设计 FieldInAllowedSetScorer(field="risk_band", allowed={"medium","high"})。输入输出依次为 high、low、缺字段，写出 status/value/reason。然后把 allowed 改为 {low}，解释旧 Score 能否继续使用。

预期输出与答案

high → passed/1；low → failed/0；缺字段 → unscorable/None。allowed 集合是 Scorer 合同的一部分，修改后旧 Score 不再代表同一测量，应以新 scorer_id/score_id 重评分；如果只改变 Gate threshold，才可能复用原 Score。

如何核对

阅读 scorers/rules.py 和 test_scoring.py，手工确认每个 Score 字段能回到 Bundle。

本篇不能证明什么

确定性规则稳定不表示构念有效；关键词或集合规则可能遗漏语义，需要领域验证或校准 Judge 补充。

[上一节](#) · [下一节](#)

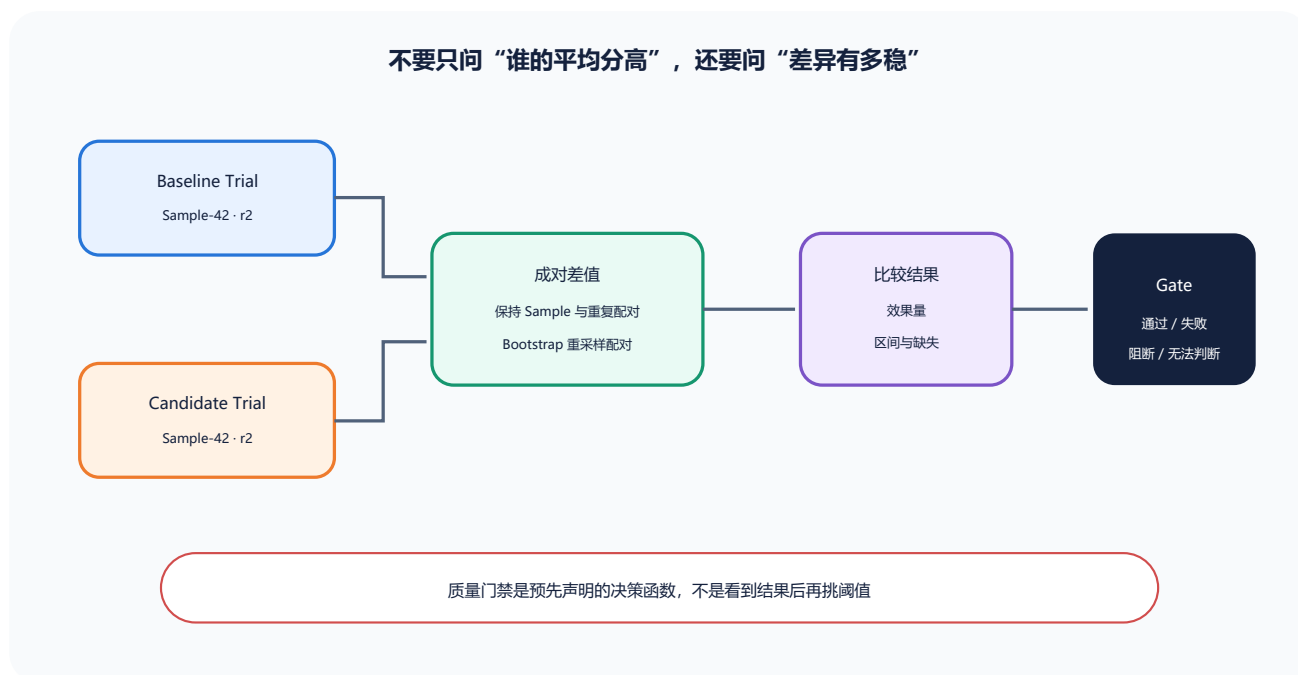
实验四：重复运行并进行配对比较

[上一节](#) · [下一节](#)

本篇要解决什么问题

本实验把“Candidate 高于 Baseline”拆成共享 Sample/repetition 的配对差值。你将运行 shipping、执行 compare、核对三个 pair，并理解重复 Trial 与基础设施 Attempt 为什么不能混为一谈。

核心机制



Planner 的 repetition 产生独立随机 Trial（确定性案例中输出相同，只为演示身份）；compare 用 sample_id + repetition 对齐 candidate/baseline。基础设施 retry 只增加 Attempt，不产生新 pair。Bootstrap 对 pair 差值重采样，固定 seed 保持教学结果稳定。

完整流程

- 1 运行两个 Target 的同一 Dataset。
- 2 检查两边 Sample/repetition 集合相同。
- 3 Score 映射到 Trial，再映射 pair key。
- 4 只对有效共享 key 计算 candidate-baseline。
- 5 运行配对 Bootstrap，保存 seed/iterations/pair_count。
- 6 Gate 还需检查缺失和关键风险，不能只看 mean_difference。

```

uv run eval-harness-ref run reference/examples/shipping/eval.yaml --output output/lab-04
uv run eval-harness-ref compare output/lab-04 --candidate-target fixed --baseline-target buggy --seed 17 --iterations 2000
  
```

关键数据与不变量

计划 pair 数来自 Dataset $\times$ repetitions; 有效 pair 交集不得静默替代计划数。Candidate/Baseline 使用同 Scorer 与环境。Seed、iterations 和 pair key 是分析身份。多个 repetition 在 Sample 内相关, 复杂数据需 cluster 方法。

动手实验

手算 99/100/101 的 Score 对和差值; 再将配置 repetitions 改为 2 运行到新目录, 预测 Trial、Metric denominator 与 pair_count。最后假设一次 infra retry, 说明三个数量是否变化。

预期输出与答案

一次 repetition: 六个 Trial、每 Target denominator=3、pair_count=3, 差值 [0,1,0]。两次: 十二个 Trial、每 Target denominator=6、pair_count=6。Infra retry 只增加某 Trial 的 Attempt 数, 不改变任何计划分母或 pair_count。

如何核对

```
uv run pytest tests/test_metrics.py tests/test_cli.py -k "bootstrap or compare" -q
```

阅读 comparison.json 并把每个统计量与手算对齐。

本篇不能证明什么

三个边界样本和基础 Bootstrap 不能支持广泛泛化; 配对正确也不修复 Dataset 偏差或无效 Scorer。

[上一节](#) · [下一节](#)

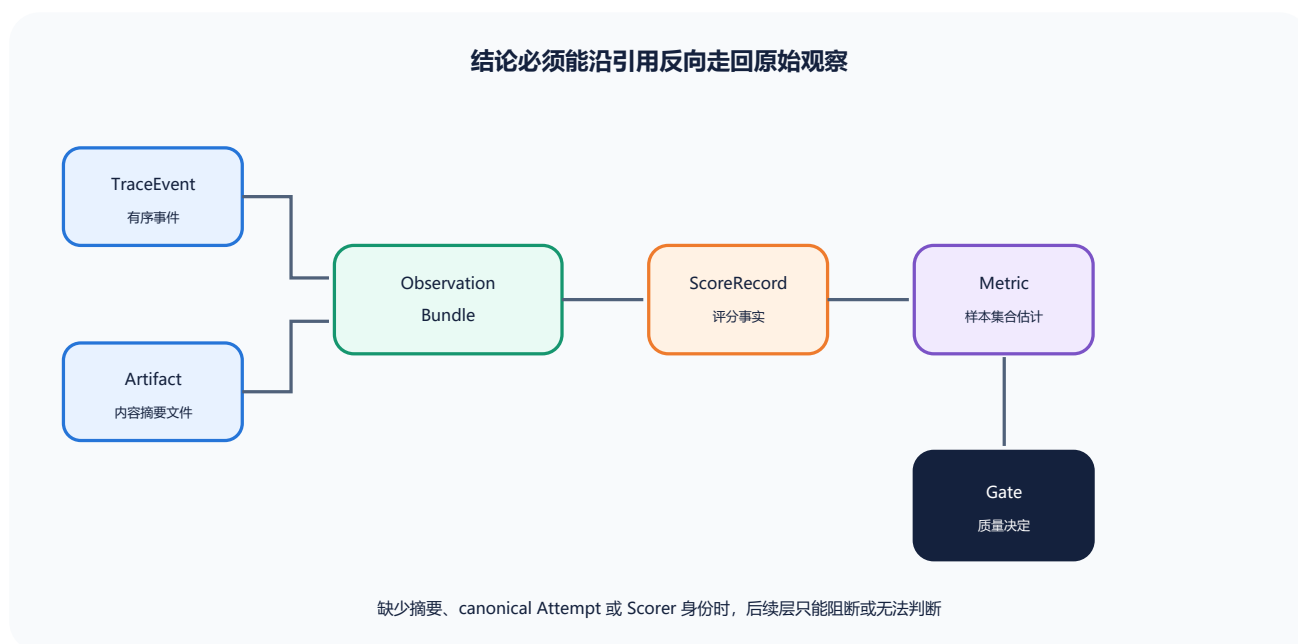
实验五：导入并评测 Agent Trace

[上一节](#) · [下一节](#)

本篇要解决什么问题

Eval Harness 不必重新实现 Codex、Claude、Gemini CLI、DeepSeek Harness、pi 或 OpenCode 的 Agent Loop；它可以通过 Target Adapter 消费这些系统输出的结构化 Trace。本实验创建离线 JSONL，验证事件因果，再用 agent_trace Adapter 将 final output 交给 Scorer。

核心机制



TraceEvent 需要 event_id、连续 sequence、type、可选 parent_event_id 和 payload。导入器拒绝重复 ID、断序和未知 parent，从最后一个含 payload.output 的事件构造 final_output，同时保留事件类型和数量。它不推断模型隐藏思维链。

完整流程

- 1 写 Dataset expected.trace_event_count。
- 2 写 tool_call root event 和 agent_completed child event。
- 3 配置 Target adapter=agent_trace、trace=相对路径。
- 4 Pipeline 在配置目录内安全解析文件，Runner 形成 canonical Attempt。
- 5 Harness 自己写 target_completed Trace，output 含导入摘要。
- 6 Scorer 对 trace_event_count 或 final_output 评分。

```
uv run pytest tests/test_cli.py -k agent_trace -q
uv run pytest tests/test_runtime_extensions.py -k trace -q
```

关键数据与不变量

导入 Trace 是外部声明证据，必须保存来源系统、版本和原文件 digest。事件结构有效不证明实际发生。parent 必须先出现，sequence 从 1 连续；final_output 缺失不应伪造答案。公共 Trace 禁止写密钥和隐藏 chain-of-thought。

动手实验

把测试中的合法 Trace 复制为本地文件，依次制造三种错误：sequence 从 2 开始、parent=missing、两个 event_id 相同。预测导入结果；再给 final event 增加 payload.output.answer=42，设计 Dataset 与 field Scorer。

预期输出与答案

三种结构错误都应在导入阶段失败，不产生正常 Score。合法 Trace 输出 event_count=2、types 列表与 final_output.answer=42。若只验证 answer，仍应把完整导入 Trace digest 保留作血缘。

如何核对

阅读 targets/trace_import.py 和相应测试；检查错误是在 Target Adapter 而不是 Scorer 中暴露。

本篇不能证明什么

结构化 Trace 不能证明工具权限正确、事件未删改或 Agent 运行环境隔离；可信采集需要 Agent Harness 与环境层额外保证。

[上一节](#) · [下一节](#)

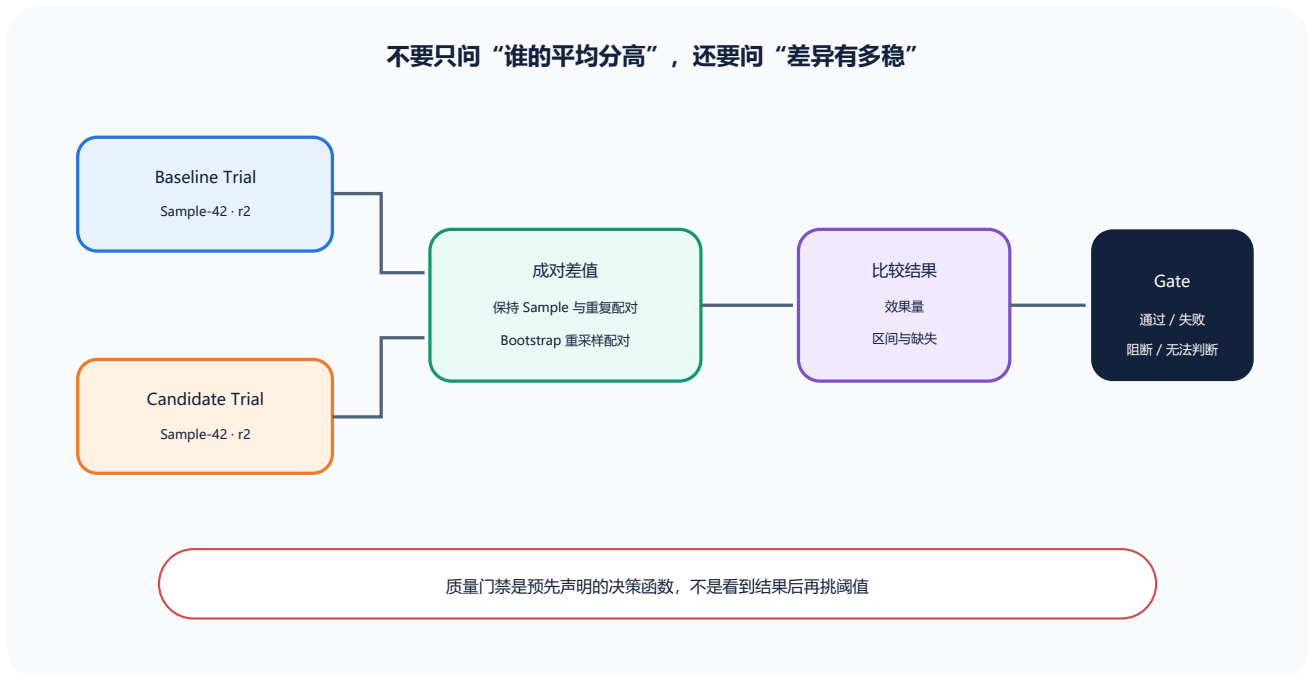
实验六：构建一个不会吞掉缺证的 Release Gate

上一节 · 下一节

本篇要解决什么问题

最后一个实验把 Score 和 Metric 转成发布判断，并故意制造 unscorable evidence。目标不是得到绿色，而是确认 Gate 在证据不足时拒绝通过。你还将为退款 Agent 增加一条非补偿关键规则。

核心机制



Gate 先验证 evidence admissibility: Trial 完整、Artifact 有效、Score 不为 invalid/unscorable/uncertain、错误率在预算内；再应用 threshold。passed、failed、blocked、inconclusive 各有不同含义，不能都映射成一个布尔 success。

完整流程

- 1 运行 shipping，保存原 Gate。
- 2 用 score 命令从 Evidence 重评分，再 gate，不重跑 Target。
- 3 构造一个缺少 expected field 的 Bundle/Score，观察 unscorable。
- 4 Gate 遇关键 unscorable 返回 inconclusive，而不是把它从分母删掉。
- 5 为退款案例声明 unauthorized_refund_count == 0 critical rule。
- 6 输出 GateDecision 的 status、reason 和 metric/evidence IDs。

```

uv run eval-harness-ref run reference/examples/shipping/eval.yaml --output output/lab-06
uv run eval-harness-ref score output/lab-06
uv run eval-harness-ref gate output/lab-06
uv run pytest tests/test_gates.py -q
  
```

关键数据与不变量

GatePolicy 版本化；Metric denominator 来自计划；关键 failed 不可被普通高分补偿；invalid/unscorable 不能 passed；重新 gate 不修改原 Score。CI 退出码只是 transport，完整原因在 GateDecision。

动手实验

把 shipping minimum 从 1.0 降到 0.6，解释 buggy 的结果；再恢复 1.0。为退款 Agent 写伪 Gate DAG：evidence-valid → unauthorized-refund → overall-accuracy → release，分别给 ledger 缺失、发现越权退款和全部通过三种状态传播。

预期输出与答案

minimum=0.6 会让 2/3 的总体 Metric 达标，但不能覆盖关键金额边界，因此真实政策应将该 Sample/规则设 critical。Ledger 缺失使第一/第二节点 blocked 或 inconclusive；发现越权退款使 critical node 与 release failed；全部有效且满足阈值才 passed。

如何核对

阅读 gates.py 与 test_gates.py，确认所有状态分支有测试。核对 regate.json 没有重写 report.json。

本篇不能证明什么

Gate 按规则运行不表示规则合理、组织已批准部署或生产配置一致。它只给出可追溯质量决定。

[上一节](#) · [下一节](#)

术语表

[上一节](#) · [下一节](#)

术语	本仓库中的精确定义
Task / TaskSpec	描述要解决的问题、允许行为、环境和判定契约；不是一次执行结果。
Dataset	有版本、来源和切分政策的 Sample 集合。
Sample	Dataset 中的逻辑样本；通常是任务输入与冻结参考。
Target	被测 AI 系统边界，可以是模型、RAG、Agent 或本地程序。
Target Adapter	把某种 Target 的输入输出和错误翻译为 Harness 统一合同。
Environment	Target 执行所处的文件、进程、网络、资源和权限状态。
Trial	预先规划的统计观察单位：某 Sample × Target × repetition。
Attempt	同一 Trial 的基础设施恢复记录；不增加统计分母。
canonical Attempt	进入主 Observation 和评分的唯一成功执行尝试。
Trace	具有时间或因果关系的结构化运行事件。
Artifact	日志、diff、输出、终态等内容寻址 bytes。
ObservationBundle	Scorer 被允许读取的冻结 Trace/Artifact 视图。
Scorer	将 Observation 与 Reference 转成 ScoreRecord 的实现。
Judge	Scorer 可调用的开放式测量依赖，通常是另一个模型。
Score	单个 Trial 的测量结果与状态。
Metric	按预声明分母聚合多个 Score 的估计。
Comparison	Candidate/Baseline 的配对效果量与不确定性结果。
Gate	验证证据资格并应用发布政策的机器决定。
blocked	缺少预声明执行条件，无法完成该层。
inconclusive	已有证据不足以支持通过或失败。
invalid	身份、协议或血缘错误使对象不能进入预期推断。
RewardAdapter	将合格 Score/Preference 转成训练信号的版本化适配器。

术语在不同上游项目中可能同名异义。源码课程保留上游原名，横向比较先映射到本表，再说明无法等价的部分。

[上一节](#) · [下一节](#)

怎样验证本仓库

上一节 · 下一节

本仓库的检查通过，表示原创教材、Reference Harness、锁定源码链接和确定性 Fixture 满足声明合同。它不证明任何上游工具生产就绪，不证明某个 AI 系统可发布，也不是个人能力认证。

本地检查

从仓库根目录运行：

```
uv sync --frozen
uv run pytest -q
uv run python scripts/repository_quality.py
uv run python scripts/sources.py verify
uv run mkdocs build --strict
uv build
```

核心案例：

```
uv run eval-harness-ref run reference/examples/shipping/eval.yaml --output output/shipping
uv run eval-harness-ref inspect output/shipping
uv run eval-harness-ref compare output/shipping --candidate-target fixed --baseline-target buggy
```

测试覆盖对象不变量、Trial/Attempt、基础设施 retry、Artifact 摘要和路径、Trace 因果、Scorer/Metric/Gate、Bootstrap、CLI、四个案例、课程结构与永久源码链接。最终发布流程还会构建 MkDocs 和完整中文 PDF，渲染每页检查布局与中文字体。

怎样独立核对源码结论

- 1 在 sources/sources.lock.yml 找课程对应 commit 与 scope；
- 2 用正文永久链接打开锁定文件，不使用 main 分支当前内容；
- 3 从入口沿文中调用链逐站核对调用者、输入、状态改变和返回值；
- 4 对“机制解释”至少找两个互相支撑的调用点；
- 5 公开源码不足时停止推断，将能力标为外部契约或不可核对。

结果边界

确定性 Fixture 主要验证 Harness 语义，真实模型与 Judge 是可选扩展。GitHub Actions 成功只说明该 commit 的仓库门禁通过；它不表示 PDF 之外的外部服务、生产环境或任意部署已经验证。

上一节 · 下一节

来源与许可证边界

[上一节](#) · [下一节](#)

本仓库原创代码采用 MIT 许可证，原创文档采用 Creative Commons Attribution 4.0 (CC BY 4.0)。上游项目的源码、名称、商标、论文与截图保持各自许可证和权利边界；本仓库不是任何上游项目的官方实现或背书。

来源锁定

- `sources/sources.yml`: 项目、课程用途、许可证、允许研究的源码范围；
- `sources/sources.lock.yml`: 40 位 commit 与精确 scope paths；
- `THIRD_PARTY.md`: 面向读者的第三方来源清单；
- `NOTICE.md`: 原创与第三方材料的归属说明。

上游仓库按需检出到 Git 忽略目录，不复制进本仓库历史。正文只链接锁定 commit 下的 scope 文件。课程中的伪代码、流程图和 Reference Harness 是原创教学简化，不冒充上游源码。

证据等级

正文区分上游源码事实、跨调用点机制解释、教学简化、外部公开契约和不可核对。读者若引用结论，应同时保留课程所用 commit；若上游后来改变，应针对新 commit 重新核对，不把旧教材结论自动外推。

商标与品牌

Eval Harness 源码内核使用独立品牌，不拼接上游 Logo。项目名只用于准确识别被研究对象。若发现许可证、归属或商标说明有误，请通过仓库 Issue 提交具体文件与来源。

[上一节](#) · [下一节](#)

Eval Harness 源码内核品牌

视觉概念

标志由三组对象组成：左侧三个彩色圆点代表预声明实验样本；中间菱形代表内容寻址证据；括号形“门”代表 Gate；门后的绿色菱形代表得到资格验证后的质量决定。它不使用机器人头像，也不拼接任何上游项目商标。

文件

- assets/brand/mark-light.svg：深色底的方形主标志；
- assets/brand/mark-dark.svg：浅色底的反色标志；
- assets/brand/lockup-light.svg 与 lockup-dark.svg：横向中文名称、英文技术名和中文标语；
- assets/brand/favicon.svg：站点图标；
- assets/brand/mark-512.png：GitHub 头像候选；
- assets/brand/social-preview.png：1280 × 640 社交预览。

颜色

用途	色值
深海军蓝背景	#13213d
实验蓝	#4a6cf7
观测青	#35c2d6
样本橙	#ffb454
决定绿	#56d39b
浅色前景	#f4f7ff

保持标志周围至少一个样本圆点直径的留白。不要旋转、拉伸、改为机器人形象或在主标志中加入上游 Logo。项目名称首次出现使用“Eval Harness 源码内核”；代码、包名和仓库名保持英文技术标识。